

Requirements

Important! Products created with this SDK can run in Reason 10.1 and later.

Jukebox SDK System Requirements

macOS

- Internet connection for uploading Rack Extensions to Reason Studios build server
- 4 GB RAM or more (8 GB recommended)
- 15 GB free hard disk space
- macOS 10.15 or later
- Monitor with at least 1280×768 resolution
- 1 GB NVidia or ATI/AMD graphics card recommended
- Core Audio compliant audio interface or built-in audio hardware
- MIDI interface and a MIDI keyboard recommended

Windows

- Internet connection for uploading Rack Extensions to Reason Studios build server
- Intel or AMD processor with dual cores
- 4 GB RAM or more (8 GB recommended)
- 15 GB free hard disk space
- Windows 10 or later (64 bits only)
- Monitor with at least 1280×768 resolution
- 1 GB NVidia or ATI/AMD graphics card recommended
- Audio Interface with ASIO driver
- MIDI interface and a MIDI keyboard recommended

Where to get help

Online Forum

<https://developer-support.reasonstudios.com/hc/en-us/community/topics>

If you cannot find the answer to your question on the developer forum, do not hesitate to contact us:

<https://developer-support.reasonstudios.com/hc/en-us/requests/new>.

Getting started

The Jukebox SDK contains three major components:

Reason Recon - The development version of Reason for testing locally built Rack Extensions, based on Reason Intro. This version of Reason Recon is distributed as a beta build, which means that it includes test code, asserts and extensive logging.

On Mac, Recon can be started in Apple or Intel mode, using "Open using Rosetta" option in macOS app settings or the *arch* prefix command. See [6. Compile a test project](#).

Tip!

SDK - Header files, compiler toolchain and example projects.

RE2DRender - A tool for compiling hi-res 2D GUI definitions into a format suitable for Reason.

To get started, do the following:

1. Read the Rack Extensions Developer's Guide document

If you haven't already, spend a little time reading through the *Rack Extensions Developer's Guide*. This will give you an essential overview of the Rack Extensions system, which really helps understanding the different concepts and terms that will be introduced when getting started with Rack Extensions development.

2. Install Reason

It is assumed that a full version of Reason is installed on your computer.

3. Install Reason Recon

The installer can be found under the "Reason Recon" directory.

4. Ensure python 3 is installed.

The build scripts use python 3. We recommend / support python.org (<http://www.python.org/download/>) version 3.10 or later. (It might work with other python 3 versions as well, but in case of unexpected results with the SDK/example build scripts, please try the supported version of python as a first stage of troubleshooting.)

Make sure python is in your PATH. Open command prompt / Terminal and try `python --version`. Check that it's the expected version. If not, try `python3 --version` instead.

Replace python in the example command lines with the name used to launch python3 in your setup (python / python3 / py -3, other ways to launch).

5. Documentation and example projects

SDK documentation can be found on <https://developer.reasonstudios.com/>.

A number of example projects can be found in the `SDK/Examples` folder.

6. Compile a test project

You should now be ready to try the Jukebox APIs as well as getting to know the GUI design process!

The SDK example projects can be built from a command prompt. On Windows, open a Visual Studio Command Prompt, on macOS, open a Terminal window. Navigate to one of the example project folders, e.g.,

```
.../SDK/Examples/SimpleInstrument .../SDK/Examples/VerySimpleSampler
```

To build a testing version:


```
python build45.py local45 Testing
```

Run the `build45.py` script without parameters to see all options.

When building a local45, the code is always compiled for 64 bits. You can do all testing using the local45 version of your Rack Extension. Sidenote: Both 64 and 32 bits code is still built and uploaded as part of your universal45 archives.

Important!

On Mac, a local45 can be built for Apple and/or Intel CPU. The build scripts always target Intel, and target both CPUs if an Apple CPU is detected by default (see tip about LOCAL45_ARCHS below).

Important!

On Windows, the command line prompt that you use to invoke the python build script must be specific to compiling for 64 bits.

Tip!

`build45.py` contains a few options that can save compile time while iterating locally. `STATIC_ANALYSIS` can be set to `False` to skip automatic analysis / linting steps when compiling.

`LOCAL45_ARCHS` can be used on Mac to target Apple or Intel CPU only, instead of auto-detect. If you use `LOCAL45_ARCHS` to save compile time, make sure you start Reason Recon with the same mode / CPU type (i.e. Intel / Rosetta to test x64 build, Apple to test arm64 build - only local45s for matching CPU are found/available).

Built Rack Extensions are automatically put in:

Windows: `%APPDATA%\Propellerhead Software\RackExtensions_Dev`

macOS: `~/Library/Application Support/Propellerhead Software/RackExtensions_Dev`

Reason Recon will scan these folders for new Rack Extensions when it is restarted.

If you make changes to a Rack Extension that does not involve changing its name or its 2D GUI resources you can reload it from within Reason without quitting: remove all instances of the device from the rack, recompile the project, and create a new instance of the device.

Tip!

A simple pattern for testing is to create a device in the rack, connect it to other devices for testing (if needed), make changes to the source code, delete the device from the rack, rebuild it and then add it again in the rack by invoking "undo". This will recreate the device with all connections and settings intact. Please note however that undo uses the document / patch format, so will not work if the changes break compatibility. In case of breaking changes, create a new instance instead of using undo.

Important!

For 2D GUIs, the the Rack Extension project folder (e.g., `VerySimpleSampler`) must contain a folder called `GUI2D` (e.g., `VerySimpleSampler/GUI2D`) that contains the high-resolution 2D graphics resources. When you compile the resources to lower resolution images for the local 45 using `RE2DRender`, you must instruct it to put them in the `GUI` folder (`VerySimpleSampler/GUI`). This will create a new folder called `Output(VerySimpleSampler/GUI/Output)` with the lo-res 2D resources and `gui.lua`. The build script assumes that this folder exists and contains 2D assets when it compiles the Rack Extension. See the *2D GUI Designer Manual* for details.

IDE projects

The SDK examples also include Visual Studio and Xcode projects.

The Visual Studio projects require Visual Studio 2013, 2015, or 2017.

The Xcode projects require Xcode 9.2 or later.

The Visual Studio projects comes with the following build configurations:

- Debugging,
- Testing,
- Deployment, and
- Universal45.

The Debugging configuration can be used for local debugging. If you compile the Debugging configuration and then attach the debugger to Reason Recon, you can debug your Rack Extension as normal, setting breakpoints and so on.

The Xcode projects have two build schemes:

- Local 45, and
- Universal 45

By building the Local 45 target for debugging, it is possible to debug your Rack Extension locally by attaching the Xcode debugger to Reason Recon.

The Universal45 target builds the project using the LLVM toolchain and packages the 45 for submitting to Reason Studios. The resulting `.u45` archive is put in the `Output` folder in the project folder, e.g.,

```
.../SDK/Examples/SimpleInstrument/Output
```

Creating a new project

macOS

When you create a new project, it is easiest to start from a copy of one of the example projects.

1. Duplicate the entire folder and rename it to whatever you want to call your Rack Extension.
2. Then rename the Xcode project file inside the folder.
3. Open the project file in Xcode. Note that you can rename files inside Xcode to better match your new Rack Extension name.

If you don't want to keep your Rack Extension projects inside the Examples folder in the SDK, you can move them somewhere else. You will, however, need to fix a few things to help the Xcode project find the SDK.

1. Move the project folder to wherever you want to put it.
2. Edit `build45.py` to point to the correct SDK folder: the `JUKEBOX_SDK_DIR` must be a relative or absolute path to the SDK folder. If you make the path relative, it must be relative to the Xcode project file for your Rack Extension.
3. Open the Xcode project file. The reference to the SDK folder, and everything within it, will be broken. Repair it by right-clicking (or control-clicking) on the SDK folder in the project file list, and select Get Info. In the Info panel that appears, click on the Choose... button to select where the SDK folder resides on disk. Navigate to where you have the SDK folder and click Choose. This should repair all the broken file references in the project.

Windows

When you create a new project, e.g., `MyRackExtension`, it is easiest to start from a renamed copy of one of the example projects, e.g., `SimpleInstrument`:

1. Rename `SimpleInstrument.sln` to `MyRackExtension.sln`.
2. Rename `SimpleInstrument.vcproj` to `MyRackExtension.vcproj`
3. Open `MyRackExtension.sln` in a text editor and replace all instances of `SimpleInstrument` with `MyRackExtension`.
4. Open `MyRackExtension.vcproj` in a text editor and replace all instances of `SimpleInstrument` with `MyRackExtension`.

If you don't want to keep your Rack Extension projects inside the Examples folder in the SDK, you can move them somewhere else. You will, however, need to fix a few things to help the Visual Studio project find the SDK.

1. Move the project folder to wherever you want to put it.
2. Edit `build45.py` to point to the correct SDK folder: the `JUKEBOX_SDK_DIR` must be a relative or absolute path to the SDK folder. If you make the path relative, it must be relative to the Visual Studio project file for your Rack Extension.

Debugging and logging

On macOS, logs from Recon are written to

`~/Library/Logs/Propellerhead Software/Reason Recon Logs`

On Windows, Recon logs are written to

```
Documents/Propellerhead Logs/Reason Recon Logs
```

The `JBOX_TRACE()` function can be used to write debug strings from a Rack Extension. On macOS, the output is written to

```
~/Library/Logs/Propellerhead Software/Rack Extension Logs
```

On Windows, the output is written to

```
Documents/Propellerhead Logs/Rack Extension Logs
```

Using macOS, you can use the Xcode debugger to set breakpoints in your DSP code, inspect variables and step through your code etc. To do this, you need to build a local 45 using the Debugging configuration. Either build using the command line:

```
python build45.py local45 Debugging
```

or from Xcode by selecting the "Local 45" scheme and then do *Build For Running*: Click the "Product" menu, then "Build For...", "Running". Next you need to tell Xcode to use Reason Recon as the executable for debugging. You do this by adding a "custom executable" to your project, or by attaching to Reason Recon while it is running.

The procedure for Windows is similar: choose the Debugging configuration and attach the debugger to a running Reason Recon.

Tip! Turn on "break on exception" to get a call stack if there is a DSP error.

Recon tools menu

Reason Recon has a menu called "Recon Tools". It allows you to do the following:

- Inspect the state of the Motherboard for a running Rack Extension.
- Inspect the tail end of the device log for a running Rack Extension.
- Send a reset audio request to all Rack Extensions currently in the rack.
- Instruct Recon to blow the fuse when a Jukebox C++ API call results in a warning (warnings are printed to the device log).
- Output custom display rendering benchmark data to the device log.

More information about the Motherboard, reset audio requests, the device fuse, and custom displays can be found elsewhere in the SDK documentation.

Running automated acceptance tests

When you submit a 45 to the Reason Studios build server, the server will run a number of automated tests in Reason Recon. It is possible to run these tests on a local 45 before you submit to the server.

macOS

On macOS, open a Terminal and navigate to the folder containing the Reason Recon executable, e.g.,

```
cd /Applications/Reason Recon/Reason Recon 11 RESDK41 Logging.app/Contents/MacOS
```

Then start Recon like this:

```
./"Reason Recon" "--validate_re" "--re_dir=RE_DIR" "--re_log=RE_LOG" "--re=RE"
```

where

`RE_DIR` is the absolute path to the folder containing the Rack Extension, e.g.,

```
/Users/johndoe/Library/Application Support/  
  Propellerhead Software/RackExtensions_Dev/SimpleInstrument
```


`RE_LOG` is the absolute path to a file where Recon should write the results of the tests, e.g.,

```
/Users/johndoe/Desktop/test.log
```

and `RE` is the product ID of the Rack Extension, e.g.,

```
se.propellerheads.SimpleInstrument
```

Windows

On Windows, open a command prompt and navigate to the Reason Recon executable, e.g.,

```
cd c:\Program Files\Propellerhead\Reason Recon
```

Then start Recon like this:

```
"Reason Recon" "/validate_re" "/re_dir=RE_DIR" "/re_log=RE_LOG" "/re=RE"
```

where

`RE_DIR` is the absolute path to the folder containing the Rack Extension, e.g.,

```
C:\Users\johndoe\AppData\Roaming\  
    Propellerhead Software\RackExtensions_Dev\SimpleInstrument
```

`RE_LOG` is the absolute path to a file where Recon should write the results of the tests, e.g.,

```
C:\Users\johndoe\Desktop\test.log
```

and `RE` is the product ID of the Rack Extension, e.g.,

```
se.propellerheads.SimpleInstrument
```

Known issues

- Due to a bug in a third-party library that RE2DRender relies on, compiling 2D GUIs may fail if the source or destination paths contain unicode characters, or begin with a numeric character. If this happens, rename the folder paths for your Rack Extension project so that they only contain non-numerical ASCII characters.
- Unicode characters in patch file names are not fully supported by the Reason Studios build server.

What's new in SDK 4.5.0?

- Added new required property to info.lua that specifies device categories
- Added description of device categories in scripting specification, including guidelines on how to apply them
- Updated all example projects to include device categories
- Reason Recon updated to correspond to Reason 13.0.0, also shows device_categories in the motherboard inspector

What's new in SDK 4.4.0?

- Reason Recon updated to correspond to Reason 12.6.1d72, with support for arm64 on macOS (fat binary).
- Build and test local45 in native Apple mode on macOS.
- Compiler toolchain is based on LLVM 15.0.7.
- All tools can run in native Apple mode on macOS (faster, no need to have Rosetta installed).
- RE2D tools updated (minor fixes). Mac version is no longer a .app bundle, instead uses the same file layout as on Windows.
- Build scripts require python 3 / removed support for python 2.x.

What's new in SDK 4.3.0?

- Reason Recon updated to correspond to Reason 12
- RE2D tools updated with hi-res support
- Hi-res images for built in elements included
- Example updated with hi-res custom display background
- Example GUIs pre-rendered in hi-res
- Build script fix for working with the latest Xcode version
- Minimum requirements change: macOS 10.13 or later. Windows 10 or later.

What's new in SDK 4.2.0?

- 3D GUI support has been deprecated
- Updated Recon to work in debug mode on MacOS Catalina
- All included binaries are signed and notarized for MacOS Catalina
- RE2DRender fixes for issues introduced when migrating from python 2 to 3

What's new in SDK 4.1.0?

- Updated software license model
- Build scripts upgraded to python 3
- MacOS Catalina support for Recon, RE2DRender and RE2DPreview
- Reason Recon updated to correspond to Reason 11
- Post-build validation (Recon ValidateRE) working for all platforms and build servers
- Fixed memory scanning issue in post build validation (Recon ValidateRE)
- Fix where big integers would be written in a lossy format in patches
- Included reduced Factory Sound Bank for Recon to reduce SDK package size
- Documentation moved from downloadable archive to <https://developer.reasonstudios.com>

What's new in SDK 4.0.0?

- Compiler toolchain is now based on LLVM 7.0
- Support for C++17
- Support for source level debugging with the LLVM compiler
- Updated example projects for Visual Studio 2013/2015/2017 and Xcode 9.2

What's new in SDK 3.0.0?

- Support for Players
- Support for pattern lane automation
- Sample drop zones
- Additional sequencer information, such as: loop points, bar position, and run/stop events
- Reason Recon updated to correspond to Reason 10.1 and Reason Intro 10.1
- Reason Recon updated with more built-in devices

What's new in SDK 2.5.0?

- Support for RT owned strings, which enables displays drawing DSP data
- Support for user samples, browsable by the end user
- Support for modifier keys in custom display gestures
- Hideable widgets enabling panels and tabs
- Popup menus in custom displays
- Reason Recon updated to correspond to Reason 9.2 and Reason Essentials 9.2

What's new in SDK 2.2.0?

- GUIs may now be defined using high resolution 2D images
- Reason Recon updated to correspond to Reason 8.2 and Reason Essentials 8.2
- Added example project files for Visual Studio 2012

What's new in SDK 2.1.0?

Reason Recon

- Reason Recon updated to correspond to Reason 8.1 and Reason Essentials 8.1
- Added Thor
- Added RPG8
- Added Spectrum Analyzer window
- Faster custom display sandboxing
- New Recon Tools menu (see above)
- Bug fixes

Rack Extension Designer

- Significantly reduced project loading/reloading time
- Significantly reduced filmstrip rendering time
- Significantly reduced temporary storage requirements when rendering filmstrips

- New `<fresnelAmount>` parameter in the Phong, Anisotropic, and Glass materials that can be used to reduce or completely remove Fresnel reflection
- 3D engine updated to use OpenGL 3.2
- New menu option for rendering a single widget
- New menu option for showing front and back panels together
- New menu option for toggling the scenegraph view
- New menu options for showing widget hit boxes
- New menu options for saving/loading current camera state
- Removed "Clamp to zoom" menu option: clamping now happens automatically when dynamic LOD switching is enabled
- Increased zoom speed for orthographic views
- Automatic checks for invalid widget bounds
- XML attributes can now be specified in any order in scenegraph and material files
- New tweak window slider for moving the back plane of the shadow camera (this allows the entire device to be shadowed when showing the front and back panels together)
- Improved error handling and error reporting
- Bug fixes
- Added sections on texturing and normal mapping to GUI Designer Manual

LLVM and compiler toolchain

- Compiler toolchain is now based on LLVM 3.5
- Jukebox now offers limited support for vector (SIMD) instructions (see Jukebox Scripting Specification for details)
- Compiler toolchain tweaked to produce faster LLVM bitcode

Acceptance testing

- Level-of-detail meshes (and/or materials) are no longer required

Part Library

- Lots of new assets, including knobs, buttons, and decorative elements
- New UV mapping for front panels that minimizes texture stretching
- New individual thumbnails for each asset with correct relative sizes and transparent backgrounds
- Various visual tweaks and bug fixes

What's new in SDK 2.0.1?

- Reason Recon updated to correspond to Reason 7.1.1 and Reason Essentials 2.1.1.
- Fixed minor errors, typos, and omissions in documentation.

What's new in SDK 2.0.0?

- New custom display widget
- New static decoration widget
- Faster math functions (requires rebuilding with SDK 2.0.0 and re-submitting to Reason Studios).
- Tighter Lua parsing in Jukebox hosting; more errors are now reported instead of accepted quietly.
- Changed memory alignment requirements: all new/malloc-allocated memory is now 16-byte-aligned. Added new toolbox function for querying optimal alignment for FFT functions. Deprecated old memory alignment querying functions.
- Improved performance when updating rt_owner properties from Realtime.
- Improved stability of build server pipeline.
- It is now possible to animate the color and intensity of <lightNode>:s in scenegraphs.
- Many under-the-hood improvements in Jukebox.
- Rack Extension Designer: Intermediate folder is now deleted after compiling 2D resources
- Completely rewritten documentation
- Various bug fixes

Introduction

Jukebox is a technology from Reason Studios for developing portable, robust and long-lived software instruments and effects. It is a central part of the Reason Studios Rack Extensions system, which makes it possible for Reason Studios and third parties to create and distribute devices for the Reason Studios Reason rack.

This document describes the main concepts of the Jukebox technology and how the different Jukebox APIs and subsystems work together on a higher level. For more detailed information, refer to the Jukebox Scripting Specification document. To get an introductory overview of the Rack Extensions system, read the "Rack Extension Developer's Guide".

It is expected that readers of this document are C/C++ developers, familiar with digital signal processing and realtime concepts. Basic knowledge about the Lua scripting language can help. It is also expected that the reader knows about general music application concepts such as note numbers, velocity and automation. Knowing about more Reason - specific concepts such as control voltages (CV), auto-routing or Remote™ is really helpful as well.

Design goals

Jukebox has been designed with focus on portability, robustness and longevity. We want any implementation based on Jukebox technology to be able to run on all platforms that Reason Studios currently supports, but also on future platforms that we are not running on today.

It should be hard to make mistakes that bring the whole host down, and if bad things do happen, the end user's working document should be kept safe.

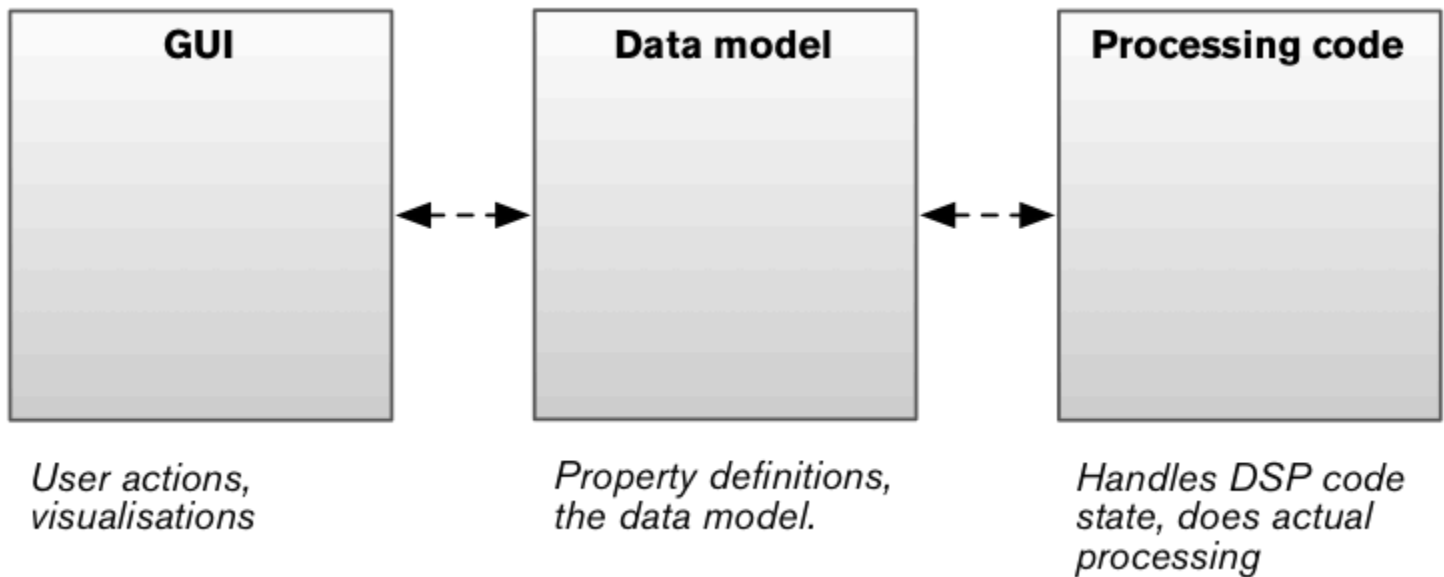
Jukebox isolates implementers from operating system and other platform issues and makes it the responsibility of the host to keep up with system updates and changing runtime dependencies.

Main concepts

A device in the Jukebox world is called a "45". 45's have a number of audio inputs and audio outputs, a piece of C++ code that performs some kind of signal processing and a set of properties that control how this

processing is performed. To be able to tweak and visualize the current value of properties, there is a GUI system that ties user gestures and graphical representations to properties in a portable manner.

A Jukebox 45 can be divided into three major parts, from user end to the actual signal processing: GUI, data model and processing code.



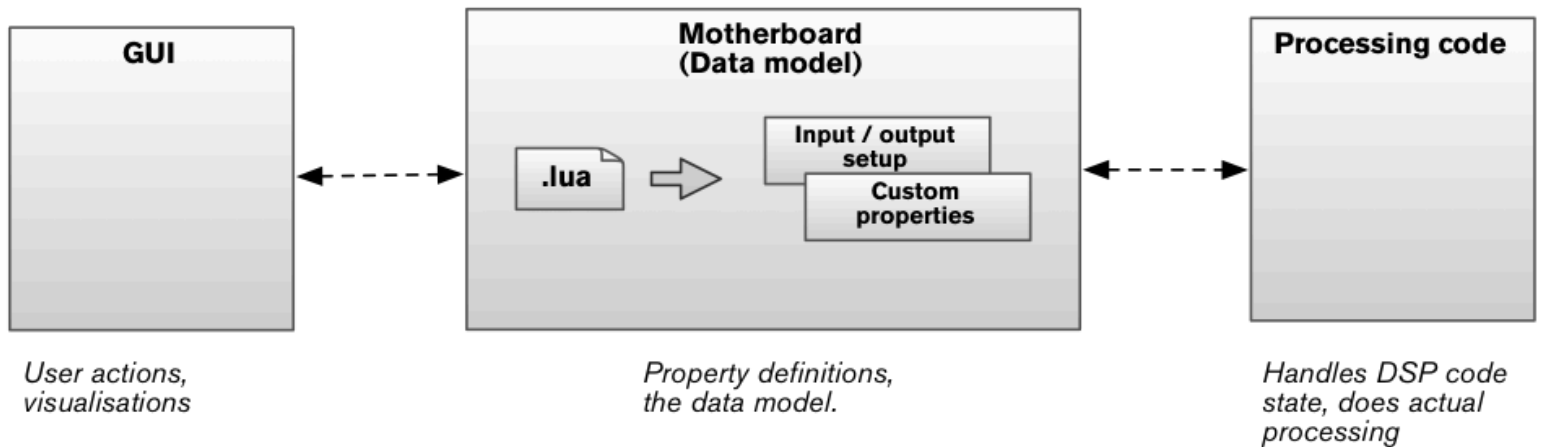
The different design goals described earlier put demands on the feature set of each of these parts, but also limitations in terms of flexibility.

Jukebox 45's are hosted in a sandboxed environment that provides much of the functionality needed to get a fully functional 45. A data driven approach has been used for major parts of the data model and GUI, and it is the responsibility of the host to provide robust handling of that data and to maintain platform compatibility.

The native code that makes up the realtime part runs in a sandboxed environment, which is both limited in terms of functionality and keeps 45's under close supervision. There is for example no direct access to OS libraries, and a 45 that tries to allocate memory while rendering audio will be shut down.

A special version of Reason called Reason Recon is provided with the Jukebox SDK for testing and debugging purposes.

Motherboard

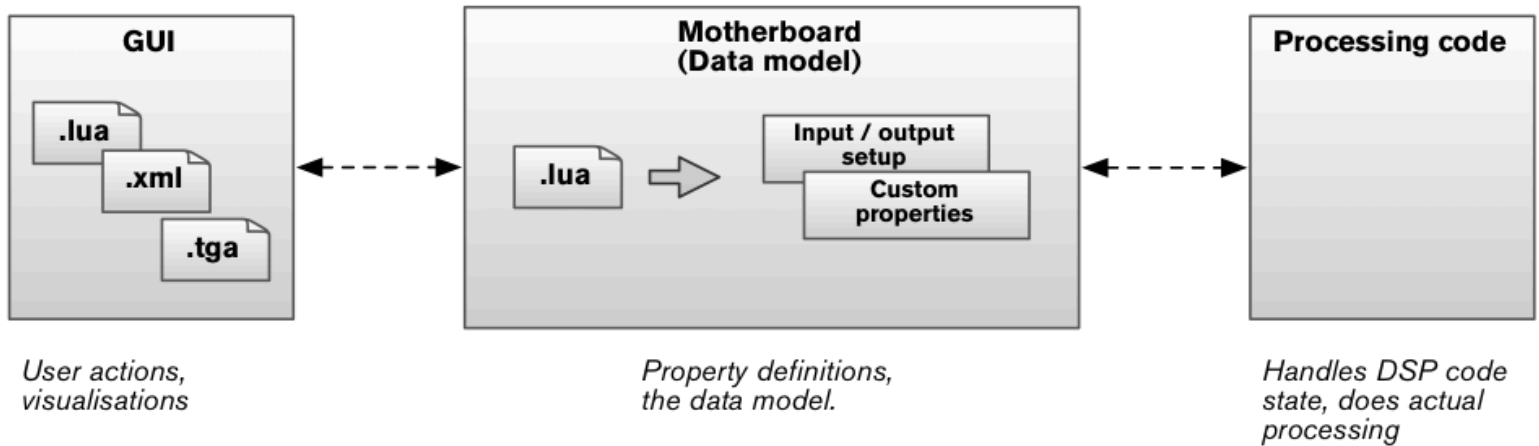


The data model in Jukebox is called the Motherboard and is defined using the Lua scripting language. The Motherboard holds *properties* that are organized into *property set objects*. Some of the objects are built-in and defined by the host; others are defined specifically for each 45 implementation. 45-specific properties are known as *custom properties*.

The Motherboard also contains information about other things, such as input and output sockets, MIDI CC and Remote™ mappings, and automatic routing hints.

Once defined, the Motherboard is under complete control of the host, even though some parts can be customized by the use of Lua scripts. Because of this, the host can provide complete undo handling, generic patch handling and so on.

GUI

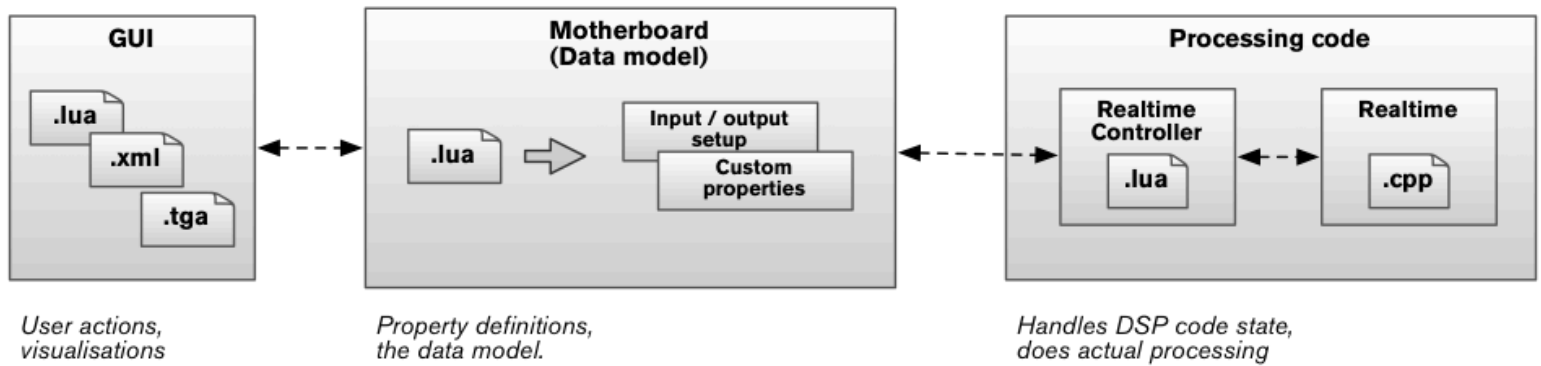


The Jukebox design goals put special demands on the GUI side. A Jukebox GUI must have:

- Scalable graphics that works from lower resolution small displays to higher resolution displays of tomorrow.
- Pixel-unaware drawing and user actions. In fact, there can be no assumptions made on any kind of input method. That is all handled by the host.

In Jukebox, a GUI is defined as a set of high resolution 2D images that are connected to the motherboard via Lua scripts. The panels and parts that make up a device are represented by 2D images.

Processing code



For performance reasons, data processing is done in native code, but must still live up to the Jukebox design goals. All data processing in Jukebox is done in portable C++ by implementing two functions:

- `JBox_Export_RenderRealtime()` (which is responsible for processing audio), and
- `JBox_Export_CreateNativeObject()` (which is primarily responsible for allocating memory).

The C++ code runs in a limited environment with no access to operating system functions or external libraries other than those compiled from source code or provided by Jukebox.

Realtime

The processing side of Jukebox is often referred to as the *Realtime*, even though not all processing is done in real time, as we will see later.

Realtime Controller

To keep the Realtime as simple as possible, the host is responsible for much of the housekeeping. That part of the host is called the *Realtime Controller* (RTC). For situations where more control is needed, it is possible to extend the built-in Realtime Controller behavior by the use of Lua scripts.

The virtual fuse

If a 45 misbehaves or generates an exception or error, the host will halt the 45 and stop sending information to it. This is referred to as *blowing the fuse*. After the fuse has blown the 45 is "dead" in the sense that it cannot process sound and cannot be modified by the user. However, the user's song document can still be manipulated and saved.

Anatomy of a 45

All 45 projects contain the following:

- Lua scripts:
 - `info.lua` (contains general settings)
 - `motherboard_def.lua` (motherboard definition)
 - `realtime_controller.lua` (realtime controller)
 - `hdgui_2D.lua` (GUI widget definitions)
 - `display.lua` (code that draws into custom displays)
- C++ source files (realtime DSP processing)
- Localized text resources
- GUI resources and metadata files

For the 2D GUI pipeline, the 45 project contains (see 2D GUI Designer Manual for details):

- Static images
- Film strip images
- Graphics definitions file (`device_2D.lua`)

A 45 may also contain any number of

- Binary resources (Binary Large Objects)
- Samples
- Patches (device patches and/or Combinator patches)

It can also contain a `<Product>.rsmeta` file that holds metadata about publicly visible patches and samples included with the 45.

Important! All non-binary files, including all Lua scripts, must be encoded as ASCII or as UTF-8 Unicode without BOM.

Resources

A Jukebox 45 can include any number of files that are either private to the 45 or publically available to other devices or subsystems in the same host. An example of a private resource could be a wavetable that will be loaded as a BLOB (Binary Large Object) and interpreted by 45 specific code. There is no need for that resource to be exposed to other devices. Obvious examples of public resources are the patches that come with a 45. These must be publically available, as the host will expose them for browsing, etc. Public resources live in the `Resources/Public` resource subfolder of a Jukebox project and private in the `Resources/Private` resource subfolder. Assets that do not live in any of these folders are internal to the Jukebox system and will only be accessed by the host. When referring to resources in scripts, a forward slash separated resource path is used.

The `Public` subfolder will be the root of the publically browsable file system as viewed by for example the Reason browser. Putting samples in the `Public` subfolder will make them available for export and use by other devices. For now, we recommended keeping samples private unless they are part of the public patches for the 45, in which case they **must** be included in the `Public` folder (see next note).

If you choose to store Combinator and/or other kinds of patches in the `Public` folder, it is very important that any files referenced by those patches, such as samples, are made available to the Rack Extension by including them in the `Public` folder. (Samples that are part of the Factory or Orkester sound banks do not have to be included.) To do this, follow these steps: 1) Put the files you want to reference in the `Public` folder and rebuild the Rack Extension. 2) Create the patch in Recon. When you point to referenced files, browse into the Rack Extension. **Do not point to a file on your hard drive!** 3) Save the patch. 4) Copy the patch to the `Public` folder and rebuild the Rack Extension.

Localized text resources

With a few exceptions, all strings that are shown in the GUI of a 45 must be localized, including property names and values. In situations where you refer to a localized string, you provide a *UI text key* rather than the string itself. The host uses the key to fetch the appropriate string for the currently selected language from the 45's set of localized text databases. There is one database for each language; English is required. The other languages are German, French, and Japanese. See the Jukebox Scripting Specification for information about the format and location of the database files.

General configuration

The `info.lua` file is the main descriptor of a 45. It is used to uniquely identify the 45, and also contains host-specific configuration settings.

Example:

None

```
format_version = "1.0"

product_id = "se.propellerheads.Parsec"
version_number = "1.0.1f4"

manufacturer = "Reason Studios"
long_name = "Parsec Spectral Synthesizer"
medium_name = "Parsec"
short_name = "Parsec"

device_type = "instrument"
device_categories = { "Synth" }

accepts_notes = true

supports_patches = true
default_patch = "/Public/Glistening.repatch"

auto_create_track = true
auto_create_note_lane = true
supports_performance_automation = true
device_height_ru = 9
automation_highlight_color = {r = 60, g = 255, b = 2}
```

The `format_version` entry specifies which version of the `info.lua` format that the file follows. All files in Jukebox (except C++ files) have an individual format version. As the Jukebox API is extended or updated, the format of the files may change. The Jukebox Scripting Specification document contains detailed information about all Jukebox configuration and script file formats, including which features are available in which file format version.

The `product_id` is a unique identifier for the product and may not change once the product has been uploaded to the Reason Studios servers. It is never shown in host GUIs.

The `version_number` specifies the product version of the 45. It must be unique (once you have uploaded a universal 45 with a certain version number you cannot upload the same version number again). The version number is shown to end-users in the Reason Studios Shop. The number has the following format:

`A.B.Cr`

where `A` is the *major number*, `B` is the *minor number*, `C` is the *revision number*, and `r` is a letter that indicates the *stage* of the 45 (`d` for testing/debugging, `b` for beta, and `f` for final/release candidates). The stage is not shown in the Reason Studios Shop. It is recommended to follow the following conventions:

- Update the major number to indicate paid upgrades.
- Update the minor number to indicate feature additions.
- Update the revision number to indicate bug fixes or minor updates.

The `manufacturer` string contains the name of the manufacturer. It is shown in the Shop and in the host.

The `long_name` string contains the name of the device. It is shown in the Shop and in the host.

The `medium_name` string contains a shorter version of the device name. It is used when the host needs to identify the device but has limited space to do so. For example, Reason uses the medium name in the Undo menu ("Undo create Synthesizer").

The `short_name` string contains a short version of the device name. It is used in situations where the host has very little space to display the device name. For example, Reason uses the short name when auto-naming new instances of the device in the Rack view.

The `automation_highlight_color` entry specifies the color of automation indicators in Reason. It should not be changed unless absolutely necessary.

The `device_height_ru` entry contains the device height in Rack Units. The value must match the panel graphics in the GUI.

The `default_patch` entry contains the name of the patch to load by default, if the device supports patches (see below).

The `supports_patches` flag specifies whether the device is able to load/save patches or not.

The `device_type` specifies whether the 45 is an instrument, effect, or utility device. The setting is used to configure certain Jukebox functionality (such as automatic routing) but may also be used by the host to organize products into different categories.

The `device_categories` entry assigns one or more categories to the device for more fine-grained filtering in the host. The categories must be chosen from the list of predefined categories specified in the Jukebox Scripting Specification.

The other parameters in the example above are Reason-specific configuration parameters.

See the Jukebox Scripting Specification for detailed information about the format of `info.lua`.

Patches

All Jukebox devices share a common patch format, which is a simple XML-based format. The file extension for patch files is `.repatch`. See the Jukebox Scripting Specification document for details.

The `supports_patches` boolean setting in `info.lua` controls whether a 45 uses patches or not. To allow the user to browse for patches, add a `patch_browse_group` and a `patch_name` GUI widget pair to the front and folded front panels. See the GUI Designer Manual and the Jukebox Scripting Specification for details.

It is not necessary for a 45 to support patches. If it does, however, the `Public` resources folder must contain a default patch (which is specified in `info.lua`). Creating a default patch manually (by editing an XML file) is not recommended. Instead, set `default_patch` in `info.lua` to any patch and create an instance of the 45 in Reason. Reason will fail to load the patch (and show an error message) but will not blow the fuse. Since your 45 is now running, it is now straightforward to create a patch, save it to the `Publicresources` folder, and point to it in `info.lua`.

If you wish to create patches that use references to samples you will supply, you need to include the samples in your Public folder and build the 45 first. After building the 45, load the samples from the devices public folder by navigating to Rack Extensions ⇒ 45 in the Reason/Recon browser in order to get the correct references in the patch. The same goes for building combi-patches that reference samples that should be included in your 45.

Devices that support patches must include a reasonable number of well organized patches.

It is also strongly recommended to assign categories, tags and author information to included patches and samples as described in the Jukebox Scripting Specification. This will make patch and sample content much easier to find in hosts that support it.

Motherboard definition

The motherboard is where the data model and much of the logic of a 45 lives. This is also the part that provides a lot of the core functionality of the Jukebox environment. The motherboard is defined in the `motherboard_def.lua` file.

The most important part of the motherboard is the *motherboard object model* (MOM). It contains the properties that the host uses to communicate with the 45 (and vice versa). Part of the MOM is fixed and provided by Jukebox, and includes properties containing the current sample rate and song position. The other properties in the MOM are *custom properties* and are defined by the 45 in the `motherboard_def.lua` file. All custom properties are automatically hooked into the host environment, which enables features such as undo, loading and saving patches.

The motherboard definition also contains audio and CV inputs and outputs, Remote™ and MIDI CC configurations, and hints to the host about how the device should be routed to other Jukebox devices.

It can optionally also define **user samples**, allowing the use of audio sample files selected by the end user. See section [User Samples](#) for more.

Example:

None

```
format_version = "2.0"

--[ Motherboard Object Model (MOM) configuration ]--

-- Declaration of a motherboard property set object
custom_properties = jbox.property_set{
    -- Properties in the document owner scope
    document_owner = {
        properties = {
```

```

-- Property declaration (number type)
volume = jbox.number{
    property_tag = 42,
    default = 0.7,
    ui_name = jbox.ui_text("Volume_UI"),
    ui_type = jbox.ui_percent{},
},

-- Performance properties
modWheel = jbox.performance_modwheel{},
pitchBend = jbox.performance_pitchbend{},
},

-- Properties in the Realtime Controller owner scope
rtc_owner = {
    properties = {
        -- Native object to hold C++ state for the 45 instance
        instance = jbox.native_object{ },
    },
},

-- Properties in the Realtime owner scope
rt_owner = {
    properties = {
        -- Property declaration (boolean type)
        note_on = jbox.boolean{
            default = false,
            ui_name = jbox.ui_text("NoteOn_UI"),
            ui_type = jbox.ui_linear{min=0, max=1,
units={{decimals=0}}},
        },
    },
},
}

--[[ MIDI configuration ]]--

```

```

midi_implementation_chart = {
    -- Mappings between MIDI CC and properties in the document owner
scope
    midi_cc_chart = {
        [12] = "/custom_properties/volume",
    },
}

--[[ Remote(TM) configuration ]]--

remote_implementation_chart = {
    ["/custom_properties/volume"] = {
        internal_name = "Volume",
        short_ui_name = jbox.ui_text("Volume_Short"),
        shortest_ui_name = jbox.ui_text("Volume_Shortest")
    },
}

--[[ Property grouping for host's GUI ]]--

ui_groups = {
    {
        ui_name = jbox.ui_text("Group_Volume"),
        properties = {
            "/custom_properties/volume",
        },
    },
}

--[[ Input and output sockets ]]--

-- CV input sockets
cv_inputs = {
    note_cv = jbox.cv_input{
        ui_name = jbox.ui_text("CV_In_Note")
    },
}

```

```

        gate_cv = jbox.cv_input{
            ui_name = jbox.ui_text("CV_In_Gate")
        },
    }

-- Audio output sockets
audio_outputs = {
    left = jbox.audio_output{
        ui_name = jbox.ui_text("Audio_Out_Left")
    },
    right = jbox.audio_output{
        ui_name = jbox.ui_text("Audio_Out_Right")
    },
}

--[[ (optional) User Samples ]]--

user_samples = {
    jbox.user_sample{
        ui_name = jbox.ui_text("text_Sample0_UIName"),
    },
}

--[[ Automatic routing configuration ]]--

-- The 45 should be routed as a stereo instrument
jbox.add_stereo_instrument_routing_hint{
    left_output = "/audio_outputs/left",
    right_output = "/audio_outputs/right"
}

-- Sockets on the back panel of the 45 that the host can auto-route to
jbox.add_stereo_audio_routing_target{
    signal_type = "normal",
    left = "/audio_outputs/left",
    right = "/audio_outputs/right",
    auto_route_enable = true
}
jbox.add_cv_routing_target{

```

```

        signal_type = "gate",
        path = "/cv_inputs/gate_cv",
        auto_route_enable = true
    }
    jbox.add_cv_routing_target{
        signal_type = "pitch",
        path = "/cv_inputs/note_cv",
        auto_route_enable = true
    }

    -- Let the host know that these two sockets form a audio stereo L+R pair
    jbox.add_stereo_audio_routing_pair{
        left = "/audio_outputs/left",
        right = "/audio_outputs/right"
    }

```

After the `format_version` specification, this example declares a *property set object* with the name `custom_properties`. A property set object is, as the name suggests, a container for properties. All properties are named, and the name must be unique inside the property set object.

Important! Property names must contain ASCII characters only. A property name cannot be longer than 34 characters.

The `/custom_properties` property set object (that holds "your" properties) is not the only property set in the Motherboard of a 45. The host automatically adds other property sets for, e.g., receiving note states or keeping track of sequencer state. These are described in details in the Jukebox Scripting Specification.

A property is referred to by its *property set path*. For example, the property named `volume` in the `custom_properties` object would be referred to as

```
/custom_properties/volume
```


In Lua, the functions `jbox.load_property()` and `jbox.store_property()` are used to read from and write to properties.

In C++, you use the `JBox_GetMotherboardObjectRef()` function to obtain a reference to a motherboard object, followed by `JBox_MakePropertyRef()` to obtain a reference to a property. These references never change during the lifetime of a 45 and may be stored in C++ data structures.

You may also assign a *tag* to a property. In the example above, the property named `volume` has been given the tag `42`. Property tags allow you to refer to properties via numbers instead of via their property paths, which is useful, e.g., in `switch` statements.

Property ownership

All properties in a property set object have an *owner scope*. The owner scope defines which part of the 45 that is allowed to write to the property. There are four owners:

- `gui_owner` - Properties in this scope can be modified by the host's GUI widgets. The properties are normally not stored in documents or patches. In Reason, GUI property changes are saved "transparently" in the undo history in the sense that all GUI property changes are "remembered" but they are not visible to the user in the Undo menu.
- `document_owner` - Properties in this scope may be stored in documents and/or patches. The properties can be modified by the host's GUI widgets and/or by loading a patch or song.
- `rtc_owner` - Properties in this scope can be modified by the Realtime Controller. They can not be stored in documents or patches.
- `rt_owner` - Properties in this scope can be modified by the Realtime. They can not be stored in documents or patches.

Tip! You use the `persistence` flag to specify whether properties in the document owner scope should be saved to patches and/or documents. Set this flag to `"song"` for Dry/Wet knobs on effects! This will store the Dry/Wet setting in songs, but not in patches.

MOM property types

The Jukebox property system is strictly typed. These are the types:

- Number - Double precision floating point. Can be either continuous or discrete (stepped).
- Boolean - True or false.
- String - Non-localized 8-bit character string.
- Sample - An audio sample, stereo or mono.
- BLOB - Binary Large Object that contains raw bytes. BLOBs usually contains static data that the 45 needs to compute sound, such as pre-computed waveform tables. **See note on endianness below!**
- DSP Buffer - An array of 32-bit float samples. DSP buffers are used for sending audio data from the host to the 45's C++ sound rendering code (and vice versa).
- Native Object - Holds data returned by `JBox_Export_CreateNativeObject()`. A native object is a memory block that has been allocated dynamically by the 45 and that is used in its C++ sound rendering code. See section [Realtime](#) for details.

Important! All Rack Extension code must be endian safe. It is incorrect to assume that the host has any specific endianness. Reading integer data from a BLOB should only be done on a byte by byte basis (using bit shifts to create the multibyte value). You may not assume that the host's endianness for floating point values is the same as its endianness for integers. You may not assume that the floating point representation is IEEE 754 or any other specific representation. If you need to store floats in a BLOB you can use ASCII strings, or store the exponent and mantissa separately as integers.

When you read a value from a property in C++, the value is represented as an opaque type called `TJBox_Value`. You use toolbox functions to access the actual contents of the `TJBox_Value` instance. Conversely, when you update the value of a property in C++, you construct a `TJBox_Value` using toolbox functions and send it to the host.

Important!

You must never store `TJBox_Value` instances in C++ data structures! A `TJBox_Value` instance is only guaranteed to be correct in the scope of a single call to `JBox_Export_RenderRealtime()`.

In Lua, number, boolean, and string property values are represented by the corresponding built-in Lua types. The other property types are represented by opaque types. For these types, there exist Lua functions for accessing the contents and for constructing new values.

All property types have a well-defined default value. Some property types require that you supply a default value explicitly when you create the property (see Jukebox Scripting Specification for details).

Performance properties

If the host responds to MIDI Continuous Controller messages, it may require special handling for messages for modulation wheel, pitch bend, sustain pedal, expression, breath control, and aftertouch. Therefore, these messages use special-purpose property types. In the example above, the properties named `modWheel` and `pitchBend` are defined as performance properties. See the Jukebox Scripting Specification for details.

User Samples

A device with user samples takes part in the native host features to browse for audio sample files, and create new files by sampling, bouncing or editing (sample editor). Optionally, the device can also specify to include one or more sample meta-data properties (the **Sample Parameters**) in its patch data. User samples also enable host properties under the `/device_host` object, which the device can use to integrate with the user samples hosting.

Example:

None

```
format_version = "2.0"
user_samples = {
    jbox.user_sample{
        ui_name = jbox.ui_text("text_Sample0_UIName"),
        sample_parameters = {
            "root_key",
            "tune_cents",
            "play_range_start",
            "play_range_end",
            "loop_range_start",
            "loop_range_end",
            "loop_mode",
            "preview_volume_level"
        }
    },
}
```

This example defines properties for a user browsable sample item and all of its sample parameters. The property path of the sample item is `/user_samples/0/item`, and each of the sample parameters are members/fields of the same object instance (e.g. `/user_samples/0/root_key`). See the Jukebox Scripting Specification for details.

User interface names and types

The *user interface name* (`ui_name`) of a property is a localized string that contains the name of the property. It is used by the host to identify the property in its GUI. Since the string is localized, the `jbox.ui_text()` function is used to fetch the appropriate string from the localized text database. In the example above, the UI name for the `volume` property uses the `"Volume_UI"` key, which means that the localized text databases would all have entries like this:

Resources/English/texts.lua:

```
None
format_version = "1.0"
texts = {
    ["Volume_UI"] = "Volume"      -- English
}
```

Resources/German/texts.lua:

```
None
format_version = "1.0"
texts = {
    ["Volume_UI"] = "Volumen"     -- German
}
```

etc.

The *user interface type* (`ui_type`) entry in a property definition specifies how the value of the property should be presented in the host's GUI. The host may allow the user to enter values for the property, so the user interface type may be used for both input and output. See the Jukebox Scripting Specification for details on how to set up UI types.

Host properties

The host will automatically populate the MOM with a collection of property set objects that it uses to expose internal state to the 45. These properties are called *host properties*. See the Jukebox Scripting Specification for details.

MIDI implementation

The `midi_implementation_chart` section in `motherboard_def.lua` contains MIDI-related configuration settings for the 45. In the current version of Jukebox, it has one entry, the *MIDI CC chart*. This chart associates properties in the `document_owner` scope with MIDI Continuous Controllers. When such an association is in place, the user can modify the property via MIDI CC messages (if this is supported by the host).

Important!

A number of CC:s are reserved by Jukebox for internal use.
See the Jukebox Scripting Specification for details.

The **MIDI Implementation Chart** document (which can be found in the Reason documentation folder) contains the MIDI CC associations for all legacy devices in Reason. If your 45 is similar to one of the legacy devices, we strongly recommended to use similar associations if possible.

Remote™ implementation

The Remote™ protocol is designed to simplify communication between Reason and third-party hardware keyboards and controllers. The protocol translates MIDI messages into messages that are specific to Reason. In order to integrate properly with the Remote system, the 45 must provide the Jukebox host with information about how its various controls (knobs, faders, etc.) should be mapped to Remote-enabled hardware controllers.

From the user's perspective, a Remote-enabled hardware controller is "wired" to work with Reason: when a Reason device is selected in the rack, for example, the hardware control surface items (such as physical knobs and faders) are automatically associated with the appropriate controllers on the rack device. Status indicators on the rack device or other indicators in Reason can be mirrored on lamps and LEDs on the hardware controller (e.g., the status of the SOLO and MUTE buttons in the Reason mixer). If the hardware controller contains a display, the Remote protocol allows the hardware controller to show the values of a rack device parameter, or the location of the song position, etc.

Remote™ codecs

Each hardware controller that supports Remote must have a corresponding *codec* (provided by the hardware manufacturer). All Remote codecs that are available on the user's computer are scanned by Reason on startup, which allows the user to select which controller to use in the Control Surfaces tab in Preferences. The codec contains a list of the available control surface items such as keyboard, faders, buttons, etc. The codec also contains a set of rules that are used to translate MIDI messages sent from the hardware controller to Reason (and vice versa).

Remote™ maps

The Remote codec contains information about the physical layout of a hardware controller, together with information about how MIDI messages should be sent to and from the controller. But the codec does not contain any information about how to map specific control surface controls to rack device widgets in Reason. Such information is collected in *Remote maps*.

For example, the following excerpt from a Remote map for a hypothetical hardware controller with eight knobs defines mappings for the Malström instrument. It first sets up three groups (*Filter*, *Osc A*, and *Osc B*), or variations, that the user can switch between when controlling Malström. In the first variation the eight knobs control the filters, in the second variation the knobs control oscillator A, etc. After the group definition statement, a list of specific mappings follow. Each mapping associates a control surface ID (e.g., *Knob 1*,

which would be defined in the codec) with a *remotable item* (e.g., `Filter A Freq`), and which group (`Filter`, `Osc A`, or `Osc B` in this case) that the mapping belongs to.

Scope Reason Studios Malstrom Grintable Synthesizer

Define Group		Keyboard Shortcut Variations	Filter	Osc A	Osc B
Map	Knob 1	Filter A Freq	Filter		
Map	Knob 2	Filter A Resonance		Filter	
Map	Knob 3	Filter B Freq	Filter		
Map	Knob 4	Filter B Resonance		Filter	
Map	Knob 5	Filter Env Attack	Filter		
Map	Knob 6	Filter Env Decay	Filter		
Map	Knob 7	Filter Env Sustain		Filter	
Map	Knob 8	Filter Env Release		Filter	
Map	Knob 1	Oscillator A Motion		Osc A	
Map	Knob 2	Oscillator A Index		Osc A	
Map	Knob 3	Oscillator A Shift		Osc A	
Map	Knob 4	Oscillator A Attack		Osc A	
Map	Knob 5	Oscillator A Decay		Osc A	
Map	Knob 6	Oscillator A Sustain		Osc A	
Map	Knob 7	Oscillator A Release		Osc A	
Map	Knob 8	Oscillator A Gain	Osc A		
Map	Knob 1	Oscillator B Motion		Osc B	
Map	Knob 2	Oscillator B Index		Osc B	
Map	Knob 3	Oscillator B Shift		Osc B	
Map	Knob 4	Oscillator B Attack		Osc B	
Map	Knob 5	Oscillator B Decay		Osc B	
Map	Knob 6	Oscillator B Sustain		Osc B	
Map	Knob 7	Oscillator B Release		Osc B	
Map	Knob 8	Oscillator B Gain	Osc B		

Since the list of available legacy devices in Reason varies depending on the Reason version, the list of remotable items also varies with the Reason version. As a result, hardware controller manufacturers normally release new Remote maps for their devices whenever Reason is updated.

Because Remote maps are ASCII text files and have an open format, they can also be created by the user, or be overridden in Reason by right-clicking on a control and select the Edit Remote Override Mapping menu option.

For further details about the Remote protocol, see the Remote SDK documentation (available for registered Remote developers) or contact Reason Studios developer support.

Remote™ implementation charts for Rack Extensions

All 45:s must provide a remotable item definition for each property that can be controlled by the user (i.e., that is exposed in the GUI and/or MIDI CC implementation). This allows hardware controller manufacturers (and users) to update their Remote maps to support the new 45.

The `remote_implementation_chart` entry in `motherboard_def.lua` associates remotable items with Rack Extension properties. In the example above, a remotable item with the internal name `Volume` is associated with the `volume` property.

See the Jukebox Scripting Specification for details on how to set up remotable items.

Property UI grouping

The `ui_groups` table of `motherboard_def.lua` allows you to create groups of logically related motherboard properties for UI display purposes. The host may use the grouping information in any way it sees fit, including changing the order of the the groups and the properties in each group. In the current version of the Reason combinator, for example, groups are not shown if the motherboard contains fewer than ten automatable properties.

See the Jukebox Scripting Specification for details on how to set up property UI groups.

Inputs and outputs

A Jukebox 45 can communicate with the outside world via MIDI, and audio/CV inputs and outputs. These input and output sockets are defined in `motherboard_def.lua` and are read from and written to by the C++ Realtime code. For receiving and sending MIDI, see section [Receiving and sending notes](#).

Each input/output are individual property set objects. For example, the motherboard definition above has two audio outputs, `left` and `right`, and two CV inputs, `note_cv` and `gate_cv`. Both outputs have two properties, `buffer` (the DSP buffer that the Realtime writes to) and `connected` (a boolean property that the host sets to `true` when a cable is connected to the socket). The property path to, say, the `left` socket's `buffer` is

```
/audio_outputs/left/buffer
```

Tip!

For stereo audio input pairs, use the `connected` property to set the 45 to mono operation mode if only the left socket is connected.

Automatic routing

Jukebox allows devices to connect to each other via their input and output sockets. The host may offer different kinds of automatic routing functionality to improve the user experience. In Reason, for example, such functionality come into play when the user connects cables manually (connecting the left socket in an audio stereo pair will automatically connect the right socket), when the "Auto-route" menu option is invoked for a selected device, when a device is added to or deleted from the rack, etc.

In order to facilitate automatic routing, your Jukebox device may provide information about how it should be routed. If it does not provide routing information, the host may not be able to offer automatic routing functionality for the device (in which case the user must connect all sockets manually). The exact automatic routing behavior depends on the host and the current connection state of all involved devices.

To set up automatic routing, you need to provide the following information:

- One or several *routing hints* that describes the general function of your device.
- A set of *routing targets*, i.e., input and/or output sockets on the back panel of your device that the host can use for automatic routing.
- *Routing pair definitions* for all L+R stereo audio socket pairs. It is strongly recommended to supply routing pair definitions for all L+R audio socket pairs even for those pairs that are not part of any routing target definition.

The `motherboard_def.lua` example above defines automatic routing hints suitable for a stereo instrument.

See the Jukebox Scripting Specification for details on how to set up automatic routing.

Run

Rack Extensions that have an internal sequencer (what we call pattern devices), should implement the Run behaviour. In general, pattern devices should start and stop in sync with master sequencer transport. With Run, the user can start/stop the internal sequencer independently of Reason's sequencer:

- When Reason is stopped, the user can Run the pattern device on it's own to preview and edit patterns
- When Reason is running, the user can pause the pattern device

A good example of the Run behaviour is the Redrum, so give it a spin to get a feel for the behaviour! There are some issues to consider with Run:

- Run button should not be automatable. If you want the user to be able to automate the pattern playback, please use a separate property (Sequencer On/Off) or an empty pattern. The reason for this is that we do not want setting the Play position to a PPQ where Run is automated to On to start the device sequencer. Please note that even if the Run button should not be automatable, it should still be a remoteable item.
- In Bypass All mode, Players should stop and not listen to Run messages.
- The Combinator has a Run Pattern Devices button, that updates the `/transport/request_run` property to all devices in the Combi.

Players: ON/OFF and Bypass All Modes

Players can be disabled in two different ways:

1. Using the required ON/OFF button on the device front panel
2. Using the Bypass All switch in the Player Header

These two settings are somewhat overlapping, and how the device is designed can make difference to how it handles the two states. In general it can be thought of like this:

The ON/OFF button should work like turning the power on and off for the device. In OFF mode, the device should only bypass incoming MIDI, and not generate any MIDI or CV of its own. It is recommended that custom displays are cleared or “unlit” to make the device look inactive.

In Bypass All mode, the device should still be responsive (power on, custom displays can be editable), but it is disconnected from the MIDI chain by Reason. It should not generate or process any MIDI or CV, or Run when Reasons transport is running. Even if the device is still ON, we recommend indicating somehow that it is bypassed (for example by writing “Bypassed” in a display or similar).

Motherboard versions and compatibility

An update to a released 45 must keep patches and songs from earlier versions sound and function exactly the same as before. In other words, the default settings of new properties must not affect the sound. In addition, certain rules must be followed to ensure motherboard compatibility:

- Properties in the `document_owner` scope and `gui_owner` scope that are stored in patches and/or songs must not be removed in the new Motherboard version.
- The persistence of a `document_owner` property or a `gui_owner` property may never be reduced, i.e. change from "patch" to "song", or from "song" to "none". (It is allowed to increase from "none" to "song", or from "song" to "patch", however.)
- Properties cannot change type between Motherboard versions.
- MIDI CC and Remote mappings cannot change between Motherboard versions.
- The number of steps for discrete (stepped) number properties cannot be decreased between Motherboard versions.
- CV and audio inputs/outputs cannot be removed from the new Motherboard version.
- CV and audio inputs/outputs must be declared in the same order in both Motherboard versions.

Realtime Controller

The Realtime Controller (RTC) is responsible for serving the C++ realtime code with data, almost like a driver. The most common use of the RTC is to create a native object for the realtime code's own instance data, often in the form of a C++ class instance that represents the "signal processing engine" of the 45. Other common uses for the RTC is to reconfigure the 45 when the host changes the sample rate, or for loading BLOBs or samples.

The RTC is configured in a Lua file called `realtime_controller.lua`.

RTC bindings

The RTC can be used to set up properties that change their value as a result of changes in one or several other properties. For example, let's say that we want to reload a sample when the host changes the sample rate. To do this, we would create an association between the `/environment/system_sample_rate` property and our sample property, e.g., `/custom_properties/my_sample`. That way, the `my_sample` property will be automatically updated whenever `system_sample_rate` changes. Such derived properties (which must be `rtc_owner`, since they are modified by the RTC) are updated via Lua functions that you specify in `realtime_controller.lua`.

For example:

```
None
rtc_bindings = {
  {
    source = "/environment/system_sample_rate",
    dest = "/global_rtc/change_sample"
  },
}
```

```
global_rtc = {  
    change_sample = function(source_path, new_value, old_value)  
        local new_sample_rate = new_value  
        local sample_name = ...  
        jbox.store_property("/custom_properties/my_sample", sample_name)  
    end,  
}
```

The only information that can be accessed inside a RTC function is:

- The input arguments.
- The value of the source(s) of the binding (which would be `system_sample_rate` in the example outlined above). You may use `jbox.load_property()` to read these values.
- Standard Lua structures and utility functions.
- Any non-Jukebox tables set up in the Lua file.
- A subset of the Jukebox Lua toolbox functions.

A RTC function can not have any side-effects, except for updating properties in the Motherboard. This means that all creation and updating of global data in the Lua virtual machine is forbidden. If you need to communicate information between two RTC functions, you must do so via `rtc_owner` properties.

See the Jukebox Scripting Specification for further details.

Sample rate configuration

All 45s must support the following sample rates: 22050, 44100, 48000, 88200, 96000, and 192000 Hz. At least one of these must be supported natively. If the 45 does internal resampling to support one of these rates, that rate should be declared as converted. This allows the host to do sample rate conversion to one of the native sample rates if needed.

See the Jukebox Scripting Specification for details.

Property change notifications to the Realtime

When the Realtime signal processing code is called upon to render audio, it can read any property in the Motherboard (except those in the `gui_owner` scope). However, the Realtime code is often only interested in those properties that have changed their values since the last render callback. One of the arguments to `JBOX_Export_RenderRealtime()` (the audio rendering function that your 45 makes available and the host calls) is a list of such *property diffs*.

For efficiency reasons, the RTC must tell the host which of the properties in the MOM that the Realtime should receive diffs for. Those properties are listed in the `rt_input_setup` section of the RTC configuration script. See the Jukebox Scripting Specification for details.

Realtime

The Realtime sound rendering code of a 45 is written in C++. The 45 must implement and export two functions to the host:

- `JBox_Export_RenderRealtime()` - Callback for audio rendering
- `JBox_Export_CreateNativeObject()` - Callback for heap memory allocation

The host will continuously call `JBox_Export_RenderRealtime()` to let the 45 do audio processing. Most of the time, this will happen as a result of a callback from the operating system audio subsystem, but it might also be called from other contexts as well, e.g., to do background processing or stream audio to disk.

Important! Updates of `document_owner` and `gui_owner` properties (which are initiated from GUI widgets, including custom displays) are reported to `JBox_Export_RenderRealtime()` once per batch. Property changes may be merged by the host. In other words, the Realtime has access to the latest (current) value of such properties at the start of each batch, but not necessarily all the property changes that were made since the previous call to `JBox_Export_RenderRealtime()` returned.

It is vital that `JBox_Export_RenderRealtime()` carries out its work quickly; it will be interrupted by the host (and the fuse will be blown) if it does not return within a certain amount of time.

`JBox_Export_RenderRealtime()` may call all C++ functions in the Jukebox API (see the C++ Specification for details), but it may not allocate or delete memory on the heap. Note that this also includes creating instances of standard C++ classes that allocate memory on the heap, such as `std::vector`.

`JBox_Export_CreateNativeObject()` is called by the host to process requests initiated from the RTC Lua functions `jbox.make_native_object_ro()` and `jbox.make_native_object_rw()`. Calls to `JBox_Export_CreateNativeObject()` are normally carried out with a lower priority than audio processing. The host will interrupt the call (and blow the fuse) if it does not return within a certain amount of time.

`JBox_Export_CreateNativeObject()` can only access data provided as input arguments and it cannot have any side-effects (except for allocating memory). Therefore, accessing the motherboard from `JBox_Export_CreateNativeObject()` is not allowed. `JBox_Export_CreateNativeObject()` may call all C++ functions in the Jukebox API except for the motherboard access functions (`JBox_LoadMOMProperty()`, `JBox_StoreMOMProperty()`, etc.) and `JBox_GetNativeObjectRW()`.

`JBox_Export_CreateNativeObject()` must always return a non-NULL value.

A data block allocated in

`JBox_Export_CreateNativeObject()` may only contain pointers to addresses within the data block itself, never to addresses outside the data block! This means that data

Important! allocated in `JBox_Export_CreateNativeObject()` must never contain pointers into sample or BLOB data, for example. Furthermore, the data block may never contain any `TJBox_Value` instances, nor any pointers to `TJBox_Value` instances.

The following example illustrates how you can use `JBox_Export_CreateNativeObject()` to create an instance of a C++ class that represents the 45, and how to use that instance to do audio processing in `JBox_Export_RenderRealtime()`.

`realtime_controller.lua:`

None

```
rtc_bindings = {
    {
        source = "/environment/system_sample_rate",
        dest = "/global_rtc/new_instance",
    },
}

global_rtc = {
    new_instance = function(source_path, new_sample_rate,
old_sample_rate)
        local new_no = jbox.make_native_object_rw("create",
{new_sample_rate})
        jbox.store_property("/custom_properties/instance", new_no);
    end,
}
```

C++ code:

C/C++

```
class CDSPInstance {
public: CDSPInstance(TJBox_Float64 iSampleRate);
public: void RenderBatch();
private:
    // Private state goes here.
    // Must not contain any TJBox_Value instances
    // or external pointers!
};

void* JBox_Export_CreateNativeObject(
    const char iOperation[],
    const TJBox_Value iParams[],
    TJBox_UInt32 iCount)
{
    JBOX_ASSERT(strcmp(iOperation, "create") == 0);
    JBOX_ASSERT(iCount == 1);
    JBOX_ASSERT(JBox_GetType(iParams[0]) == kJBox_Number);

    TJBox_Float64 sampleRate = JBox_GetNumber(iParams[0]);
    return new CDSPInstance(sampleRate);
}

void JBox_Export_RenderRealtime(
    void* iPrivateState,
    const TJBox_PropertyDiff /*iPropertyDiffs*/[],
    TJBox_UInt32 /*iDiffCount*/)
{
    JBOX_ASSERT(iPrivateState != NULL);
    CDSPInstance* instance = static_cast<CDSPInstance*>(iPrivateState);
    JBOX_ASSERT(instance != NULL);
    instance->RenderBatch();
}
```

C++ toolchain

The Jukebox C++ toolchain is based on a Jukebox-specific variation of the LLVM Clang compiler (see <http://llvm.org/>) and a collection of supporting tools. This toolchain is used to create LLVM bitcode, which is submitted to Reason Studios as part of the Universal 45. The bitcode is translated to binary format for the various supported native platforms (e.g., Windows and macOS) on the Reason Studios servers before it is being distributed to end-users. This means that the 45 is running platform-specific code on the end user's machine. To allow the code to run at the required speed for realtime operation, there are limits to the kinds of errors the realtime sandbox can detect.

These errors are caught in realtime (which blows the fuse):

- API errors (calling a toolbox function with invalid parameters)
- Access violations and other hardware exceptions
- Taking too long to respond to a callback from the host

However, during testing and validation, Reason Studios will build submitted 45's with extra instrumentation code that validates all memory references. This makes it possible to catch a larger amount of bugs during non-realtime stress tests. This means that a 45 built for testing (you select this when you upload the Universal 45 to the Reason Studios servers) will run at a slower speed than a 45 built for deployment.

C++ standard library

The Jukebox SDK provides a limited version of the C/C++ standard libraries. The limitations are mostly due to the requirements of the runtime environment (no direct access to files, etc.) and long term portability issues.

Memory alignment

Memory that you allocate with `malloc()` and `new` will be aligned to 16 bytes. However, certain toolbox functions will perform faster if the memory you provide to them is aligned on other address boundaries. See the Jukebox Scripting Specification and the C++ Specification documents for details.

Floating point contract

To ensure portability, robustness, and performance, Jukebox enforces a specific floating point contract. See the Jukebox Scripting Specification for details.

Tip!

Be careful about operations that depend on precision that lies on the border of the currently used floating point type! Do not compare floating point values for equality, etc. Be aware that single precision floats only have about 7 significant decimal places.

Latency

If a 45 introduces DSP latency, for example by doing buffered processing such as FFT, the 45 should tell the host what the current latency is for each relevant audio and CV output. The current latency is a per output setting that specifies the latency in frames. All output sockets have a `rtc_owner` property called `dsp_latency` for this purpose. Note that both audio and CV sockets can have DSP latency specified and that latency can differ between outputs.

It is allowed to initiate a change of the latency from within `JBox_Export_RenderRealtime()` (this requires defining a `rt_owner` property in the MOM and creating a binding to that property in `realtime_controller.lua`), but note that doing so may introduce audible clicks or breakups in the sound output.

Silence detection

All 45:s must handle *silence*, i.e., stop rendering when all inputs are silent (or shortly after inputs become silent). An audio input is considered to be silent when all values in the buffer are smaller than or equal to the C++ constant `kJBox_SilentThreshold`. If a 45 does not touch any of the values in the DSP buffer of the `buffer` property in an audio output socket during a batch, the host considers the socket to be *silent*. Note that filling the buffer with zeros does not constitute silence in this sense. If the 45 does choose to write to the DSP buffer, then all values must be written to. It is not permitted to write `nan:s` or `inf:s` to the buffer. See the "Silence Detection Demo" example for details.

Performance optimization

It is important that Rack Extensions do not waste computational resources. A device that uses excessive computational resources may be rejected by Reason Studios.

Here are some basic recommendations for improving performance:

- Avoid computational "spikes", i.e., avoid carrying out complex computations in some batches only. Instead, try to spread out complex computations across multiple batches.
- Each MOM load/store introduces a small overhead, so try to minimize the number of accesses to motherboard properties.
 - Do not create GUIs that require updating large numbers of properties (i.e., rt-owned properties that drive `sequence_meter` or `value_display` widgets) per batch. 50 such property updates per batch is considered expensive, and more than 100 updates per batch is unacceptable in most cases. Also note that the host GUI is always updated at a considerably lower rate than the audio Realtime, so it is not necessary to update properties that drive the GUI every batch. A new alternative for display updates available in SDK 2.5 is the RT owned string. It can be used to send arrays from the DSP a custom display.
 - Do not change RT strings every batch. RT strings are read by a custom display, which will only be updated about 20 times per second. Too frequent RT string updates will blow the fuse.
 - CV values are only propagated through the engine once per batch, so you should avoid writing to CV outputs more than once per batch. All updates except for the last one carried out in the batch will be ignored by the Jukebox engine.
 - Avoid subscribing to property changes that your device ignores.
 - Define as few custom properties as possible in the MOM.

The DSP meter in Reason

The built-in DSP meter in Reason is a useful indicator of the performance of your Rack Extension, but you should be aware of that it does not measure the same thing as a code profiler does.

Reason renders sound in batches of 64 samples. The maximum time (in seconds) that may be spent on computing sound for a batch - let us denote it by $T_{\max}$ - depends on a number of factors, including the current sample rate, the audio card buffer size, and the CPU and RAM specifications of the host computer. Let us say that the time Reason actually spends computing audio for a batch is T seconds. If $T > T_{\max}$, Reason does not have enough computational resources to compute audio fast enough, and the result will be stutter in the output. Conversely, if $T < T_{\max}$, Reason has time to "idle" until the next batch has to be computed.

The DSP meter in Reason is essentially a visualization of $(T_{\max} - T)$: it tells the user how much "idle-time" Reason has available. Double-clicking the DSP meter in Reason Recon brings up more detailed information. But even if there is only one Rack Extension in the rack, the DSP meter value invariably involves many other performance factors too. Therefore, it can be difficult to "separate" performance issues that stem from the Rack Extension from issues that stem from other parts of Reason using the DSP meter alone.

Code profiling

To measure how much time your Rack Extension actually spends in the various functions of your Realtime code, you must use a code profiler tool. When profiling your 45 it is recommended to build a local 45 with compiler optimizations turned on. You can do that by building your local 45 in the Deploymnnet configuration.

Profiling on macOS

To profile on macOS, do this:

- Build your local 45 in the Deployment configuration. You can do this from Xcode by selecting "Build For" and then "Profiling" from the "Product" menu.
- Start Recon.
- In Xcode, click the "Xcode" menu, select "Open Developer Tool", and then "Instruments".
- In Instruments, select the "Time Profiler" template, attach to the Reason Recon process, and press the record button.
- Create an instance of your device and interact with it.
- In Instruments, press the stop button. Select the part of the timeline graph where you interacted with your device.
- In the Call Tree option view, check the "Hide System Libraries" checkbox. (This removes irrelevant items from the analysis view.)
- To narrow the analysis view to show code paths that involve your DSP code, enter "JBox_Export_RenderRealtime" in the "Involves Symbol" search text box.
- Expand the call stacks to drill down to your code functions (you can also check the "Invert Call Stack" to make them appear at the top).

Profiling on Windows

The following steps describes how to profile in Visual Studio 2017 Professional:

- Build your local 45 in the Deployment configuration.
- Start Recon if necessary and create an instance of your device.
- In Visual Studio open the "Performance Profiler" from the Debug menu.
- Press "Change Target" (The big button with an icon) and select "Running Process".
- Find Reason Recon and press "Select".
- Back in the "Performace Profiler" make sure "CPU Usage" is selected and nothing else.
- Now press the "Start" button!
- Interact with you device.
- Press "Stop collection to view CPU usage data" and your profiling data will appear.

Receiving and sending notes

All Jukebox devices are able to receive MIDI notes, with one exception (see "Combinator issues" below). There are 128 MIDI notes [0, 127]. By default, note number 69 corresponds to A440 (i.e., 440 Hz for Middle-A on a standard piano keyboard), but this can be offset by +/- one semitone by the user via the Master Tune setting (see below).

Each MIDI note has a corresponding Jukebox property in the

```
/note_states
```

property set object. The property key for each note is set to the note number, i.e.,

```
/note_states/0  
/note_states/1  
/note_states/2
```

```
...  
/note_states/127
```

The property tag for each note is also set to the note number, so the simplest way to detect note changes is by doing something similar to

```
void CheckForNoteOn(  
    const TJBox_PropertyDiff iPropertyDiffs[],  
    TJBox_UInt32 iDiffCount)  
{  
    TJBox_ObjectRef noteStates = JBox_GetMotherboardObjectRef("/note_states");  
    for (TJBox_UInt32 i = 0; i < iDiffCount; ++i)  
    {  
        if (iPropertyDiffs[i].fPropertyRef.fObject == noteStates)  
        {  
            TJBox_Tag midiNoteNumber = iPropertyDiffs[i].fPropertyTag;  
            TJBox_Float64 velocity =  
                JBox_GetNumber(iPropertyDiffs[i].fCurrentValue);  
            ...  
        }  
    }  
}
```

As the example above illustrates, the property value is the velocity of the note. The velocity is a discrete value in the range [0, 127], where a value of 0 means note off and a value of 127 means maximum velocity.

As of Jukebox version 3.0, there is also a C++ toolbox function for converting property values to note events:

```
...  
if (iPropertyDiffs[i].fPropertyRef.fObject == noteStates)  
{  
    const TJBox_NoteEvent& noteEvent = JBox_AsNoteEvent(iPropertyDiffs[i]);  
    ...  
}
```

Note that in order to receive note update notifications, you must subscribe to `note_state` changes, i.e., you must say

```
rt_input_setup = {
    notify = {
        "/note_states/*",
    }
}
```

in `realtime_controller.lua`.

Tip!

Although tags must be unique for all properties within a property set object, it is possible for two properties in *different* objects to have the same tag value. Be careful so you do not confuse properties in the `note_states` object with properties in the `custom_properties` object!

Player devices have the ability to send note events using the function `JBox_OutputNoteEvent`.

Player devices, for certain product designs, should forward incoming MIDI note events to the next device (another Player or the instrument). Player devices in off mode should always forward incoming note events. The following example shows how to forward incoming MIDI note events:

```
void ForwardNoteEvents(
    const TJBox_PropertyDiff iPropertyDiffs[],
    TJBox_UInt32 iDiffCount)
{
    TJBox_ObjectRef noteStates = JBox_GetMotherboardObjectRef("/note_states");

    for (TJBox_UInt32 i = 0; i < iDiffCount; ++i)
    {
        if (iPropertyDiffs[i].fPropertyRef.fObject == noteStates)
        {
```

```

        const TJBox_NoteEvent& noteEvent = JBox_AsNoteEvent(iPropertyDiffs[i]);
        JBox_OutputNoteEvent(noteEvent);
    }
}

```

Tuning

To convert from MIDI note to frequency (Hz), you can use this formula:

```
TJBox_Float32 frequency = std::powf(2.0f, (note - 69.0f) / 12.0f) * 440.0f;
```

To convert from frequency (Hz) to MIDI note, you can use this formula:

```

TJBox_Float32 log2(TJBox_Float32 n) {
    return std::logf(n) / std::logf(2.0f);
}

```

...

```
TJBox_Float32 note = 69.0f + 12.0f * log2(frequency / 440.0f);
```

Master Tune

Reason follows the convention that the octave is divided into 12 semitones of 100 cents each. The Master Tune setting offsets the frequency of A440 (MIDI note 69) in cents. It is assumed that all other notes are offset accordingly.

Important! All instruments must respond to the Master Tune setting.

The `/environment/master_tune` property (`kJBox_EnvironmentMasterTune` property tag) contains the Master Tune setting in the range [-100,100].

To apply master tuning, you can do this:

```
TJBox_Float64 mtProp = ...; // Value of "/environment/master_tune" property
TJBox_Float32 note = ...;   // MIDI note to play

TJBox_Float32 mt = static_cast<TJBox_Float32>(mtProp) / 100.0f;
TJBox_Float32 frequency =
    std::powf(2.0f, ((note + mt) - 69.0f) / 12.0f) * 440.0f;
```

or you can modify the frequency after it has been computed:

```
TJBox_Float64 mtProp = ...; // Value of "/environment/master_tune" property
TJBox_Float32 frequency = ...; // Frequency to be modified by master tune

TJBox_Float32 mt = static_cast<TJBox_Float32>(mtProp) / 100.0f;
const TJBox_Float32 kSemiTone = std::powf(2.0f, 1.0f / 12.0f);
frequency *= std::powf(kSemiTone, mt);
```

Combinator issues

Reason's Combinator device allows the user to group together different devices into a single "meta-instrument".

45:s where the `device_type` is set to "instrument" in `info.lua` will receive notes in Combinators by default. 45:s where `device_type` is set to one of the other values do not receive notes in Combinators by default. The user can override this by unchecking the "Receive Notes" checkbox in the Combinator programmer, unless the `accepts_notes` flag has been set to `false` in `info.lua`. If `accepts_notes` is `false`, the 45 will never be able to receive notes inside Combinators, and the "Receive Notes" checkbox will be disabled in the Combinator programmer.

Sequencer integration

As of version 3.0, Jukebox supports sequence-based and pattern-based Rack Extensions. A 45 can receive information about the current master transport play position, tempo, loop positions, bar start position, and whether the sequencer is running or not. This makes it possible to create Rack Extensions with tempo-synced LFOs, patterns, and similar.

However, it is very difficult or even impossible to create Rack Extensions that react correctly to the *absolute* master transport position.

The 45 can obtain the master transport play position at the start of each batch (of 64 frames). The reason is, quite simply, that `JBox_Export_RenderRealtime()` is only called once per batch, and that any property value that it obtains from Jukebox (with the exception of notes) is from a "snapshot" of the MOM as it was at the start of the batch.

The notes that are received within a batch are normally located just to the right of the master transport play position in the sequencer. But if the sequencer happened to reached a loop point during the batch, or if the user has moved the transport position explicitly, the play position may actually have jumped during the batch. If so, the notes reported to `JBox_Export_RenderRealtime()` may actually originate from completely different locations in the sequencer. Indeed, although unlikely, the play position may actually have jumped more than once during the batch.

As a result, a 45 can only "react" to transport play position jumps "after the fact". In practice, this often leads to artefacts like dropped notes or "untight" triggering of notes.

Pattern and Pattern Clips

Devices with a built-in sequencer can have different patterns (separate sequences or rhythms). To allow the user to control this, Jukebox has special Pattern Automation properties and clips. This system is based on the original pattern system in for example Redrum, and supports up to 32 patterns. See the Jukebox Scripting Specification for details.

CV standard in Reason

CV sockets contain a floating point value that represents the CV signal for the current batch. For compatibility with other Reason devices, it is important to follow the CV level standards used by Reason.

TYPE	NOMINAL RANGE	DESCRIPTION

Unipolar	[0.0, 1.0]	Reason standard unipolar signals, as produced by the Matrix in the corresponding mode.
Bipolar	[-1.0, 1.0]	Reason standard bipolar signal, as produced by the Matrix in the corresponding mode.
Thor Bipolar	[-0.5, 0.5]	Thor internally uses this CV range for bipolar values, which is exposed on CV outputs when connected to such sources.
Note/Gate (velocity) CV	[0.0, 1.0]	This is much like Unipolar, but quantized to 127 steps to fit the legacy note and velocity resolution of MIDI.
Thor frequency control	[-1.0, 1.0]	7 octaves / volt pitch control signal.

The range specified for a CV type represents the signal level needed for nominal full range modulation of the corresponding parameter. That is, a full range LFO producing standard Matrix bipolar output should result in a signal in range [-1.0, 1.0] on the corresponding CV output.

Typically, the nominal full range corresponds to 100% modulation, but it is up to the receiving device to decide what levels are usable. A CV mixer might allow signals greater than 1.0 for instance, and some devices might allow modulating filter frequencies to 130%.

Devices with CV inputs are responsible for clamping incoming CV signals to a useful range before mapping to internal parameters.

It is common for CV inputs to have a trim knob that can be used to scale the signal before clamping and mapping. The scaling applied by trim knobs ranges from 0.0 to 1.0. By default, trim knobs are set to 1.0.

As described above, note and velocity values in Jukebox have the same range as in the MIDI standard, [0, 127]. To convert from a note or velocity value to CV, divide the value by 127:

```
TJBox_Float32 noteCV = noteValue / 127.0f;  
TJBox_Float32 velocityCV = velocity / 127.0f;
```

For historical reasons, the conversion from CV to note or velocity must be done slightly differently. Please use the following conversion to ensure consistency with Reason's legacy devices:

```
// Truncates a float value to an integer and  
// limits the result to [lower, upper]  
TJBox_Int32 clamp(TJBox_Float32 in, TJBox_Int32 lower, TJBox_Int32 upper) {  
    ...  
}  
  
...  
  
TJBox_Int32 noteValue = clamp(noteCV * 127.0f + 0.1f, 0, 127);  
TJBox_Int32 velocity = clamp(velocityCV * 127.0f, 0, 127);
```

Velocity CV is commonly called "Gate" since it is used for monophonic triggering as well as transferring velocity information. Jukebox devices that react to gate should trigger on the positive flank of the velocity

signal's corresponding integer value. That is, when the value of the gate CV changes from below 1/127 to 1/127 and above.

Important! Instrument devices must retrigger notes if the note CV changes while the gate is open (1/127 and above).

Here is a code snippet that illustrates how notes are triggered in the built-in Reason devices:

```
// Only call this function when GateInputCV is connected.
void HandleGateCVInput(
    TJBox_Float32 iNoteCVInput,           // Value of the note CV input
    TJBox_Float32 iGateInput,             // Value of the gate CV input
    TJBox_Bool iNoteCVInputIsConnected,   // Is note CV input connected?
    TJBox_Int32& ioPrevNote,              // IN/OUT: Previous note value
    TJBox_Int32& ioPrevVelocity)         // IN/OUT: Previous velocity value
{
    TJBox_Int32 velocity = static_cast<TJBox_Int32>(iGateInput * 127.f);
    if (velocity < 0) {
        velocity = 0;
    }
    else if (velocity > 127) {
        velocity = 127;
    }

    // If only gate is connected, always use note 64.
    TJBox_Int32 newNote = 64;

    TJBox_Bool trigged=false;
    if (ioPrevVelocity > 0 && velocity == 0) {
        HandleAllNotesOff();    // Kill all notes trigged from CV-Gate.
    }

    if (iNoteCVInputIsConnected) {
        // Slightly odd rounding for compability. See text above.
        newNote = static_cast<TJBox_Int32>(iNoteCVInput * 127.f + 0.1f);
        if (newNote < 0) {
            newNote = 0;
        }
        else if (newNote > 127) {
            newNote = 127;
        }

        // Trig when note is changing, even if no new gate.
        if (newNote != ioPrevNote) {
            if (velocity > 0) {
                ioPrevNote = newNote;
            }
        }
    }
}
```

```

        HandleNoteOn(newNote, velocity);
        trigged = true;
    }
}

// Normal trig from gate.
if (ioPrevVelocity == 0 && velocity > 0 && !trigged){
    ioPrevNote = newNote;
    HandleNoteOn(newNote, velocity);
}
ioPrevVelocity = velocity;
}

```

The standard for pitch control in Thor is slightly different, and is used internally for both controlling the pitch of oscillators and the cutoff frequency of filters. Thor uses a 7 octave per volt mapping where zero volts corresponds to middle C.

```

const TJBox_Float32 kMiddleC = 261.6256f;
TJBox_Float32 frequencyHz = std::powf(128.0f, frequencyCV) * kMiddleC;

```

Audio reset requests

Sometimes, the host may want to re-initialize the 45 to a "silent" state. The most common situation is just before a song or loop is exported to an audio file. In such situations, any still-sounding reverb tails or synthesizer voices should be silenced immediately. (Otherwise, the tail or voice release will be heard at the beginning of the exported audio.) Note that the host may request an audio reset in other situations, such as when the device is first created, or when the sample rate changes.

To signal that a 45 should reset itself, the host sets the property `/transport/request_reset_audio` to new value. The property is an integer counter with an unspecified non-negative initial value. The 45 should reset itself at the batch frame where the property was updated, and then continue processing as normal for the following frames in the batch.

GUI specification

Important!

The original GUI format, based on 3D assets is no longer supported since SDK version 4.2. If you are updating a device built using the old 3D format, you can convert it to the current format using the 3D to 2D conversion tool, found on the SDK downloads page link: [./downloads/rack-extension-sdk](#)

The [GUI Designer Manual](#) contains detailed information on how to create GUIs for Jukebox. The format of the `hdgui_2D.lua` script is specified in the [Jukebox Scripting Specification document](#)

Custom displays

Jukebox SDK 2 introduced a new widget type, *custom display*. A custom display is a flat rectangular surface on the panel that the 45 can draw into using Lua code. The 45 can also receive user gestures via the custom display widget and use Lua code to turn those gestures into Motherboard property changes.

Once a display has been bound to a property it will receive a notification whenever that property is changed (e.g., via another widget or automation). A custom display may only modify properties that it is bound to.

You can bind properties from all four owner scopes to a custom display (`gui_owner`, `document_owner`, `rtc_owner`, and `rt_owner`), but the display can only write to properties in the `gui_owner` and `document_owner` scopes. (The display can read the value of all properties that it is bound to.)

Motherboard snapshots for custom displays

When a custom display is redrawn, it is sent a snapshot of the Motherboard state, i.e., it is sent a local copy of the values of all properties that are bound to the display. Jukebox does not guarantee any specific update rate for custom displays, nor does it guarantee that all values in the Motherboard snapshot originated in the same `JBox_Export_RenderRealtime()` batch.

For example, if an `rt_owner` property is bound to a custom display and it is modified by `JBox_Export_RenderRealtime()`, the host flags the display as dirty (assuming the modified value is different from the last time the display was redrawn). The property may be modified several times before the host has time to actually redraw the display. The snapshot of the Motherboard is taken just before the display is redrawn.

Drawing into custom displays

A custom display has a virtual "video memory" that consists of a rectangular buffer of "virtual pixels" that the 45 can modify. The width and height of the video memory cannot change at run-time. When the display (or a part of it) must be repainted, the corresponding area in the video memory is cleared to black, or to a (static) bitmap that the 45 provides. It is not possible to read from the video memory.

To create a custom display, you first need to add a flat, rectangular surface to the device's front or folded front panel (custom displays cannot be added to the back panels), and assign a custom display material to it (see the GUI Designer Manual and Jukebox Scripting Specification for details). Then, you create a custom display widget in `hdgui_2D.lua` similar to this:

None

```
jbox.custom_display{
    graphics={
        main_node="/root/Display/Background",
    },
    display_width_pixels = 200,
    display_height_pixels = 160,
    invalidate_function="invalidate",
    draw_function="draw",
    gesture_function="gesture",
    values={
        "/custom_properties/property_1",
        "/custom_properties/property_1"
```

```
    },  
  },
```

The `graphics` entry is the same as all other widgets, except that it cannot have `boundaries` or `hit_boundaries` modifiers.

The `display_width_pixels` and `display_height_pixels` entries are the dimensions of the "video memory".

The `invalidate_function` is the name of an Lua function in a file called `display.lua` (that you provide). The host will call this function whenever one or several of the properties that have been bound to the display have changed. The invalidate function is responsible for informing the host about which areas of the display that need to be repainted, based on the property changes. This is done by calling the Lua toolbox function `jbox_display.invalidate()` once for each rectangle in the display that needs to be redrawn. For example:

None

```
function invalidate(property_values, last_property_values, display_info)  
  -- display_info contains information about the display  
  local w = display_info.width  
  local h = display_info.height  
  
  -- check if a property value has changed and invalidate the display  
  if property_values[1] ~= last_property_values[1] then  
    local rect = { left = 0, top = 0, right = w / 2, bottom = h / 2  
  }  
  
    jbox_display.invalidate(rect)  
  end  
end
```

```
...  
end
```

If the invalidate function does not make any calls to `jbox_display.invalidate()`, the host will assume that the display does not have to be repainted.

Tip!

Providing an invalidate function is optional, but it is important to remember that it is a useful tool for performance optimization. If your 45 does not provide an invalidate function, the host has to repaint the entire display whenever a display-bound property changes.

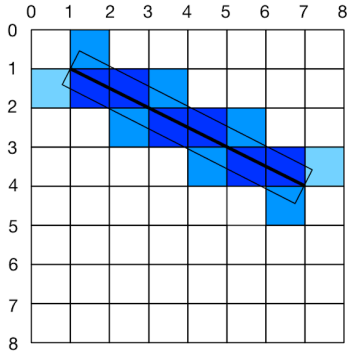
The `draw_function` is the name of a Lua function (also in `display.lua`) that the host calls if the display must be repainted. The host passes the rectangle that needs to be repainted (the "dirty rect") to the function. Note that the dirty rect may not be the same rectangle that the `invalidate_function` provided (the entire display has to be repainted when it is drawn for the first time, for example). The display function is also provided with the current values for all properties that are bound to the display. For example:

None

```
function draw(property_values, display_info, dirty_rect)  
    -- use a property value to compute a color and fill the dirty rect  
    local color = {r = property_values[1] * 255, g = 0, b = 0}  
  
    jbox_display.draw_rect(dirty_rect, color)  
end
```

The invalidate function and the draw function may not have any side-effects, except for updating the virtual video memory. You cannot pass information between the invalidate and draw functions, or between them and the rest of Jukebox, except via Motherboard properties.

All drawing commands create antialiased primitives. For example, using the `jbox_display.draw_line()` toolbox function to draw a rectangle that is one pixel thick between two 2D coordinates will result in something similar to this:



The pixels that the (mathematical) 1-pixel-wide rectangle covers will be colored according to the area that is covered by the rectangle.

Jukebox does not guarantee that the results of a drawing command will be pixel-equivalent on different hosts or operating systems.

Tip! To draw "non-antialiased" horizontal and vertical lines, add 0.5 to all x and y coordinates.

All drawing commands accept RGBA colors in the sRGB color space. Alpha blending works like the "Normal" layer blend mode in Photoshop, i.e.,

$$\text{RGB} = \text{RGB}_{\text{in}} \times A_{\text{in}} + \text{RGB}_{\text{pixel}} \times (1 - A_{\text{in}})$$

where RGB is the new color of a pixel in the video memory, RGB_{pixel} is the current color of that pixel, and RGB_{Ain} is the color that you pass to the drawing toolbox function.

Adding glass

Most displays look more realistic if there's a glass pane in front of them. To add a glass in front of a custom display, position a transparent geometry in front of the display. Then create a static decoration widget and point to the glass geometry in its widget definition. See the GUI Designer Manual and Jukebox Scripting Specification for details.

Responding to user gestures

As mentioned above, it is possible to configure a custom display to receive user gestures. The display does not work directly with window system events, this is the host's responsibility. Instead, the host will interpret incoming window system events as more abstract *gestures* that it then forwards to the custom display.

To receive user input, you provide the host with the name of a *gesture recognizer* function, which is a Lua function that you define in `display.lua`. If the gesture recognizer function decides that the custom display should respond to the gesture, it returns a *gesture definition table*. (If not, the gesture recognizer function must return `nil`.)

The gesture definition table contains two entries:

1. The name of one or several *gesture handler callback*s that the host should call as the gesture evolves.
2. A table containing custom data that the 45 can use to configure the gesture, or for passing information between the gesture handler callbacks.

The gesture handler callbacks are allowed to update any number, string, or boolean property in the `gui_owner` or `document_owner` scope in the `custom_properties` property set object, assuming that the property has been bound to the custom display. (Performance properties cannot be modified by custom displays.)

Here is an example. We begin by specifying a gesture recognizer function in `hdgui_2D.lua`:

```
None
jbox.custom_display{
    ...
    gesture_function="gesture",
    ...
},
```

In this example, the display always responds to tap/clicks (it doesn't use any custom data):

None

```
function gesture(property_values, display_info, gesture_start_point)
  return {
    handlers = {
      on_tap = "handle_tap"
    }
  }
end
```

The gesture handler, `handle_tap`, updates one of the properties that are bound to the custom display:

```
function handle_tap(property_values, display_info, gesture_info, custom_data)
  return {
    gesture_ui_name = jbox.ui_text("click")
    property_changes = {
      [1] = gesture_info.current_point / display_info.width
    },
  }
end
```

The `gesture_ui_name` is a localized string that the host uses to present the gesture to the user, e.g., in its undo system.

The details of how undo works is host-specific. Here is how it currently works in Reason:

- All property changes are always recorded in the undo system, but only changes for `document_owner` properties are actually presented to the user.
- One complete gesture always corresponds to one undo step.

- If you return different `gesture_ui_name` strings from the gesture handler callbacks, the first one returned is used to identify the gesture in Reason's undo menu item.
- If you change one or several `document_owner` properties in the gesture handler callbacks, the gesture becomes visible in Reason's undo system.
- If you change one or several `document_owner` properties and one or several `gui_owner` properties in the gesture handler callbacks, the gesture becomes visible in Reason's undo system.
- If you only change `gui_owner` properties in the gesture handler callbacks, the gesture is not visible in Reason's undo system. Any `gesture_ui_name` string returned is ignored.

Introduction

Jukebox is a technology from Reason Studios for developing portable, robust and long-lived software instruments and effects. It is a central part of the Reason Studios Rack Extensions system, which makes it possible for Reason Studios and third parties to create and distribute devices for the Reason Studios Reason rack.

This document contains the specification and contracts for all scripts that are part of a Jukebox Rack Extension. The specification and contracts for the Jukebox C++ API can be found in a separate document.

General information

Important! All scripting files, including Lua files and XML-based graphics resources, must be encoded as ASCII or as UTF-8 Unicode without BOM.

info.lua

The `info.lua` file contains the general configuration of the Rack Extension.

All configuration attributes are defined as global variable assignments, e.g.,

None

```
format_version = "1.0"  
product_id = "se.propellerheads.SimpleInstrument"  
accepts_notes = true  
automation_highlight_color = {r = 60, g = 255, b = 2}
```

Some attributes are generic and are valid for all hosts (e.g., `product_id`), whereas some attributes are relevant only for specific hosts, such as Reason.

ATTRIBUTE	DESCRIPTION
<code>format_version</code>	File format version of the <code>info.lua</code> file. Must be "1.0". Required.
<code>product_id</code>	Product identifier. Required. Must be unique and follow the reverse domain notation convention, e.g., <code>"se.propellerheads.SimpleInstrument"</code> . The last part of the identifier must match the name of the DLL or dylib that implements the 45, e.g., <code>SimpleInstrument.dll</code> . The only characters that are allowed are alphanumeric characters and the dot (.) and underscore (_) characters. The identifier is used in Reason Studios Rack Extension repository and databases. The identifier is never shown in the Shop. The identifier can not be changed once the product has been uploaded to the Reason Studios build server.

version_number	The version of the device. Required. Must have the format <code>N.N.NbN</code> (e.g., <code>1.0.0b10</code>), <code>N.N.NdN</code> (e.g., <code>1.0.0d10</code>) or <code>N.N.NfN</code> (e.g., <code>1.0.0f10</code>) where <code>d</code> indicates testing/debug versions, <code>b</code> indicates beta versions, and <code>f</code> indicates release candidates. Do not use leading zeroes: <code>1.0.01f1</code> , for example, is not a valid version number. It is recommended to use the major number (<code>N.N.N</code>) to indicate paid upgrades, the minor number (<code>N.N.N</code>) to indicate feature additions, and the revision number (<code>N.N.N</code>) to indicate bug fixes or minor updates.
----------------	--

manufacturer	The name of the manufacturer. Required. Shown in the device palette in Reason. Can be changed by submitting an updated version of the product. Contact Reason Studios if you need update the manufacturer name shown in the Reason Studios Shop.
--------------	--

long_name	The name of the device. Required. Must not be longer than 40 characters. Shown in the device palette and Create menu in Reason. Also shown in the Shop. Must follow the conventions described below. Contact Reason Studios if you need to change the name of a device after it has been released.
-----------	--

medium_name	Shorter version of the device name. Required. Used in situations where Reason "talks" about the device, e.g., "Undo create Synthesizer". Must not be longer than 20 characters.
-------------	---

short_name	Short version of the device name. Required. Used for auto-naming new instances of the device in Reason, e.g. "Synth 1", "Synth 2". Must not be longer than 10 characters.
------------	---

<code>supports_patches</code>	Set to <code>true</code> if the device should be able to load/save patches. Required. If <code>true</code> , the device must have a patch browser and patch browse button group in the GUI, and it must have a default patch.
<code>default_patch</code>	The name of the patch to load by default (if the device supports patches). Optional. Must be non-empty if the device supports patches. If the device does not support patches, do not include this attribute.
<code>device_type</code>	Specifies the device type. Required. Allowed values are <code>"instrument"</code> , <code>"creative_fx"</code> , <code>"studio_fx"</code> , <code>"helper"</code> , and <code>"note_player"</code> . <code>"creative_fx"</code> and <code>"studio_fx"</code> automatically receives the host property <code>/custom_properties/builtin_onoffbypass</code> in their motherboard <code>"note_player"</code> automatically receives the host property <code>/environment/player_bypassed</code> . See section [Host_properties] for more information. The <code>device_type</code> also affects your options for automatic routing (see section [Automatic_routing]). In Reason, the <code>device_type</code> determines in which menu in the device palette that device is placed. <code>"instrument"</code> devices receive notes in Combinators by default in Reason, while other device types do not.
<code>device_categories</code>	Specifies the categories for the device, used by the device palette in Reason. Required. Specify one or more categories as a list of strings, e.g. { <code>"Drums"</code> , <code>"Sequencer"</code> }. For the full list of available categories and how to use them, see section [Device_category_guidelines] .
<code>device_height_ru</code>	The device height in Rack Units. Required. Must match the panel graphics in the GUI.

<code>automation_highlight_color</code>	Reason-specific. Required. Color of automation indicators in Reason. Should be set to <code>{r = 60, g = 255, b = 2}</code> . If you want to use a different color, please contact Reason Studios developer support first.
---	--

<code>auto_create_track</code>	Reason-specific. Required. If <code>true</code> , create a sequencer track for the device automatically when it is instantiated. If <code>false</code> , the user has to create the track manually via the "Create Track for ..." menu option.
--------------------------------	--

<code>accepts_notes</code>	Reason-specific. Required. If <code>true</code> , it is possible to create a note lane for the device and the user can choose whether the device should be able to receive notes inside Combinators. If <code>false</code> , it is impossible to create a note lane for the device, and the device is unable to receive notes inside Combinators. Devices that do not respond to notes should set <code>accepts_notes</code> to <code>false</code> and <code>auto_create_note_lane</code> to <code>false</code> .
----------------------------	---

<code>auto_create_note_lane</code>	Reason-specific. Required. If <code>true</code> , then a note lane will be automatically created in the track for the device. If <code>false</code> , the user must explicitly create a note lane in the track (note that this option is disabled if <code>accepts_notes</code> is <code>false</code>). Devices that respond to notes should set <code>auto_create_note_lane</code> to <code>true</code> .
------------------------------------	---

<code>supports_performance_automation</code>	Reason-specific. Required. If <code>true</code> , the user can choose to record automation into note clips. If <code>false</code> , the device will always record automation into separate lanes, even if the user has enabled "Record Automation into Note Clip" in the Options menu. Devices that respond to notes should set <code>supports_performance_automation</code> to <code>true</code> . Devices that do not respond to notes should set <code>supports_performance_automation</code> to <code>false</code> .
--	--

Device naming conventions

The device names in the Create menu in Reason need to be consistent and easy to understand. Therefore, all Rack Extensions should be named like this:

[product name] [short description]

For example: *SubTractor Analog Synthesizer*

Each word should be capitalized. The manufacturer name should not be part of the name. Reason Studios may accept names that diverge from this format if they are well motivated.

Device category guidelines

Below is a list of the available device categories along with guidelines on when to apply each one. Please note that the device categorization is subject to acceptance testing by Reason Studios and may be rejected if the below guidelines are not adhered to.

CATEGORY	USED FOR (TYPE)	MEANING
Amps	Effects	Amp emulators like guitar amps etc.

Arpeggio	Utilities, Players, (possibly Instruments)	Arpeggiators. Could be used for an instrument if its functionality is centered around a built-in arpeggiator (e.g. a CV arpeggiator configured as an instrument).
Audio Meter	Effects, Utilities	Meters or scopes accepting audio signals
Audio Mixer	Effects, Utilities	Device for mixing/merging multiple audio signals
Bass	Instruments, Effects	An instrument specialized on bass sounds (e.g. Reason Electric Bass, Monotone) or an effect specifically for bass (e.g. a bass amp). An allround synth should typically not be categorized Bass even though it can do bass sounds.
Chords	Utilities, Players, Instruments	Devices that generate chords in the form of MIDI, CV or possibly audio. Not a synth that is good for playing chords on; chords should be central to the function.

CV Meter	Utilities	Meters or scopes accepting CV signals.
CV Processor	Utilities, Players	Device that takes CV input, changes it and outputs CV
CV Source	Utilities, Players, possibly Instruments	Device whose main purpose is to generate CV, both static and moving. Matrix and Pulsar are both CV sources (but Thor is not; it can do it but it's not a main purpose)
Delay	Effects	Audio delay/echoes. Note that an instrument does not get "Delay" category even though it has delay onboard. That goes for all effects.
Dist	Effects	Distortion, bit crushing etc. An amp emulator does not have to be "Dist" as well, unless it has specific fuzz effects or something.
Drums	Instruments, Utilities, Players.	Instrument focused on drum and percussion sounds. Players and utilities which generate drum beats (MIDI or CV).

Dynamics	Effects, possibly Utilities	Effects that work with levels (compressors, de-essers, noise gates, pumpers etc). There's one or two utilities doing gain reduction CV from audio inputs; these could also be Dynamics
EQ	Effects	Adjusting levels of frequency bands. Multi-band splitters could go here as well.
Filter	Effects	Typically LPF/HPF/BPF/Notch filters.
Guitar	Instruments, Effects, possibly Players	Instruments focused on guitar sounds. Effects specifically for guitars (e.g. guitar amps). Possibly Players that emulate guitar playing techniques.
Keys	Instruments, possibly Players	Instruments focused on piano, electric piano and organ sounds. Possibly players that emulate keyboard playing techniques.
Loops	Instruments, Effects	Instruments that play back loops, like Dr Octo REX. Also effects that create loops, like Beat Loopers/Slicers.

Misc	all types	Stuff that don't fit in with the categories. Some display-only REs go here, like name label holders.
Modulation	Effects, Utilities	Effects like Chorus, Phasers, Tremolos, Ring Modulation etc. Utilities that generate modulating CV (like LFOs, Envelopes) go here too.
Note FX	Players	Players that manipulate incoming MIDI notes. Note Echo, Scales & Chords.
Orchestral	Instruments	Instruments with some focus on strings, brass, orchestral stuff.
Pitch	Effects	Effects that change the pitch of audio. Autotune, harmonizers.
Reverb	Effects	Reverb effects.
Samples	Instruments	Instruments based on sample loading or sample playback. Both samplers like NN-XT and romplers like ID8. Scenic is a synth but also gets "Samples" because you can load a sample here. Malström is just a synth, because even though it uses

built-in samples, it behaves and feels like a synth.

Sequencer

Utilities,
Players,
Instruments

Mainly devices that generate CV or note sequences. Occasionally instruments where the built-in sequencer is central to the operation (e.g. Redrum).

Stereo

Effects,
Utilities

Effects that manipulate stereo/panning. Can be sorted as utilities sometimes.

Synth

Instruments

Yeah, synths.

Vocoder

Effects,
possibly
Instruments

Vocoders. They can be sorted as effects or instruments.

motherboard_def.lua

The `motherboard_def.lua` file contains declarations for all properties in the motherboard of a Rack Extension.

It has the following format:

None

```
format_version = "3.0"

custom_properties = jbox.property_set{
    ...
}

midi_implementation_chart = {
    ...
}

remote_implementation_chart = {
    ...
}

ui_groups = {
    ...
}

cv_inputs = {
    ...
}

cv_outputs = {
    ...
}

audio_inputs = {
    ...
}

audio_outputs = {
    ...
}

-- optional table of user samples
user_samples = {
    ...
}
```



```
-- optional table of patterns
patterns = {
    ...
}

-- calls to toolbox functions to configure automatic routing
```

ENTRY	DESCRIPTION
<code>format_version</code>	File format version of the <code>motherboard_def.lua</code> file. Should be "3.0" for this version of the SDK. Required.
<code>custom_properties</code>	The set of custom properties for this device. Optional. See section [Custom_property_definitions] for details.
<code>midi_implementation_characteristics</code>	MIDI configuration for the 45. Optional. See section [MIDI_configuration] for details.
<code>remote_implementation_characteristics</code>	Remote™ configuration for the 45. Optional. See section [Remote_configuration] for details.
<code>ui_groups</code>	Custom property groups (for UI purposes). Optional. See section [UI_groups] for details.

<code>cv_inputs</code>	The set of CV inputs for this device. Optional. See section [Inputs_and_outputs] for details.
------------------------	---

<code>cv_outputs</code>	The set of CV outputs for this device. Optional. See section [Inputs_and_outputs] for details.
-------------------------	--

<code>audio_inputs</code>	The set of audio inputs for this device. Optional. See section [Inputs_and_outputs] for details.
---------------------------	--

<code>audio_output s</code>	The set of audio outputs for this device. Optional. See section [Inputs_and_outputs] for details.
---------------------------------	---

<code>user_samples</code>	The set of user samples and (optionally) the set of sample parameters for each user sample. Optional. See section User Samples for details.
---------------------------	---

To facilitate automatic routing (see section [\[Automatic_routing\]](#) for details), the `motherboard_def.lua` file may optionally contain calls to the following Lua toolbox functions:

- `jbox.add_cv_routing_target`
- `jbox.add_mono_audio_routing_target`
- `jbox.add_stereo_audio_routing_target`
- `jbox.add_stereo_audio_routing_pair`
- `jbox.add_stereo_effect_routing_hint`
- `jbox.add_stereo_instrument_routing_hint`
- `jbox.add_mono_instrument_routing_hint`

Custom property definitions

The `custom_properties` property set object has the following overall format:

None

```
custom_properties = jbox.property_set{
  gui_owner = {
    properties = {
      ...
    }
  },

  document_owner = {
    properties = {
      ...
    }
  },

  rtc_owner = {
    properties = {
      ...
    }
  },

  rt_owner = {
    properties = {
      ...
    }
  }
}
```

TABLE	DESCRIPTION

gui_owner

Table containing GUI-owned property definitions. Optional. Properties in this scope may be saved to patches and/or documents. They cannot be accessed from the Realtime.

document_owner

Table containing document-owned property definitions. Optional. Properties in this scope may be saved to patches and/or documents.

rtc_owner

Table containing Realtime Controller-owned property definitions. Optional.

rt_owner

Table containing Realtime-owned property definitions. Optional.

Each `properties` table contains zero or more property declarations, e.g.,

None

```
properties = {  
  my_number_property = jbox.number{...},  
  my_boolean_property = jbox.boolean{...},  
}
```

The name given to properties in the `property` table is combined with the name of the property set object to create a *property path*, e.g.,

/custom_properties/my_number_property

Important! Property names must contain ASCII characters only. The length of all property names must be shorter than or equal to 34 characters.

Number properties

Number properties contain single precision floating point values. The values can be continuous or discrete (stepped). Continuous number properties must be in the range [0.0, 1.0]. Discrete number properties are integer valued floating point numbers in the range [0.0, 1.0, ... , (*steps* - 1.0)] where *steps* is the number of steps.

Number properties are declared as follows:

None

```
my_number_property = jbox.number{
  default = ... ,
  steps = ... ,
  ui_name = ... ,
  ui_type = ... ,
  persistence = ... ,
  property_tag = ... ,
}
```

COMPONENT	DESCRIPTION
<code>default</code>	The default value of the property. Required. If <code>steps</code> is declared, the value must be discrete in the range [0.0, 1.0, ... , (<code>steps</code> - 1.0)]. If <code>steps</code> is not declared, the value must be in the range [0.0, 1.0].
<code>steps</code>	If present, defines a discrete number property with <code>steps</code> values. Optional. If defined, <code>steps</code> must be strictly larger than 1 and smaller than or equal to 4194304.
<code>ui_name</code>	A (localized) string that is used to identify the property in the user interface of the host. Required. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string.
<code>ui_type</code>	Specification for how the value of the property should be presented in the host's user interface. Required. See [ui_type] for details.
<code>persistence</code>	Specifies whether the property should be stored in song documents, or patches, or not at all. May only be specified for properties in the <code>document_owner</code> and

`gui_owner` scopes. Optional. See [\[Persistence\]](#) for details.

`property_tag`

Associate this property with a specific integer value. The tag can be used as a shortcut to identify the property in C++ code. Optional. The value must be positive (≥ 0). The value must be smaller than 2,147,483,630. The value must be unique in the property set object in which the property is defined. If no tag is provided, the tag for the property will be set to `kJBox_InvalidPropertyTag`.

Boolean properties

Boolean properties contain boolean values (true/false). They are declared as follows:

None

```
my_boolean_property = jbox.boolean{
    default = ... ,
    ui_name = ... ,
    ui_type = ... ,
    persistence = ... ,
    property_tag = ... ,
}
```

COMPONENT	DESCRIPTION
<code>default</code>	The default value of the property. Required. Must be <code>true</code> or <code>false</code> .
<code>ui_name</code>	A (localized) string that is used to identify the property in the user interface of the host. Required. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string.
<code>ui_type</code>	Specification for how the value of the property should be presented in the host's user interface. Required. See [ui_type] for details.
<code>persistence</code>	Specifies whether the property should be stored in song documents, or patches, or not at all. May only be specified for properties in the <code>document_owner</code> and <code>gui_owner</code> scopes. Optional. See [Persistence] for details.
<code>property_tag</code>	Associate this property with a specific integer value. The tag can be used as a shortcut to identify the property in C++ code. Optional. The value must be positive (≥ 0). The value must be smaller than 2,147,483,630. The value must be unique in the property set object in which the property is defined. If

no tag is provided, the tag for the property will be set to `kJBox_InvalidPropertyTag`.

String properties

String properties contain non-localized 8-bit string values. The string must use UTF-8 encoding if it is to be used for display purposes in the host. The string may contain any character, including embedded zeros. When defining string constants in Lua scripts, Lua escape sequences can be used. The length of string a property is limited to 2 megabytes in size.

String properties can be owned by the Realtime (available since `motherboard_def.lua format_version="2.0"`), but there are some restrictions in that case. The RT string is pre-allocated to a given `max_size`. You are free to encode whatever data you like in an RT string. Typical use cases: sending DSP analysis data to a display or sending sample display data (see the Very Simple Sampler example provided with the SDK).

Note: If the Rack Extension updates the same RT string value more than once every 25 milliseconds, some of the updates will be dropped and a warning will be logged. If "blow fuse on warnings" is enabled in Recon, this will blow the fuse. You can use that logging to tweak the update frequency of your device, so that it doesn't waste time preparing data that will be dropped.

String properties are declared as follows:

None

```
my_string_property = jbox.string{
    default = ... ,
    ui_name = ... ,
    ui_type = ... ,
    persistence = ... ,
    property_tag = ... ,
}

my_realtime_string_property = jbox.string{
    max_size = ... ,
}
```

COMPONENT	DESCRIPTION
default	The default value of the property. Optional. If present, it must follow the requirements described above. If not present, the string property will contain the empty string. Not available for RT strings.
ui_name	A (localized) string that is used to identify the property in the user interface of the host. Optional. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string. Not available for RT strings.

<code>ui_type</code>	Specification for how the value of the property should be presented in the host's user interface. Optional. See [ui_type] for details. Not available for RT strings.
----------------------	--

<code>persistence</code>	Specifies whether the property should be stored in song documents, or patches, or not at all. May only be specified for properties in the <code>document_owner</code> and <code>gui_owner</code> scopes. Optional. See [Persistence] for details. Not available for RT strings.
--------------------------	---

<code>property_tag</code>	Associate this property with a specific integer value. The tag can be used as a shortcut to identify the property in C++ code. Optional. The value must be positive (≥ 0). The value must be smaller than 2,147,483,630. The value must be unique in the property set object in which the property is defined. If no tag is provided, the tag for the property will be set to <code>kJBox_InvalidPropertyTag</code> . Not available for RT strings.
---------------------------	--

<code>max_size</code>	Required for realtime strings, not available for other strings. Maximum size for a realtime string. The value must be > 0 and ≤ 2048 . RT strings need a max size because they must be pre-allocated, since the realtime code cannot allocate. The actual length of a realtime string may vary, but it must not be larger than <code>max_size</code> .
-----------------------	---

Sample properties

Property values of type sample refer to an audio sample file (stereo or mono).

A **custom sample** property (custom property of type sample) is private to the device, can only refer to built-in samples under the Private folder tree of the device, and can only be browsed/alterd by the code in the device.

A **user sample** item property (see [User Samples](#) - available since motherboard_def.lua format_version="2.0") is only browsed/alterd by the host and user (can not be browsed/alterd by the code in the device.)

Audio sample files under the Private folder tree of a device are identified by their file paths in the device file tree, e.g., `"/Private/MySample2.wav"`.

Audio sample files under the Public folder tree of a device become part of the sample database locations in the host and can be referenced from other devices, patches and song document files, much like it goes for a refill file.

Samples are loaded by the host from any file format supported by the host and are made available to the 45 as 32-bit `float` sample values. Sample values are in the range `[-1.0, 1.0]`, regardless of the source data format (normalization is carried out automatically).

Sample data is loaded asynchronously. Sample properties are accessible immediately after creation, even if the actual sample data has not yet been loaded into memory. It is possible to track the loading progress (see `jbox.get_sample_info()` and `jbox.get_sample_meta_data()` for details). Accessing data from a partially loaded sample is allowed, assuming the data being accessed is resident in memory.

Custom Sample

Custom sample properties can only be owned by the Realtime Controller. They are declared as follows:

None

```
my_sample_property = jbox.sample{
  default = ... ,
  ui_name = ... ,
  ui_type = ... ,
  property_tag = ... ,
}
```

COMPONENT	DESCRIPTION

default

The default value of the property. Optional. If present, it must be a string that contains the path to the sample. If not present, the sample property will contain the empty sample.

ui_name

A (localized) string that is used to identify the property in the user interface of the host. Optional. You must use the `jbox.ui_text()` toolbox function to obtain the string.

ui_type

Specification for how the value of the property should be presented in the host's user interface. Optional. See [\[ui_type\]](#) for details.

property_tag

Associate this property with a specific integer value. The tag can be used as a shortcut to identify the property in C++ code. Optional. The value must be positive (≥ 0). The value must be smaller than 2,147,483,630. The value must be unique in the property set object in which the property is defined. If no tag is provided, the tag for the property will be set to `kJBox_InvalidPropertyTag`.

User Sample

See [User Samples](#).

BLOB properties

Binary Large Object (BLOB) properties contain raw bytes. BLOBs are identified by their file name paths in the 45's local file system, e.g., `"/Private/MyBLOB.data"`. BLOBs may only reside in the `Private` folder. BLOBs can only be accessed by the 45 itself, they cannot be made available to other devices.

Important! All Rack Extension code must be endian safe. It is incorrect to assume that the host has any specific endianness. Reading integer data from a BLOB should only be done on a byte by byte basis (using bit shifts to create the multibyte value). You may not assume that the host's endianness for floating point values is the same as its endianness for integers. You may not assume that the floating point representation is IEEE 754 or any other specific representation. If you need to store floats in a BLOB you can use ASCII strings, or store the exponent and mantissa separately as integers.

BLOB data is loaded asynchronously. BLOB properties are accessible immediately after creation, even if the actual data has not yet been loaded into memory. It is possible to track the loading progress (see `jbox.get_blob_info()` for details). Accessing data from a partially loaded blob is allowed, assuming the data being accessed is resident in memory.

BLOB properties can only be owned by the Realtime Controller.

BLOB properties are declared as follows:

None

```
my_blob_property = jbox.blob{
  default = ... ,
  ui_name = ... ,
  ui_type = ... ,
  property_tag = ... ,
}
```

COMPONENT	DESCRIPTION
default	The default value of the property. Optional. If present, it must be a string that contains the path to the blob. If not present, the BLOB property will contain the empty BLOB.
ui_name	A (localized) string that is used to identify the property in the user interface of the host. Optional. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string.

`ui_type`

Specification for how the value of the property should be presented in the host's user interface. Optional. See[\[ui_type\]](#) for details.

`property_tag`

Associate this property with a specific integer value. The tag can be used as a shortcut to identify the property in C++ code. Optional. The value must be positive (≥ 0). The value must be smaller than 2,147,483,630. The value must be unique in the property set object in which the property is defined. If no tag is provided, the tag for the property will be set to `kJBox_InvalidPropertyTag`.

Native Object properties

Native objects hold data that is returned by `JBox_Export_CreateNativeObject()`. See the Jukebox C++ API document for details.

Native object properties can only be owned by the Realtime Controller.

Native object properties are declared as follows:

None

```
my_native_object_property = jbox.native_object{
  default = ... ,
  ui_name = ... ,
  ui_type = ... ,
  property_tag = ... ,
}
```

Default values for native object properties are declared by providing an operation string and a table of arguments, similar to `jbox.make_native_object()`:

None

```
default = {
  "...",      -- operation string
  { ... }    -- arguments
}
```

COMPONENT	DESCRIPTION
default	The default value of the property. Optional. If present, it must be a table containing an operation string and an argument table, as described above. This will generate a call to <code>JBox_Export_CreateNativeObject()</code> when the

property is created. If `default` is not present, the property will contain the empty native object.

`ui_name`

A (localized) string that is used to identify the property in the user interface of the host. Optional. You must use the `jbox.ui_text()` toolbox function to obtain the string.

`ui_type`

Specification for how the value of the property should be presented in the host's user interface. Optional. See [\[ui_type\]](#) for details.

`property_tag`

Associate this property with a specific integer value. The tag can be used as a shortcut to identify the property in C++ code. Optional. The value must be positive (≥ 0). The value must be smaller than 2,147,483,630. The value must be unique in the property set object in which the property is defined. If no tag is provided, the tag for the property will be set to `kJBox_InvalidPropertyTag`.

DSP buffer properties

A DSP buffer property holds an array of 32-bit float samples. DSP buffers are currently used for passing audio between a 45 and the host (see section [\[Inputs_and_outputs\]](#)).

It is currently not possible to create custom DSP buffer properties.

Performance properties

If the host responds to MIDI Continuous Controller messages, it may require special handling for messages for modulation wheel, pitch bend, sustain pedal, expression, breath control, and aftertouch. Therefore, these messages cannot be assigned to properties via the ordinary MIDI CC configuration (see section [\[MIDI_configuration\]](#)). Instead, they use special-purpose property types.

Modulation wheel

Properties that are associated with modulation wheel MIDI CC messages are declared as follows:

None

```
my_modwheel_property = jbox.performance_modwheel{  
  ui_name = ... ,  
  ui_type = ... ,  
  property_tag = ... ,  
  persistence = ... ,  
},
```

COMPONENT	DESCRIPTION
<code>ui_name</code>	A (localized) string that is used to identify the property in the user interface of the host. Optional. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string.
<code>ui_type</code>	Deprecated. The <code>ui_type</code> is set to <code>jbox.ui_percent</code> by the host. See [ui_type] for details.
<code>property_tag</code>	Associate this property with a specific integer value. The tag can be used as a shortcut to identify the property in C++ code. Optional. The value must be positive (≥ 0). The value must be smaller than 2,147,483,630. The value must be unique in the property set object in which the property is defined. If no tag is provided, the tag for the property will be set to <code>kJBox_InvalidPropertyTag</code> .
<code>persistence</code>	Specifies whether the property should be stored in song documents, or patches, or not at all. Optional. See [Persistence] for details. The default value is "none".

Modulation wheel properties are continuous floating-point values in the range [0.0, 1.0].

Pitch bend

Properties that are associated with pitch bend MIDI CC messages are declared as follows:

None

```
my_pitch_bend_property = jbox.performance_pitchbend{
  ui_name = ... ,
  ui_type = ... ,
  property_tag = ... ,
},
```

COMPONENT	DESCRIPTION
ui_name	A (localized) string that is used to identify the property in the user interface of the host. Optional. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string.

`ui_type`

Deprecated. The `ui_type` is set to `jbox.ui_percent` by the host. See [\[ui_type\]](#) for details.

`property_tag`

Associate this property with a specific integer value. The tag can be used as a shortcut to identify the property in C++ code. Optional. The value must be positive (≥ 0). The value must be smaller than 2,147,483,630. The value must be unique in the property set object in which the property is defined. If no tag is provided, the tag for the property will be set to `kJBox_InvalidPropertyTag`.

Pitch bend properties are continuous floating-point values in the range [0.0, 1.0].

Sustain pedal

Properties that are associated with sustain pedal MIDI CC messages are declared as follows:

None

```
my_sustain_pedal_property = jbox.performance_sustainpedal{  
    ui_name = ... ,
```

```
ui_type = ... ,  
property_tag = ... ,  
,
```

COMPONENT	DESCRIPTION
<code>ui_name</code>	A (localized) string that is used to identify the property in the user interface of the host. Optional. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string.
<code>ui_type</code>	Deprecated. The <code>ui_type</code> is set to <code>jbox.ui_percent</code> by the host. See [ui_type] for details.
<code>property_tag</code>	Associate this property with a specific integer value. The tag can be used as a shortcut to identify the property in C++ code. Optional. The value must be positive (≥ 0). The value must be smaller than 2,147,483,630. The value must be unique in the property set object in which the property is defined. If no tag is provided, the tag for the property will be set to <code>kJBox_InvalidPropertyTag</code> .

Sustain pedal properties are continuous floating-point values in the range [0.0, 1.0]. Values ≥ 0.5 should be interpreted as ON. Values < 0.5 should be interpreted as OFF.

Expression

Properties that are associated with expression MIDI CC messages are declared as follows:

None

```
my_expression_property = jbox.performance_expression{
  ui_name = ... ,
  ui_type = ... ,
  property_tag = ... ,
  persistence = ... ,
},
```

COMPONENT	DESCRIPTION
ui_name	A (localized) string that is used to identify the property in the user interface of the host. Optional. You must

use the `jbox.ui_text()` toolbox function to obtain the string.

`ui_type`

Deprecated. The `ui_type` is set to `jbox.ui_percent` by the host. See [\[ui_type\]](#) for details.

`property_tag`

Associate this property with a specific integer value. The tag can be used as a shortcut to identify the property in C++ code. Optional. The value must be positive (≥ 0). The value must be smaller than 2,147,483,630. The value must be unique in the property set object in which the property is defined. If no tag is provided, the tag for the property will be set to `kJBox_InvalidPropertyTag`.

`persistence`

Specifies whether the property should be stored in song documents, or patches, or not at all. Optional. See [\[Persistence\]](#) for details. The default value is "none".

Expression properties are continuous floating-point values in the range [0.0, 1.0].

Properties that are associated with breath control MIDI CC messages are declared as follows:

None

```
my_breath_control_property = jbox.performance_breathcontrol{
    ui_name = ... ,
    ui_type = ... ,
    property_tag = ... ,
},
```

COMPONENT	DESCRIPTION
ui_name	A (localized) string that is used to identify the property in the user interface of the host. Optional. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string.
ui_type	Deprecated. The <code>ui_type</code> is set to <code>jbox.ui_percent</code> by the host. See [ui_type] for details.

`property_tag`

Associate this property with a specific integer value. The tag can be used as a shortcut to identify the property in C++ code. Optional. The value must be positive (≥ 0). The value must be smaller than 2,147,483,630. The value must be unique in the property set object in which the property is defined. If no tag is provided, the tag for the property will be set to `kJBox_InvalidPropertyTag`.

Breath control properties are continuous floating-point values in the range [0.0, 1.0].

Aftertouch

Properties that are associated with aftertouch MIDI CC messages are declared as follows:

None

```
my_aftertouch_property = jbox.performance_aftertouch{
    ui_name = ... ,
    ui_type = ... ,
    property_tag = ... ,
},
```

COMPONENT	DESCRIPTION
<code>ui_name</code>	A (localized) string that is used to identify the property in the user interface of the host. Optional. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string.
<code>ui_type</code>	Deprecated. The <code>ui_type</code> is set to <code>jbox.ui_percent</code> by the host. See [ui_type] for details.
<code>property_tag</code>	Associate this property with a specific integer value. The tag can be used as a shortcut to identify the property in C++ code. Optional. The value must be positive (≥ 0). The value must be smaller than 2,147,483,630. The value must be unique in the property set object in which the property is defined. If no tag is provided, the tag for the property will be set to <code>kJBox_InvalidPropertyTag</code> .
Aftertouch properties are continuous floating-point values in the range [0.0, 1.0].	

UI types

The UI type (`ui_type`) entry in a property declaration specifies how the property should be presented in the host's UI controls and tool tips. Note that the host may allow the user to enter values for the property, so the UI type may be used for both input and output.

There are four UI types: Selector, Percent, Linear, and Nonlinear.

Selector

The selector UI type creates a mapping from discrete (stepped) number properties to a set of (localized) text strings.

It is declared like this:

```
None
ui_type = jbox.ui_selector{
    jbox.ui_text(...),
    jbox.ui_text(...),
    ...
}
```

The number of strings must match the number of steps for the property.

Percent

The percent UI type creates a linear mapping between the full range of a number or boolean property to a percentage value in a specified range. It is declared like this:

None

```
ui_type = jbox.ui_percent(  
    {  
        min = ... ,  
        max = ... ,  
        decimals = ...  
    }  
)
```

COMPONENT	DESCRIPTION

`min`

The percentage value to map to when the property is at its minimum value (0.0 for number properties, or `false` for boolean properties). Optional. If not present, it is set to 0.

`max`

The percentage value to map to when the property is at its maximum value (1.0 or `(steps - 1.0)` for number properties, or `true` for boolean properties). Optional. If not present, it is set to 100.

`decimals`

The number of decimals to use. Must be in the range [0, 10]. Optional. If not present, it is set to 0.

Linear

The linear UI type creates a linear mapping between the full range of a number or boolean property to a number in a specified range. It is declared like this:

None

```
ui_type = jbox.ui_linear(  
    {
```



```
    min = ... ,  
    max = ... ,  
    units = { ... }  
}  
)
```

COMPONENT	DESCRIPTION
<code>min</code>	The value to map to when the property is at its minimum value (0.0 for number properties, or <code>false</code> for boolean properties). Optional. If not present, it is set to 0.
<code>max</code>	The value to map to when the property is at its maximum value (1.0 or (<code>steps</code> - 1.0) for number properties, or <code>true</code> for boolean properties). Optional. If not present, it is set to 100.
<code>units</code>	Rule(s) for converting the mapped property value to a text string for UI display purposes. Optional. See [units] . If not present, the host will use an unspecified built-in conversion.

Nonlinear

The nonlinear UI type creates a generic mapping for number properties. Note that the mapping must be bijective (i.e., every property value must map to a unique target value and vice versa).

It is declared like this:

```
None
ui_type = jbox.ui_nonlinear(
  {
    data_to_gui = function(data_value)
      ...
    end,

    gui_to_data = function(gui_value)
      ...
    end,

    units = { ... }
  }
)
```

COMPONENT	DESCRIPTION
<code>data_to_gui</code>	Function that maps a property value to a UI value. Required. May return <code>inf</code> and/or <code>-inf</code> if appropriate.
<code>gui_to_data</code>	Function that maps a UI value to a property value. Required. If <code>data_to_gui</code> can return <code>inf</code> and/or <code>-inf</code> , this function must detect them and map them to appropriate number property values.
<code>units</code>	Rule(s) for converting the mapped property value to a text string for UI display purposes. Required. See [units] . If not present, the host will use an unspecified built-in conversion.
Specifying units	

The `unit` entry in linear and non-linear UI types is used to fine-tune how the property value should be formatted when presented as text in the host UI. The entry is a table with the following components:

None

```
units = {  
    min_text = ... ,  
    max_text = ... ,  
    { ... },      -- formatting rule 1  
    { ... },      -- formatting rule 2  
    ...  
}
```

COMPONENT	DESCRIPTION
<code>min_text</code>	Localized formatting string that is used to present the property when the property is at its minimum value. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string. If the substring <code>^0</code> is present in the string, it is replaced by the (mapped) property value. If the <code>^0</code> substring is not present, the string will be presented as-is.
<code>max_text</code>	Localized formatting string that is used to present the property when the property is at its maximum value. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string. If the substring <code>^0</code> is present in the string, it is replaced by the (mapped) property value. If the <code>^0</code> substring is not present, the string will be presented as-is.

The formatting rules split the entire property value range (before mapping) into sub-ranges, with one formatting string per sub-range. A rule has the following format:

None

```
{
  decimals = ... ,
  min_value = ... ,
  unit = { ... },
  default = true,
}
```

COMPONENT	DESCRIPTION
<code>decimals</code>	Specifies how many decimals to use when displaying the (mapped) property value. Required. Must not be negative. Must not be larger than 10. The host is free to choose the specifics of how values are rounded.
<code>min_value</code>	Specifies the (pre-mapping) property value sub-range for which to use this rule. More specifically, this formatting rule is used if <code>abs(round(v, decimals)) >= min_value</code> where <code>v</code> is the (pre-mapping) property value and <code>round</code> is a rounding function defined by the host (the same as for <code>decimals</code>). Required.

<code>unit</code>	Formatting string for this rule (see below). Required for the default rule.
-------------------	---

<code>default</code>	Specifies that this rule is to be used if no other rules are applicable for a property value. Optional. If present, must be set to <code>true</code> . If no rule is marked explicitly as <code>default</code> , the first rule in the table will be the default. At most one rule may be marked as default.
----------------------	--

The `unit` entry is specified like this:

None

```
unit = {
  template = ... ,
  base = ...
}
```

COMPONENT	DESCRIPTION
-----------	-------------

template

Formatting string for this rule. Optional. You must use the `jbox.ui_text()` toolbox function to obtain the string. If the substring `^0` is present in the string, it is replaced by the (mapped) property value. If the `^0` substring is not present, the string will be presented as-is. If no template string at all is specified, the template string for the previous rule will be used.

base

Scaling factor. Optional. Must be strictly larger than zero. The (mapped) property value will be divided by `base` before it is presented to the user. Values entered by the user will be multiplied by `base` before they are mapped back to property values.

Jukebox provides a set of built-in common formatting strings (localized by the host):

- `jbox.UI_TEXT_ON` - "On"
- `jbox.UI_TEXT_OFF` - "Off"
- `jbox.UI_TEMPLATE_HZ` - "`^0` Hz"
- `jbox.UI_TEMPLATE_KHZ` - "`^0` kHz"

Example:

None

```
ui_type = jbox.ui_selector({jbox.UI_TEXT_OFF, jbox.UI_TEXT_ON}),
```

Persistence

Properties in the `document_owner` scope are stored in song documents and patches by default (except for performance properties, see [\[Performance_properties\]](#) for details). Properties in the `gui_owner` scope are not stored in documents and patches by default.

The `persistence` attribute can be used to override these behaviors. Allowed values for `persistence` are:

VALUE	DESCRIPTION
"patch"	The property will be stored in both song documents and patches.
"song"	The property will be stored in song documents.

"none" The property will not be stored at all.

Host properties

The host will automatically insert a number of properties in the motherboard. These properties allow the device to respond to changes in, e.g., sample rate or tempo.

Host properties are normally stored in three separate property set objects, `transport`, `environment`, and `note_states`. The `builtin_onoffbypass` property is inserted automatically in the `custom_properties` object, however. See below for details.

Note player devices and devices which include **user samples**(see [User Samples](#)) also have an object called **device_host**. The properties under `device_host` allow the host to respond to commands from the device.

Host Properties in the `transport` object

PROPERTY NAME	VALUE TYPE	C++ PROPERTY TAG	DESCRIPTION
------------------	---------------	---------------------	-------------

playing	boolean	<code>kJBox_TransportPlaying</code>	Indicates whether the master sequencer transport is playing (<code>true</code>) or not (<code>false</code>).
play_pos	number	<code>kJBox_TransportPlayPos</code>	The master transport play position in whole 15360 Pulses Per Quarter note (PPQ) units. Negative values are allowed and correspond to positions prior to the first bar (this might occur during recording with lead-in, for example.)
tempo	number	<code>kJBox_TransportTempo</code>	The approximate tempo for the current render batch in beats per minute, ranging from 1.0 to 999.999. The data is unfiltered, see the <code>filtered_tempo</code> host property below.
filtered_tempo	number	<code>kJBox_TransportFilteredTempo</code>	Lowpass filtered version of the <code>tempo</code> host property.

tempo_automation	boolean	kJBox_TransportTempoAutomation	Indicates whether tempo is being automated (<code>true</code>) or not (<code>false</code>).
time_signature_numerator	number	kJBox_TransportTimeSignatureNumerator	Numerator part of the current time signature.
time_signature_denominator	number	kJBox_TransportTimeSignatureDenominator	Denominator part of the current time signature.
loop_enabled	boolean	kJBox_TransportLoopEnabled	Indicates whether the master sequencer transport is in loop mode (<code>true</code>) or not (<code>false</code>).
loop_start_pos	number	kJBox_TransportLoopStartPos	The master transport loop start position in whole 15360 Pulses Per Quarter note (PPQ) units.

<code>loop_end_pos</code>	number	<code>kJBox_TransportLoopEndPos</code>	The master transport loop end position in whole 15360 Pulses Per Quarter note (PPQ) units. <code>loop_end_pos</code> is allowed to be less than <code>loop_start_pos</code> .
---------------------------	--------	--	--

<code>bar_start_pos</code>	number	<code>kJBox_TransportBarStartPos</code>	The master transport position of the current bar. Position is in whole 15360 Pulses Per Quarter note (PPQ) units.
----------------------------	--------	---	---

<code>request_reset_audio</code>	number	<code>kJBox_TransportRequestResetAudio</code>	Used by the host to trigger an audio reset, i.e., the 45 should silence all its voices and internal audio buffers. The value is a counter with an unspecified non-negative initial value. The host updates the counter with an unspecified amount as the trigger. You may not rely on the actual value of this property (except that it is always non-negative), only the fact that it has received a new value.
----------------------------------	--------	---	--

<code>request_run</code>	number	<code>kJBox_Transport RequestRun</code>	Changed by the host to indicate that a device with an internal sequencer should start playing. The value is a counter with an unspecified non-negative initial value. You may not rely on the actual value of this property (except that it is always non-negative), only the fact that it has received a new value.
--------------------------	--------	---	--

<code>request_stop</code>	number	<code>kJBox_Transport RequestStop</code>	Changed by the host to indicate that a device with an internal sequencer should stop playing. The value is a counter with an unspecified non-negative initial value. You may not rely on the actual value of this property (except that it is always non-negative), only the fact that it has received a new value.
---------------------------	--------	--	---

<code>pattern_index</code>	number	<code>kJBox_Transport PatternIndex</code>	The index of the currently selected pattern. If no pattern is selected, this property has the value <code>kInvalidPatternIndex</code> . Only available if <code>/patterns</code> has been defined in <code>motherboard_def.lua</code> .
----------------------------	--------	---	---

<code>pattern_start_pos</code>	number	<code>kJBox_TransportPatternStartPos</code>	The pattern start position of the currently selected pattern in whole 15360 Pulses Per Quarter note (PPQ) units. Negative values are allowed and correspond to positions prior to the first bar. If no pattern is selected this property has the value 0. Only available if <code>/patterns</code> has been defined in <code>motherboard_def.lua</code>.
--------------------------------	--------	---	--

Host properties in the `environment` object

PROPERTY NAME	VALUE TYPE	C++ PROPERTY TAG	DESCRIPTION
<code>master_tune</code>	number	<code>kJBox_Environment MasterTune</code>	Global master tune setting in cents, i.e., [-100.0, 100.0]. Jukebox follows the convention that the octave is divided into 12 semitones, and that each semitone is divided into 100 cents. The master tune setting offsets the frequency of A = 440 Hz (MIDI note 69). All other notes must be offset accordingly.

<code>system_sample_rate</code>	number	<code>kJBox_Environment SystemSampleRate</code>	The current sample rate in Hz.
<code>instance_id</code>	number	<code>kJBox_Environment InstanceID</code>	Unique ID for each initialized 45 instance.
<code>devicevisible</code>	boolean	<code>kJBox_Environment DeviceVisible</code>	Has the value <code>true</code> if the 45 is visible in the host's GUI, <code>false</code> otherwise.
<code>player_bypassed</code>	boolean	<code>kJBox_Environment PlayerBypassed</code>	Has the value <code>true</code> if a note player is bypassed by the host. <code>false</code> otherwise. A bypassed player will not receive any note events. While bypassed, a player device should not output note events or CV.

Host properties in the `device_host` object

Currently only used by player devices or device with at least one user sample. See section [User Samples](#).

PROPERTY NAME	VALUE TYPE	C++ PROPERTY TAG	DESCRIPTION
<code>sample_context</code>	number	<code>kJBox_DeviceHost SampleContext</code>	The index of the currently active user sample object, out of all declared in the motherboard. Integer set by the device, range (0 ≤ x < n) where n is the number of user samples, and the order is the top-bottom order of the list of user samples in the motherboard_def.lua
<code>delete_sample</code>	number	<code>kJBox_DeviceHost DeleteSample</code>	Momentary trigger used to make the host change the currently active user sample in the device to empty/no sample.
<code>edit_sample</code>	number	<code>kJBox_DeviceHost EditSample</code>	Momentary trigger used to make the host launch the sample editor for the currently active user sample in the device.

Host properties in the `patterns` object

A device that uses patterns should declare the number of patterns in `motherboard_def.lua` in this format:

None

```
patterns = {
    num_patterns = 8,
}
```

Where `num_patterns` can be an integer between 1 and 32.

The host will add auto-inserted properties for each pattern. Currently it only contains information about the length of each pattern. Information about each pattern is located at `/patterns/<index>`, where `<index>` is an integer in the range `[0, num_patterns)`. For example, the length property for pattern with index 0 is accessible at the path `/patterns/0/length`.

PROPERTY NAME	VALUE TYPE	C++ PROPERTY TAG	DESCRIPTION
length	number	kJBox_PatternsLength	The pattern length in whole 15360 Pulses Per Quarter note (PPQ) units. This property should be updated by the device realtime to reflect the length of each pattern.

The `builtin_onoffbypass` host property

If the 45 has set the `device_type` to "effect" in `info.lua`, the host will automatically insert a `builtin_onoffbypass` property into the `custom_properties` property set object. This is a `document_owner` property with "song" persistence that represents the three different `enabled` states of an effect: "Bypass", "On" and "Off". All effects must present this property in the GUI, as it is partially controlled by the host.

Use the following `enum` values to check for the different "enabled" states from C++:

```
None
typedef enum {
    kJBox_EnabledOff = 0,
    kJBox_EnabledOn,
    kJBox_EnabledBypass
} TJBox_OnOffBypassStates;
```

The `instance` host property

The host will automatically insert an `instance` property into the `custom_properties` property set object for all devices. This is an `rtc_owner` property that is intended to hold the native instance object that represents the C++ state of the 45. It is passed as the first argument to `JBox_Export_RenderRealtime()`.

The `instance` property will contain the empty native object (corresponding to a `NULL` pointer in C++) until you set it to a new value in `realtime_controller.lua`. See section [\[Realtime_controller\]](#) for details.

Note states

All Jukebox devices are able to receive MIDI notes (but see "Reason-specific issues" below). There are 128 MIDI notes [0, 127]. By default, note number 69 corresponds to A440 (i.e., 440 Hz for Middle-A on a standard piano keyboard), but this can be offset by +/- one semitone by the user via the host's Master Tune setting (which is exposed in the `/environment/master_tune` property).

Each MIDI note has a corresponding number property in the

```
/note_states
```

property set object. The property name for each note is the note number, i.e.,

```
/note_states/0  
/note_states/1  
/note_states/2  
...  
/note_states/127
```

The property tag for each note property is the note number.

Reason-specific issues

Instrument devices (with `device_type` set to "instrument" in `info.lua`) will receive notes in Combinators by default. The other device categories do not receive notes in Combinators by default. The user can override this by unchecking the "Receive Notes" checkbox in the Combinator programmer, unless the `accepts_notes` flag has been set to `false` in `info.lua`. If `accepts_notes` is `false`, the device will never be able to receive notes inside Combinators, and the "Receive Notes" checkbox will be disabled in the Combinator programmer.

MIDI configuration

The `midi_implementation_chart` section of `motherboard_def.lua` contains configures MIDI-related aspects of the 45. Currently, it can have one (optional) entry, `midi_cc_chart`.

The `midi_cc_chart` binds `document_owner` properties to MIDI Continuous Controller (MIDI CC) messages. The format of the chart is a Lua table:

None

```
midi_implementation_chart = {  
  midi_cc_chart = {  
    [ ... ] = "...",  
    ...  
  }  
}
```

Each entry in the table is a key-value pair, where the key is the MIDI CC number and the value is the path for the property to bind to.

Each MIDI CC number must be an integer in the range [0, 255]. The following MIDI CC numbers are reserved and cannot be used:

RESERVED MIDI CC	RESERVED FOR
0	Bank select
1	Mod wheel
2	Breath control
3	(internal)
6	Data entry MSB
9	(internal)

11	Expression
32	Bank select LSB
38	Data entry LSB
64	Sustain (damper pedal)
96-101	(internal)
120-127	Channel mode message
240-252	(internal)
253	Aftertouch (channel pressure)
254	(internal)

Example:

None

```
midi_cc_chart = {  
  [40] = "/custom_properties/myProperty1",  
  [41] = "/custom_properties/myProperty2",  
}
```

Reason-specific information

If you map a MIDI CC number to a property, it will be automatable in Reason.

The **MIDI Implementation Chart** document (which can be found in the Reason documentation folder) contains the MIDI CC mappings for all legacy devices in Reason. It is strongly recommended to use similar mappings for similar kinds of devices.

Remote™ configuration

The `remote_implementation_chart` in `motherboard_def.lua` associates remotable items with motherboard properties. The format of the chart is a Lua table:

None

```
remote_implementation_chart = {
  [ ... ] = { ... },
  ...
}
```

Each entry in the table is a key-value pair, where the key is the property path and the value is a table with the following entries:

ATTRIBUTE	DESCRIPTION
<code>internal_name</code>	Remotable item ID for the property. Used to identify the property in Remote maps. Required. Must be encoded as 7-bit ASCII. Must be shorter than 64 characters. May not contain the characters <code>,</code> (comma), <code>=</code> (equality), <code>\</code> (backslash), <code>"</code> (quote). May not begin with a number. You cannot use <code>jbox.ui_text()</code> to obtain a string for <code>internal_name</code> (the ID must be unique and is therefore not localizable).
<code>short_ui_name</code>	User-readable localized name for the property. Used by hardware controllers to display the property name to the user. Required. Must be at most 8 characters long. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string.

`shortest_ui_name`

Short version of the user-readable localized name for the property. Used by hardware controllers to display the property name to the user. Required. Must be at most 4 characters long. You must use the `jbox.ui_text()` toolbox function to obtain the string.

Example:

None

```
remote_implementation_chart = {
  [ "/custom_properties/myProperty1" ] = {
    internal_name = "myProperty1",
    short_ui_name = jbox.ui_text("Key_myProperty1_short_remote_name"),
    shortest_ui_name =
jbox.ui_text("Key_myProperty1_shortest_remote_name"),
  },
  [ "/custom_properties/myProperty2" ] = {
    internal_name = "myProperty2",
    short_ui_name = jbox.ui_text("Key_myProperty2_short_remote_name"),
    shortest_ui_name =
jbox.ui_text("Key_myProperty2_shortest_remote_name"),
  },
}
```

Reason-specific information

Reason exposes the `internal_name` of Remotable items in its "Edit Remote Override Mapping" dialog, so it is strongly recommended to use a descriptive (English) string for this attribute.

UI groups

The `ui_groups` table of `motherboard_def.lua` allows you to create groups of logically related motherboard properties for UI display purposes. The host may use the grouping information in any way it sees fit, including changing the order of the groups and the properties in each group.

Each entry in the `ui_groups` table is a table with the following format:

```
None
{
  ui_name = ... ,
  properties = { ... }
},
```

COMPONENT	DESCRIPTION
<code>ui_name</code>	Localized user-readable name for this UI group. Required. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string.
<code>properties</code>	List of property paths. Required. Performance properties are not allowed. A property may only belong to one group.

Properties that are not assigned to a group remain in the default (unnamed) group. Performance properties are automatically assigned to a group called "Performance". The `builtin_onoffbypass` host property is not put in a group by default, but it can be assigned to a group if desired.

Example:

```
None
ui_groups = {
    {
        ui_name = jbox.ui_text("Key_group1"),
        properties = {
            "/custom_properties/myProperty1",
            "/custom_properties/myProperty2",
        }
    },
    {
        ui_name = jbox.ui_text("Key_group2"),
        properties = {
            "/custom_properties/myProperty3",
        }
    },
}
```

Reason-specific information

In Reason, UI groups are used in the Combinator programmer. Groups are not shown if the motherboard contains fewer than 10 automatable properties.

Input and output sockets

A Jukebox 45 uses audio input sockets to receive audio from the host and/or other connected 45s. It uses audio output sockets to send audio to the host and/or other connected 45s. Similarly, CV input and output sockets are used to receive and send control voltage values, respectively.

All input and output sockets that you declare in `motherboard_def.lua` must have corresponding GUI widgets in `hdgui_2D.lua` (see section [\[hd_gui\]](#)).

Player devices are allowed to have audio input sockets, but not audio output sockets.

When the Realtime reads from an input socket, it is given the value of the output socket that the input socket is currently connected to. The host is free to handle routing between sockets in any way it sees fit. Feedback connections are allowed, which may introduce a single batch of delay for both audio and CV. The amount of delay depends on a setting in Reason's preferences called "Render audio using audio card buffer size setting". If this is off, the delay is a single batch (64 frames). If this is on (default) the delay depends on the buffer size set by the user in the audio preferences.

Audio sockets

Audio sockets are property set objects and are declared like this:

None

```
audio_inputs = {  
    my_audio_input = jbox.audio_input{  
        ui_name = ... ,  
    },  
}  
  
audio_outputs = {  
    my_audio_output = jbox.audio_output{  
        ui_name = ... ,  
    },  
}
```

`ui_name` is a (localized) string that is used to identify the socket in the user interface of the host. It is required. You must use the `jbox.ui_text()` toolbox function to obtain the string.

The maximum number of input sockets is 100. The maximum number of output sockets is 100.

The host inserts these properties into the socket property set object automatically:

PROPERTY NAME	PROPERTY TYPE	C++ PROPERTY TAG	DESCRIPTION
------------------	------------------	---------------------	-------------

<code>buffer</code>	DSP buffer	<code>kJBox_AudioInputBuffer</code>	A single channel of floating point audio input data for the current batch. The buffer size is equal to the batch size, which is 64 frames. Owned by the host.
---------------------	------------	-------------------------------------	---

<code>connected</code>	boolean	<code>kJBox_AudioInput Connected</code>	Indicates if another 45 is connected to this socket (<code>true</code>) or not (<code>false</code>). Owned by the host.
------------------------	---------	---	---

PROPERTY NAME	PROPERTY TYPE	C++ PROPERTY TAG	DESCRIPTION
<code>buffer</code>	DSP buffer	<code>kJBox_AudioOutput Buffer</code>	A single channel of floating point audio output data for the current batch. The buffer size is equal to the batch size, which defaults to 64 frames. Owned by the Realtime.
<code>connected</code>	boolean	<code>kJBox_AudioOutput Connected</code>	Indicates if another 45 is connected to this socket (<code>true</code>) or not

(*false*). Owned by the host.

dsp_latency	number	<code>kJBox_AudioOutput DSPLatency</code>	Specifies the current processing latency of the 45 for this socket, measured in batch frames. Owned by the Realtime Controller. Changing the latency during the lifetime of the 45 is allowed, but may cause breakups in the output audio.
-------------	--------	---	--

To obtain the auto-inserted properties for an audio socket, you must use the full property path, e.g.,

```
"/audio_inputs/my_audio_input/buffer"  
"/audio_inputs/my_audio_input/connected"  
"/audio_outputs/my_audio_output/buffer"  
"/audio_outputs/my_audio_output/connected"  
"/audio_outputs/my_audio_output/dsp_latency"
```

Silence optimization

All 45 devices must handle *silence*, i.e., stop rendering when all inputs are silent (or shortly after inputs become silent). An audio input is considered to be silent when all values in the buffer are smaller than or equal to the C++ constant `kJBox_SilentThreshold`.

If a 45 does not touch any of the values in the DSP buffer of the `buffer` property in an audio output socket during a batch, the host considers the socket to be *silent*. Note that filling the buffer with zeros does not constitute silence in this sense.

If the 45 does choose to write to the DSP buffer, then all values must be written to. It is not permitted to write `nan:s` or `inf:s` to the buffer.

Control Voltage (CV) sockets

CV sockets are property set objects and are declared like this:

None

```
cv_inputs = {
  my_cv_input = jbox.cv_input{
    ui_name = ... ,
  },
}

cv_outputs = {
  my_cv_output = jbox.cv_output{
    ui_name = ... ,
  },
}
```


`ui_name` is a (localized) string that is used to identify the socket in the user interface of the host. It is required. You must use the `jbox.ui_text()` toolbox function to obtain the string.

The maximum number of input sockets is 100. The maximum number of output sockets is 100.

The host inserts these properties into the socket automatically:

PROPERTY NAME	PROPERTY TYPE	C++ PROPERTY TAG	DESCRIPTION
<code>value</code>	number	<code>kJBox_CVInputValue</code>	The value of the incoming CV signal. Single-precision float in the range [-10000,10000]. May have been scaled by a trim knob. Owned by the host.
<code>connected</code>	boolean	<code>kJBox_CVInputConnected</code>	Indicates if another 45 is connected to this socket (<code>true</code>) or not (<code>false</code>). Owned by the host.

PROPERTY NAME	PROPERTY TYPE	C++ PROPERTY TAG	DESCRIPTION
<code>value</code>	number	<code>kJBox_CVOutputValue</code>	The value of the outgoing CV signal. Single-precision float in the range [-10000,10000]. Owned by the Realtime.
<code>connected</code>	boolean	<code>kJBox_CVOutputConnected</code>	Indicates if another 45 is connected to this socket (<code>true</code>) or not (<code>false</code>). Owned by the host.
<code>dsp_latency</code>	number	<code>kJBox_CVOutputDSPLatency</code>	Specifies the current processing latency of the 45 for this socket, measured in batch frames. Owned by the Realtime Controller. Changing the latency during the lifetime of the 45 is allowed, but may cause breakups in the CV signal and/or audio.

To obtain the auto-inserted properties for a CV socket, you must use the full property path, e.g.,

```
"/cv_inputs/my_cv_input/value"  
"/cv_inputs/my_cv_input/connected"  
"/cv_outputs/my_cv_output/value"  
"/cv_outputs/my_cv_output/connected"  
"/cv_outputs/my_cv_output/dsp_latency"
```

User Samples

User Samples are property set objects, available since `format_version="2.0"` (of `motherboard_def.lua`). The default value for the `item` property will always be an empty sample. Defining the default attribute in a `user_sample` object is considered an error. The default path can be used for defining the samples that are loaded on device creation. Devices with at least one user sample enable some related host properties under the `device_host` object, which the device can use to integrate with the user samples hosting. See `device_host` in section [\[Host_properties\]](#) for details. Also see the `sample_browse_group` ([\[sample_browse_group\]](#)) and `sample_drop_zone` ([\[sample_drop_zone\]](#)) widgets.

User samples are declared like this:

None

```
user_samples = {  
    -- a numbered user sample, object instance path /user_samples/0  
    jbox.user_sample{  
        ui_name = jbox.ui_text("text_Sample0_UIName"),  
        -- optional field to make properties for sample parameters  
        sample_parameters = {  
            "root_key",  
            "tune_cents",  
            "play_range_start",  
            "play_range_end",  
            "loop_range_start",  
            "loop_range_end",  
        },  
    },  
}
```

```

        "loop_mode",
        "preview_volume_level"
    },
},
}

```

None

```

user_samples = {
    -- a named user sample, object instance path
    /user_samples/a_named_sample
    a_named_sample = jbox.user_sample{
        ui_name = jbox.ui_text("text_Sample0_UIName"),
        -- optional field to make properties for sample parameters
        sample_parameters = {
            "root_key",
            "tune_cents",
            "play_range_start",
            "play_range_end",
            "loop_range_start",
            "loop_range_end",
            "loop_mode",
            "preview_volume_level"
        },
    },
},
}

```

COMPONENT

DESCRIPTION

<code>ui_name</code>	A (localized) string that is used to identify the property in the user interface of the host. Required. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string.
----------------------	---

<code>sample_parameters</code>	Optional. List of sample parameters. Valid values are "root_key", "tune_cents", "play_range_start", "play_range_end", "loop_range_start", "loop_range_end", "loop_mode", "preview_volume_level"
--------------------------------	---

<code>persistence</code>	Specifies whether the resulting properties should be stored in song documents, or patches, or not at all. May only be specified for properties in the <code>document_owner</code> and <code>gui_owner</code> scopes. Optional. See [Persistence] for details.
--------------------------	---

The following table lists all properties the host can create under each user sample instance, depending on the **sample_parameters** component of `jbox.user_sample{}`.

If creating one of `play_range_start`, `play_range_end`, `loop_range_start`, and `loop_range_end` all four of them must be created. Furthermore, when changing one of these parameters from a `custom_display` it is up to the Rack Extension developer to make sure they are valid (i.e `0 <= play_range_start <= loop_range_start <= loop_range_end <= play_range_end <= 1`). Note that although Reasons native devices and the sample editor do not support zero length ranges (`start == end`) we allow it in Rack Extensions in order to support checking their validity without knowing the length of the sample.

PROPERTY NAME	PROPERTY TYPE	C++ PROPERTY TAG	DESCRIPTION
item	sample	<code>kJBox_UserSampleItem</code>	The current sample item for a specific user sample instance. Owned by the document.
root_key	number	<code>kJBox_UserSampleRootKey</code>	The currently active MIDI root key for a specific user sample instance. Owned by the document.
tune_cents	number	<code>kJBox_UserSampleTuneCents</code>	The currently active tuning in cents (+/-50 cents) for a specific user sample instance. Owned by the document.
play_range_start	number	<code>kJBox_UserSamplePlayRangeStart</code>	The start position (0 <= x <= 1) of the currently active play-range for a specific user sample instance. Owned by the document.

<code>play_range_end</code>	number	<code>kJBox_UserSample PlayRangeEnd</code>	The end position (0 <= x <= 1) of the currently active play-range for a specific user sample instance. Owned by the document.
-----------------------------	--------	--	---

<code>loop_range_start</code>	number	<code>kJBox_UserSample LoopRangeStart</code>	The start position (0 <= x <= 1) of the currently active loop-range for a specific user sample instance. Owned by the document.
-------------------------------	--------	--	---

<code>loop_range_end</code>	number	<code>kJBox_UserSample LoopRangeEnd</code>	The end position (0 <= x <= 1) of the currently active loop-range for a specific user sample instance. Owned by the document.
-----------------------------	--------	--	---

<code>loop_mode</code>	number	<code>kJBox_UserSample LoopMode</code>	The currently active loop-mode (0 = Off, 1 = Forward, 2 = BwdFwd) for a specific user sample instance. Owned by the document.
------------------------	--------	--	---

To obtain the auto-inserted properties for a user sample instance, you must use the full property path (or object + tag).

```
"/user_samples/0/item"  
"/user_samples/a_named_sample/item"  
"/user_samples/a_named_sample/root_key"
```

Automatic routing

In order to facilitate automatic routing, your Jukebox device may provide information about how it should be routed. If it does not provide routing information, the host may not be able to offer automatic routing functionality for the device (in which case the user must connect all sockets manually).

The exact automatic routing behavior depends on the specifics of the host, the device type, and the current connection state of all involved devices. The host is not required to support automatic routing of any sockets, even if the 45 declares them as routing targets.

The routing system involves the following terminology:

- *Source device*: The device being routed.
- *Target device*: An existing device that the source device may (or may not) be able to connect to.
- *Routing target*: A single CV or audio socket, or an audio socket pair (L+R) on a target device. Assumes that the signal is of a certain type, e.g., pitch CV or gate CV.
- *Routing hint*: A abstract description of a source device. The host uses the hints to determine how to connect sockets on the source device to routing targets on the target device.
- *Routing pair*: An audio socket pair (L+R).

Routing hints

A routing hint defines the role that a source device has in the signal chain, i.e., it contains information that the host can use to determine how to connect the source device to one or several existing target devices.

A device may specify zero or more hints. If no hint is specified, the host may not be able to route the device automatically.

If more than one hint is specified, the host will attempt to apply the hints in the order they were specified in `motherboard_def.lua`, until one of the hints have been successfully applied.

Jukebox distinguishes between the following roles for automatic routing:

- Stereo effect
- Stereo instrument
- Mono instrument

If you choose to specify routing hints, then at least one of those hints must match the `device_type` setting in `info.lua` as described in the following table:

"instrument"	Stereo instrument or mono instrument
--------------	--------------------------------------

"studio_fx"	Stereo effect
-------------	---------------

"creative_fx"	Stereo effect
---------------	---------------

"helper"	Stereo instrument, mono instrument or stereo effect
----------	--

For example, an `"instrument"` device must (at least) declare a stereo instrument hint or a mono instrument routing hint.

Important!

Mono effects (one audio input socket + one audio output socket) can not be routed automatically. Instead, such devices should provide a stereo input pair and a stereo output pair and process the left and right channel independently of each other, and define a stereo effect routing hint.

Stereo effect routing hint

To add a stereo effect routing hint (two primary audio inputs and two primary audio outputs), you declare the following table in `motherboard_def.lua`:

None

```
jbox.add_stereo_effect_routing_hint{
  left_input = ... ,
  right_input = ... ,
  left_output = ... ,
  right_output = ... ,
  type = ...
}
```

COMPONENT	DESCRIPTION
<code>left_input</code>	Property path to an audio input socket in the <code>"/audio_inputs"</code> property set object. Defines the left input channel of the device. Required.
<code>right_input</code>	Property path to an audio input socket in the <code>"/audio_inputs"</code> property set object. Defines the right input channel of the device. Required.

left_output	Property path to an audio output socket in the <code>"/audio_outputs"</code> property set object. Defines the left output channel of the device. Required.
-------------	--

right_output	Property path to an audio output socket in the <code>"/audio_outputs"</code> property set object. Defines the right output channel of the device. Required.
--------------	---

type	Specifies how the device applies stereo signals. Required. The value must be one of the strings <code>"true_stereo"</code> , <code>"mixing_stereo"</code> , or <code>"spreading"</code> . See the table below for details.
------	--

COMPONENT	DESCRIPTION
<code>"true_stereo"</code>	For stereo input, the device processes the signal as dual mono, i.e., the left and right channels are processed independently of each other. Typical examples of such devices are filters or equalizers.

"spreading"

For mono input, the device produces a stereo output signal. For example, the device might pan the input signal. In Reason, the PH-90 Phaser and CF-101 Chorus/Flanger effects can be configured to work this way.

"mixing_stereo"

For stereo input, the device sums the left and right channel, or combines them in some other way, and produces a stereo output. Typical examples of such devices are delays or reverbs.

All effect devices must display appropriate routing icons on the back panel. See section [\[Routing_icons\]](#) for information about which icon that corresponds to which setting of `type`.

Stereo instrument routing hint

To add a stereo instrument routing hint (two primary audio outputs), you declare the following table in `motherboard_def.lua`:

None

```
jbox.add_stereo_instrument_routing_hint{
```

```
left_output = ... ,  
right_output = ...  
}
```

COMPONENT	DESCRIPTION
<code>left_output</code>	Property path to an audio output socket in the <code>"/audio_outputs"</code> property set object. Defines the left output channel of the device. Required.
<code>right_output</code>	Property path to an audio output socket in the <code>"/audio_outputs"</code> property set object. Defines the right output channel of the device. Required.

Mono instrument routing hint

To add a mono instrument routing hint (one primary audio output), you declare the following table in `motherboard_def.lua`:

None

```
jbox.add_mono_instrument_routing_hint{  
    mono_output = ...  
}
```

COMPONENT	DESCRIPTION
<code>mono_output</code>	Property path to an audio output socket in the <code>"/audio_outputs"</code> property set object. Defines the output channel of the device. Required.

Routing targets

A routing target is a single socket or a socket pair on a target device that a source device can connect to. The following routing targets are available:

- CV routing target
- Mono audio routing target
- Stereo audio routing target

A socket may only be included in one routing target definition.

You may specify several routing targets of the same type (a mixer device might define a number of stereo auto routing targets, for example).

CV routing target

A CV routing target is a single CV socket on your device that a source device can connect to. The routing target can be an input socket (in which case it will be connected to a corresponding output socket on the source device) or an output socket (in which case it will be connected to an input socket).

To declare a CV routing target, you create a CV routing target table in `motherboard_def.lua`. This object has the following format:

None

```
jbox.add_cv_routing_target{  
    signal_type = ... ,  
    path = ... ,  
    auto_route_enable = ...  
}
```


COMPONENT	DESCRIPTION
<code>signal_type</code>	Specifies the type of signal that is sent to or from this routing target. Required. Must be one of the values <code>"curve"</code> , <code>"pitch"</code> , <code>"gate"</code> , <code>"trigg"</code> , <code>"mod_wheel"</code> , or <code>"pitch_wheel"</code> . See below for details.
<code>path</code>	Property path to an input CV socket in the <code>"/cv_inputs"</code> property set object, or to an output CV socket in the <code>"/cv_outputs"</code> property set object. Required.
<code>auto_route_enable</code>	If <code>true</code> , then this target will be used for automatic routing. If <code>false</code> , this target will only be used to keep the signal chain consistent, e.g., when replacing a device with a different device. Required.

COMPONENT	DESCRIPTION
<code>"curve"</code>	CV input for curve signals, e.g., those sent on the "Curve CV" output of the <i>Matrix</i> device in Reason.

"pitch"

CV input for note pitch signals, e.g., those sent on the "Pitch CV" output of the *Matrix* device in Reason. Used together with "gate" CV signals for controlling note on/off for (single-voice) instruments.

"gate"

CV input for note gate signals, e.g., those sent on the "Gate CV" output of the *Matrix* device in Reason. Used together with "pitch" CV signals for controlling note on/off for (single-voice) instruments.

"trigg"

Generic trigger signal.

"mod_wheel"

CV input for mod wheel signals, e.g., those sent on the "Mod Weel CV Out" (sic) output of the *RPG-8* device in Reason.

"pitch_wheel"

CV input for pitch wheel signals, e.g., those sent on the "Pitch Bend CV Out" output of the *RPG-8* device in Reason.

Mono audio routing target

A mono audio routing target is a single audio socket on your device that a source device can connect to. The routing target can be an input socket (in which case it will be connected to a corresponding output socket on the source device) or an output socket (in which case it will be connected to an input socket).

To declare a mono audio routing target, you create a mono audio routing target table in `motherboard_def.lua`. This object has the following format:

None

```
jbox.add_mono_audio_routing_target{
  signal_type = ... ,
  path = ... ,
  auto_route_enable = ...
}
```

COMPONENT	DESCRIPTION
<code>signal_type</code>	Specifies the type of signal that is sent to or from this routing target. Required. Must be one of the values "send", "return", or "normal". See below for details.

path	Property path to an audio input socket in the <code>"/audio_inputs"</code> property set object, or to an audio output socket in the <code>"/audio_outputs"</code> property set object. Required.
------	--

auto_route_enable	If <code>true</code> , then this target will be used for autorouting. If <code>false</code> , this target will only be used to keep the signal chain consistent, e.g., when replacing a device with a different device. Required.
-------------------	---

COMPONENT	DESCRIPTION
"send"	Denotes the socket as an effect loop send. If you use this setting, <code>path</code> must point to an audio output socket.
"return"	Denotes the socket as an effect loop return. If you use this setting, <code>path</code> must point to an audio input socket.
"normal"	Generic audio signal.

Stereo audio routing target

A stereo audio routing target is a pair of audio sockets on your device that a source device can connect to. The routing target can consist of two input sockets (in which case it will be connected to a pair of corresponding output sockets on the source device) or a pair of output sockets (in which case it will be connected to an input socket pair).

To declare a stereo audio routing target, you create a stereo audio routing target table in `motherboard_def.lua`. This object has the following format:

None

```
jbox.add_stereo_audio_routing_target{
  signal_type = ... ,
  left = ... ,
  right = ... ,
  auto_route_enable = ...
}
```

COMPONENT	DESCRIPTION
<code>signal_type</code>	Specifies the type of signal that is sent to or from this routing target. Required. Must be one of the values "send", "return", or "normal". See below for details.

left

Property path to an audio input socket in the `"/audio_inputs"` property set object, or to an audio output socket in the `"/audio_outputs"` property set object. Required. Denotes the left channel of the stereo socket pair. If `left` is an input socket, then `right` must be an input socket too. If `left` is an output socket, then `right` must be an output socket too. Must not point to the same socket as `right`.

right

Property path to an audio input socket in the `"/audio_inputs"` property set object, or to an audio output socket in the `"/audio_outputs"` property set object. Required. Denotes the right channel of the stereo socket pair. If `right` is an input socket, then `left` must be an input socket too. If `right` is an output socket, then `left` must be an output socket too. Must not point to the same socket as `left`.

auto_route_enable

If `true`, then this target will be used for autorouting. If `false`, this target will only be used to keep the signal chain consistent, e.g., when replacing a device with a different device. Required.

COMPONENT

DESCRIPTION

"send"

Denotes the socket pair as an effect loop send. If you use this setting, `left` and `right` must both point to audio output sockets.

"return"

Denotes the socket pair as an effect loop return. If you use this setting, `left` and `right` must both point to audio input sockets.

"normal"

Generic audio signal.

Routing pair definitions

Routing pair definitions are used to specify left/right audio socket pairs. These definitions are used for automatic routing, but may also be used in other situations.

There must be a routing pair definition for each stereo audio routing target.

It is strongly recommended to create routing pair definitions for all left/right audio sockets, even if they are not included in any audio routing targets.

A socket may not appear in more than one routing pair definition.

To declare a routing pair definiton, you create a routing pair table in `motherboard_def.lua`. This object has the following format:

None

```
jbox.add_stereo_audio_routing_pair{
  left = ... ,
  right = ...
}
```

COMPONENT	DESCRIPTION
left	Property path to an audio input socket in the <code>"/audio_inputs"</code> property set object, or to an audio output socket in the <code>"/audio_outputs"</code> property set object. Required. Denotes the left channel of the stereo socket pair. If <code>left</code> is an input socket, then <code>right</code> must be an input socket too. If <code>left</code> is an output socket, then <code>right</code> must be an output socket too. Must not point to the same socket as <code>right</code> .

`right`

Property path to an audio input socket in the `"/audio_inputs"` property set object, or to an audio output socket in the `"/audio_outputs"` property set object. Required. Denotes the right channel of the stereo socket pair. If `right` is an input socket, then `left` must be an input socket too. If `right` is an output socket, then `left` must be an output socket too. Must not point to the same socket as `left`.

Bypass behavior for effects

If a song or patch contains an effect device for which the user has no license, the host will attempt to bypass the effect. More specifically, the input audio signal(s) that are sent into the effect will be connected directly to its audio output(s).

To specify the sockets that should be connected in this situation, you create an effect bypass routing table in `motherboard_def.lua`. This object has the following format:

None

```
jbox.set_effect_auto_bypass_routing{
  {
    ... ,
    ...
  },
  {
    ... ,
    ...
  }
}
```

The object contains two tables. The first table must contain two paths to audio input sockets. The second table must contain two paths to audio output sockets.

For example:

None

```
jbox.set_effect_auto_bypass_routing{
  {
    "/audio_inputs/left",
    "/audio_outputs/left"
  },
  {
    "/audio_inputs/right",
    "/audio_outputs/right"
  }
}
```

Routing icons

All effect devices must display a set of appropriate routing icons on the back panel. The table below shows the available icons and how they are associated with the `type` setting of stereo effect routing hints (see section [\[Automatic_routing\]](#)).

ICON	DESCRIPTION	STEREO EFFECT ROUTING HINT TYPE
	For mono input, the device produces mono output.	n/a
	For mono input, the device produces stereo output.	"spreading"
	For stereo input, the device processes the two channels independently of each other and produces a stereo output.	"true_stereo"
	For stereo input, the device sums the left and right channels before applying the effect. The output is in stereo.	"mixing_stereo"
	For stereo input, both channels contribute to the effect (they are not simply summed). The output is in stereo (where the two channels are not equivalent).	"mixing_stereo"

All effects must display at least two icons: one that shows how the device applies mono input signals, and one that shows how the device applies stereo input signals.

All applicable icons must be displayed. For example, a device might pan a mono input signal or not depending the setting of some GUI control. In that case, the device must display two mono routing icons on the back panel (mono in/mono out, and mono in/stereo out).

Pattern automation

Devices with a built-in sequencer can have different patterns (separate sequences or rhythms). To allow the user to control this, Jukebox has special Pattern Automation properties and clips. This system is based on the original pattern system in for example Redrum, and supports up to 32 patterns. The currently selected pattern is encoded into the property `/transport/pattern_index`. If no pattern is selected, the property has the value `kJBox_NoPatternIndex` (-1).

Pattern Clips

To the user, Pattern Clips selects what pattern is currently played. Pattern Clips live in the Pattern Lane in the sequencer, which you can create by clicking the Create Pattern Lane button above the Track List. Each Pattern Clip has a dropdown menu to select what pattern it should play. If there is no Pattern Clip at the Play Position, the device should not play. When the Play Position reaches the left edge of a Pattern Clip, the `/transport/pattern_index` is updated with the current pattern, and the device should start playing.

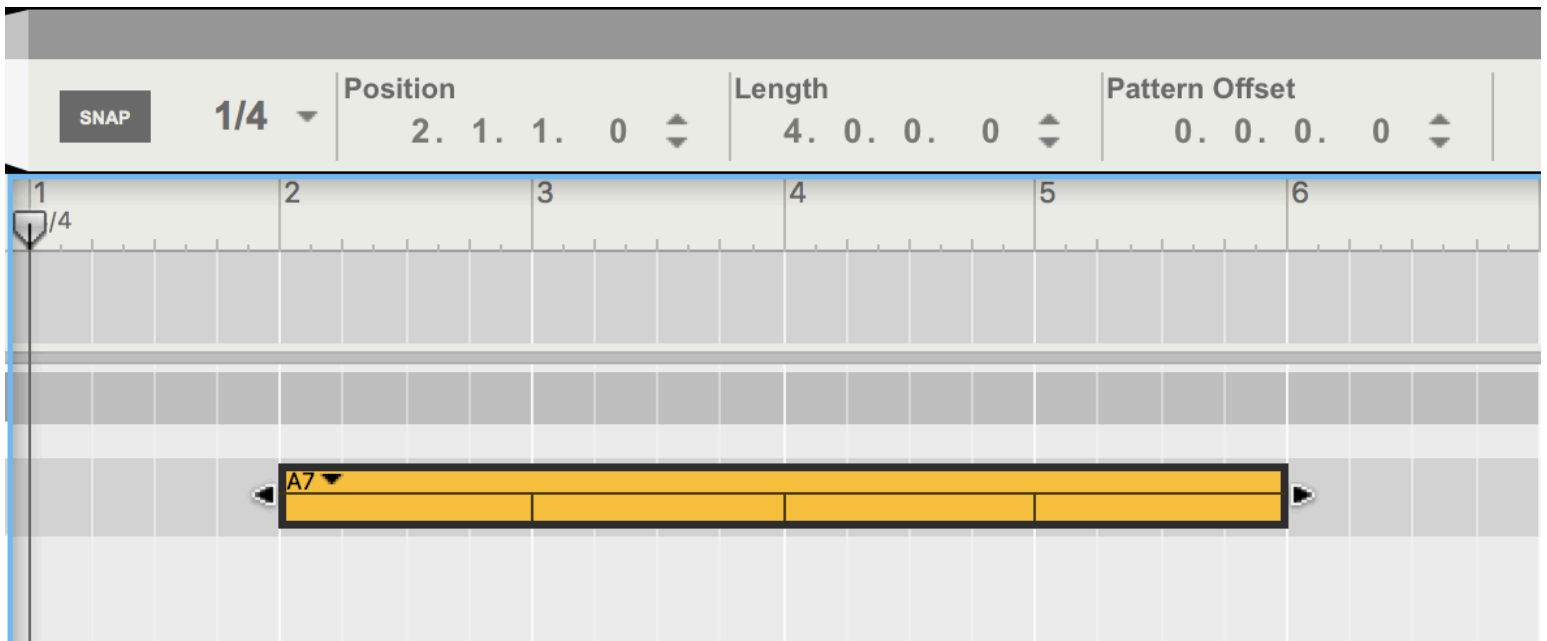
Pattern Offset

If the user resizes a Pattern Clip on its left edge, the Pattern Clip won't necessarily begin on beat 1 in the pattern. Any device that supports Pattern Automation should also support Pattern Offset, so that the pattern starts playback at the correct beat within the Pattern Clip. To show this to the user, there are vertical lines in the Pattern Clip showing beat 1, and a parameter called Pattern Offset in the sequencer Inspector.

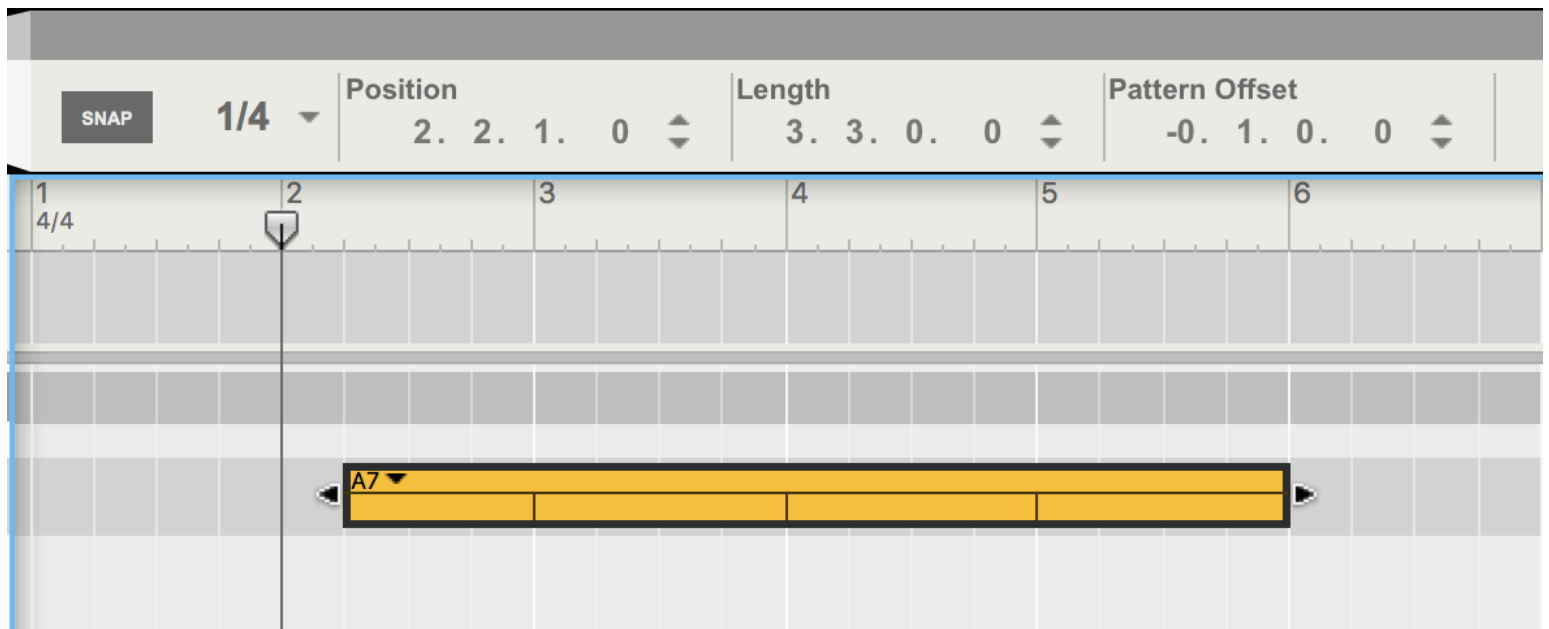
In Jukebox, Pattern Offset is encoded in the property called `/transport/pattern_start_pos`. It holds the PPQ where beat 1 of the first repetition of the Pattern in the Pattern Clip is located, regardless if the Pattern Clip is resized or not. This property should only affect pattern playback when the master sequencer transport is playing.

Example:

1. Draw a Pattern Clip with Pattern A7 at 2.1.1.0 (PPQ 61440):



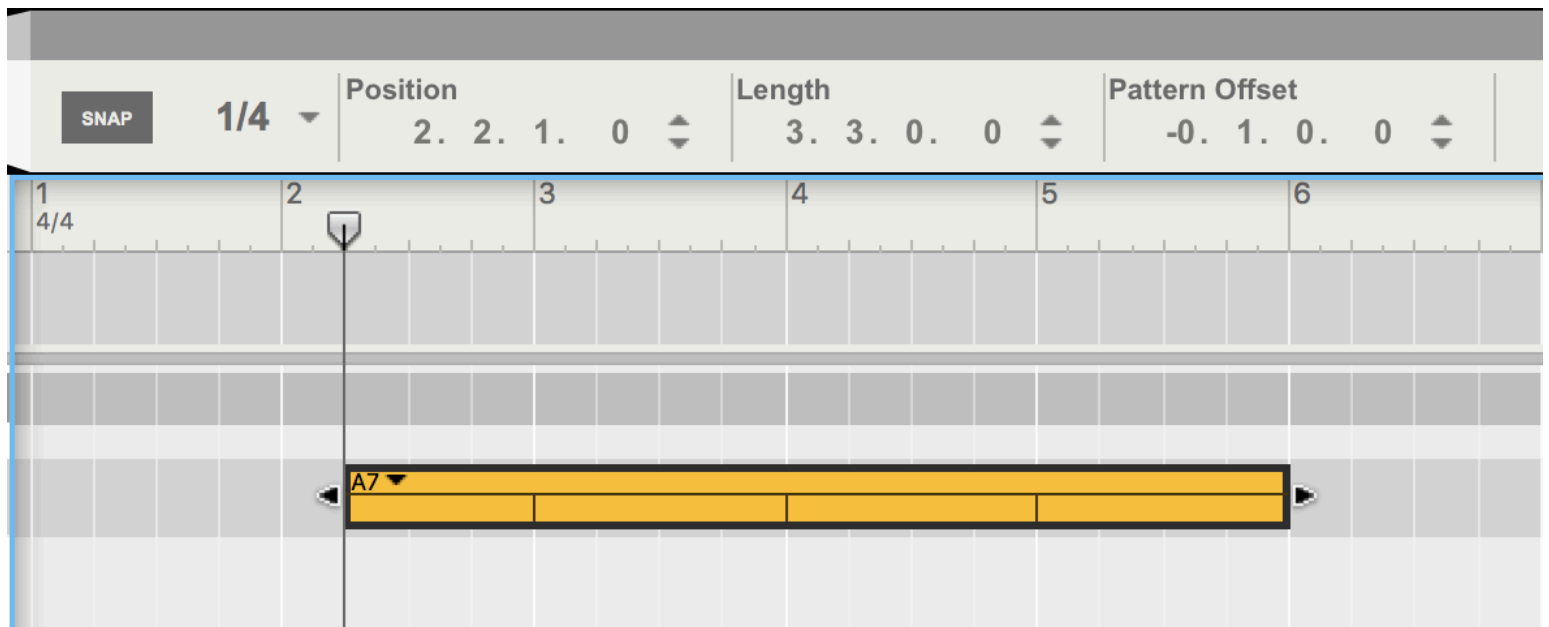
2. Resize the Pattern Clip so that it starts at 2.2.1.0. Notice that Pattern Offset is now -0.1.0.0 (PPQ -15360):



3. Before position 2.1.1.0, the properties have the following values:

- `/transport/pattern_index = kJBox_kJBox_NoPatternIndex`
- `/transport/pattern_start_pos = 0` (there is no pattern to play)

4. The Play Position reaches 2.2.1.0 (PPQ 76800)



At this point the properties are updated accordingly: - `/transport/pattern_index = 6` (pattern 7 is selected) - `/transport/pattern_start_pos = 76800` (this is where beat 1 of the pattern should be anchored)

realtime_controller.lua

The `realtime_controller.lua` file contains three different pieces of information:

- A set of Lua functions that configure the Runtime and the list of properties that trigger these functions.
- A list of sample rates that are "native" to the 45, and a list of sample rates for which the 45 uses resampling.
- A list of properties that should receive diffs in `JBox_Export_RenderRealtime()`

It has the following format:

```
format_version = "1.0"

rtc_bindings = {
    ...
}

global_rtc = {
    ...
}

sample_rate_setup = {
    ...
}

rt_input_setup = {
    ...
}
```

ENTRY	DESCRIPTION
<code>format_version</code>	File format version of the <code>realtime_controller.lua</code> file. Must be "1.0". Required.
<code>rtc_bindings</code>	List of associations between properties and Lua functions. Optional. See section [RTC_bindings] for details.
<code>global_rtc</code>	List of Realtime Controller Lua functions. Optional. See section [RTC_bindings] for details.
<code>sample_rate_setup</code>	List of natively supported sample rates and sample rates for which the 45 uses resampling. Required. See section [Sample_rate_configuration] for details.
<code>rt_input_setup</code>	List of properties. These properties will receive diffs in <code>JBox_Export_RenderRealtime()</code> . Optional. See section [Realtime_property_change_notification] for details.

Realtime controller bindings

The realtime controller (RTC) bindings section of `realtime_controller.lua` contains two tables. The first table, `rtc_bindings`, contains a list of properties that the RTC should "listen" to and the Lua function(s) to call when these properties change. The second table, `global_rtc`, contains the Lua functions.

The format of `rtc_bindings` is as follows:

None

```
rtc_bindings = {  
  {  
    source = ... ,  
    dest = ...  
  },  
  ...  
}
```

COMPONENT

DESCRIPTION

source	The property path for a property to "listen" to. Required. The path must point to a property in the <code>custom_properties</code> property set object, or to the <code>"/environment/system_sample_rate"</code> property.
dest	Path to the Lua function to call when the <code>source</code> property changes. Required. Must have the format <code>"/global_rtc/<FUNCTION>"</code> , where <code><FUNCTION></code> is the name of an entry in the <code>global_rtc</code> table.

The format of `global_rtc` is as follows:

None

```
global_rtc = {  
  
  my_lua_function = function(source_property_path, source_property_value)  
    ...  
  end  
}
```

```
end,  
  
...  
}
```

Each Lua function in the table must accept the same arguments: the source property path and the new property value. The Lua function may only read properties that are bound to it in `rtc_bindings`. The Lua function may write to any `rtc_owner` property. For more information about Lua toolbox restrictions, see section [\[Lua_toolbox\]](#).

Binding more than one property to the same Lua function is allowed, e.g.,

None

```
rtc_bindings = {  
  {  
    source = "/custom_properties/myProperty1" ,  
    dest = "/global_rtc/myFunction"  
  },  
  {  
    source = "/custom_properties/myProperty2" ,  
    dest = "/global_rtc/myFunction"  
  },  
}
```

You may not, however, bind a property to more than one Lua function.

Sample rate configuration

All 45s must support the following sample rates: 22050, 44100, 48000, 88200, 96000, and 192000 Hz. At least one of these must be supported natively. If the 45 does internal resampling to support one of these rates, that rate should be declared as converted.

The 45 declares its sample rate configuration in a table called `sample_rate_setup`. It has the following format:

None

```
sample_rate_setup = {  
  native = {  
    ...  
  },  
  converted = {  
    ...  
  }  
}
```

where the entries in the `native` table are the sample rates that are native to the 45 and the entries in the `converted` table are the sample rates that are converted. All sample rates listed above must be present in `sample_rate_setup`. Every sample rate must exist in either `native` or `converted`, not both.

Example:

None

```
sample_rate_setup = {
    native = {
        22050,
        44100,
        48000,
        96000,
        192000
    },
    converted = {
        88200
    }
}
```

Realtime property change notification

The 45 must declare which properties it needs to receive notification for (as property diffs) in the Realtime code. These properties are listed in the `rt_input_setup` table. It has the following format:

None

```
rt_input_setup = {
    notify = {
        ...
    }
}
```

where the entries in the `notify` sub-table are the properties to receive notification for. You can specify that an entire property set object should receive notifications by using the `*wildcard`, e.g.,

None

```
rt_input_setup = {  
    notify = {  
        "/note_states/*",  
    }  
}
```

An initial diff will be generated for all properties that are listed in the `notify` list when the 45 instance is created (the diffs will contain default value of the properties).

hdgui_2D.lua

The `hdgui_2D.lua` file contains bindings between 2D GUI resources and host UI widgets.

The file have the following basic structure:

None

```
format_version = "2.0"  
front = jbox.panel{  
    ...  
}  
back = jbox.panel{  
    ...  
}  
folded_front = jbox.panel{  
    ...  
}  
folded_back = jbox.panel{  
    ...  
}
```

ENTRY	DESCRIPTION
<code>format_version</code>	File format version of the <code>hdgui_2D.lua+</code> file. Should be "2.0" for <code>hdgui_2D.lua</code> with this version of the SDK. Required.
<code>front</code>	Widget bindings for the front panel. Required.
<code>back</code>	Widget bindings for the back panel. Required.
<code>folded_front</code>	Widget bindings for the folded front panel. Required.
<code>folded_back</code>	Widget bindings for the folded back panel. Required.

The `jbox.panel` object contains widget definitions for a panel. It has the following format:

```
None
... = jbox.panel{
  graphics = { ... },
  cable_origin = { ... },
  options = { ... },
  widgets = { ... },
}
```

COMPONENT	DESCRIPTION
<code>graphics</code>	The graphical representation for the widget. Required.
<code>cable_origin</code>	The graphics definition that contains the cable origin. Required for the folded back panel. Must not be present in any other panel specification.
<code>options</code>	List of option-strings - currently only supports "disable_sample_drop_on_panel" which disables the default drag and drop area for samples, which covers the panel of any device that has at least one user_sample. Meant for devices which use the sample_drop_zone widget. Optional for the front panel. Must not be present in any other panel specification.
<code>widgets</code>	The list of widget definitions for the panel. Required. May be empty.

[[2D_widget_graphics_specification]] === 2D Widget graphics specification

In the 2D GUI pipeline (`hdgui_2D.lua`), the graphical representation for a panel or a widget is specified in a table with the following format:

```
None
graphics = {
    node = "...",

    hit_boundaries = {
        left = ... ,
```

```

    top = ... ,
    right = ... ,
    bottom = ...
},
}

```

COMPONENT	DESCRIPTION
<code>node</code>	Identifier of a graphics definition in <code>device_2D.lua</code> . Required.
<code>hit_boundaries</code>	Rectangle that specifies the extents of the interactive area for this widget. Optional. Distances are specified as inwards offsets in pixels from the respective edge of the widget rectangle. The distance values must be zero or positive. If <code>hit_boundaries</code> are not specified, the widget rectangle will be used.

Widget boundaries may not overlap, except for `static_decoration` widgets or widgets with a visibility switch (which can not overlap if it is possible that they are visible at the same time).

The `hit_boundaries` are not used for the `sequence_fader` widget.

For `static_decoration` widgets it is worth noting that all widgets will be drawn in the order they are defined in `hdgui_2D.lua`, which means that if they are to be used as backgrounds for another widget they should be defined before it.

Reason-specific information

For Reason, `hit_boundaries` also define the automation rectangle.

Fonts

Some of the GUI widgets in Jukebox draw text strings. The font for these widgets can be chosen from the list below. Note that the exact look of the text styles may vary between hosts, platforms, and operating systems.

TEXT STYLE	DESCRIPTION	EXAMPLE
<code>LCD font</code>	Regular LCD-display style font	
<code>Bold LCD font</code>	Bold LCD-display style font	

Small LCD font

Small LCD-display style font

Big bold LCD font

Larger bold LCD-display style
font

Huge bold LCD font

Even larger bold LCD-display
style font



Label font

Hand-writing font suitable for
tape labels

Small label font

A small font suitable for small
labels



Arial small font -

Arial medium small font -

Arial medium font -

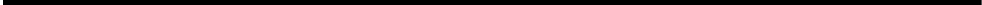
Arial medium bold font -

Arial medium large font -



Arial medium large bold font -





Arial large font -



Arial large bold font -



Arial medium large
toolbar font -



analog_knob

A standard knob.

None

```
jbox.analog_knob{
  graphics = { ... },

  value = ... ,

  value_switch = ... ,
  values = { ... },

  show_remote_box = ... ,
  show_automation_rect = ... ,
  tooltip_position = ... ,
  tooltip_template = { ... },

  visibility_switch = ... ,
  visibility_values = { ... } ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
<code>graphics</code>	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.

value	Property path to the <code>document_owner</code> or <code>gui_owner</code> property to which this widget should be connected. Optional. The property must be a number or boolean property. If <code>value</code> is not specified, then <code>values</code> and <code>value_switch</code> must be specified.
value_switch	Property path to a <code>document_owner</code> or <code>gui_owner</code> discrete (stepped) number property. The value of the <code>value_switch</code> property determines (dynamically) which of the properties in the <code>values</code> list (see below) to which this widget should be connected. Optional. If <code>value_switch</code> is specified, then <code>values</code> must be specified too. If <code>value_switch</code> is not specified, then <code>value</code> must be specified (and <code>values</code> omitted). The <code>steps</code> setting in the <code>value_switch</code> property must match the length of the <code>values</code> list.
values	List of property paths to <code>document_owner</code> or <code>gui_owner</code> properties to which this widget should be (dynamically) connected. Optional. The properties must be number or boolean properties. If <code>values</code> is specified, then <code>value_switch</code> must be specified too. If <code>values</code> is not specified, then <code>value</code> must be specified (and <code>value_switch</code> omitted). The length of the <code>values</code> list must match the <code>steps</code> setting in the <code>value_switch</code> property. All properties in <code>values</code> must have the same type.
show_remote_box	If <code>true</code>, instruct the host to show a GUI indication if this widget is under Remote™ control. Optional. If <code>show_remote_box</code> is not specified, it defaults to <code>true</code>. The setting of this attribute may be ignored by the host.

<code>show_automation_ rect</code>	If <code>true</code> , instruct the host to show a GUI indication if this widget is automated. Optional. If <code>show_automation_rect</code> is not specified, it defaults to <code>true</code> . The setting of this attribute may be ignored by the host.
--	--

<code>tooltip_position</code>	Specifies the position of the tooltip for this widget (for hosts that are capable of showing tooltips). Optional. Must be one of the strings "bottom_left", "bottom", "bottom_right", "right", "top_right", "top", "top_left", "left" or "no_tooltip" (disables tooltips for this widget). If <code>tooltip_position</code> is not specified, the host is free to choose any tooltip position. The setting of this attribute may be ignored by the host.
-------------------------------	--

<code>tooltip_template</code>	Formatting string for the tooltip text. Optional. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string. If the substring <code>^0</code> is present in the string, it is replaced by the (localized) property value. If the substring <code>^1</code> is present in the string, it is replaced by the (localized) property name. The <code>^0</code> and <code>^1</code> substrings may appear only once in the formatting string. If <code>tooltip_template</code> is not specified, the host is free to choose the formatting. The setting of this attribute may be ignored by the host.
-------------------------------	--

<code>visibility_switc h</code>	Property path to a discrete (stepped) number property. The value of the <code>visibility_switch</code> property determines (dynamically) whether the widget is shown or hidden. Optional. If <code>visibility_switch</code> is specified, then <code>visibility_values</code> must be specified too. When the value of the <code>visibility_switch</code> property matches one of the values in <code>visibility_values</code> the widget is shown, otherwise it is hidden. This setting is only valid for unfolded panels.
-------------------------------------	---

`visibility_value`
`s`

The list of values for which the widget should be visible if the `visibility_switch` property matches one them.
Optional.

zero_snap_knob

A knob that snaps to the animation frame that most closely corresponds to the zero value of a number or boolean property (after the `ui_type` mapping has been applied, see [\[ui_type\]](#) for details).

None

```
jbox.zero_snap_knob{
  graphics = { ... },

  value = ... ,

  value_switch = ... ,
  values = { ... },

  show_remote_box = ... ,
  show_automation_rect = ... ,
  tooltip_position = ... ,
  tooltip_template = { ... },

  visibility_switch = ... ,
  visibility_values = { ... } ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
graphics	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
value	Property path to the <code>document_owner</code> or <code>gui_owner</code> property to which this widget should be connected. Optional. The property must be a number or boolean property. If <code>value</code> is not specified, then <code>values</code> and <code>value_switch</code> must be specified.
value_switch	Property path to a <code>document_owner</code> or <code>gui_owner</code> discrete (stepped) number property. The value of the <code>value_switch</code> property determines (dynamically) which of the properties in the <code>values</code> list (see below) to which this widget should be connected. Optional. If <code>value_switch</code> is specified, then <code>values</code> must be specified too. If <code>value_switch</code> is not specified, then <code>value</code> must be specified (and <code>values</code> omitted). The <code>steps</code> setting in the <code>value_switch</code> property must match the length of the <code>values</code> list.
values	List of property paths to <code>document_owner</code> or <code>gui_owner</code> properties to which this widget should be (dynamically) connected. Optional. The properties must be number or boolean properties. If <code>values</code> is specified, then <code>value_switch</code> must be specified too. If <code>values</code> is not specified, then <code>value</code> must be specified (and <code>value_switch</code> omitted). The length of the <code>values</code> list must match the <code>steps</code> setting in the <code>value_switch</code> property. All properties in <code>values</code> must have the same type.

`show_remote_box`

If `true`, instruct the host to show a GUI indication if this widget is under Remote™ control. Optional. If `show_remote_box` is not specified, it defaults to `true`. The setting of this attribute may be ignored by the host.

`show_automation_
rect`

If `true`, instruct the host to show a GUI indication if this widget is automated. Optional. If `show_automation_rect` is not specified, it defaults to `true`. The setting of this attribute may be ignored by the host.

`tooltip_position`

Specifies the position of the tooltip for this widget (for hosts that are capable of showing tooltips). Optional. Must be one of the strings "bottom_left", "bottom", "bottom_right", "right", "top_right", "top", "top_left", "left" or "no_tooltip" (disables tooltips for this widget). If `tooltip_position` is not specified, the host is free to choose any tooltip position. The setting of this attribute may be ignored by the host.

`tooltip_template`

Formatting string for the tooltip text. Optional. You must use the `jbox.ui_text()` toolbox function to obtain the string. If the substring `^0` is present in the string, it is replaced by the (localized) property value. If the substring `^1` is present in the string, it is replaced by the (localized) property name. The `^0` and `^1` substrings may appear only once in the formatting string. If `tooltip_template` is not specified, the host is free to choose the formatting. The setting of this attribute may be ignored by the host.

`visibility_switch`

Property path to a discrete (stepped) number property. The value of the `visibility_switch` property determines (dynamically) whether the widget is shown or hidden. Optional. If `visibility_switch` is specified, then `visibility_values` must be specified too. When the value of the `visibility_switch` property matches one of the values in `visibility_values` the widget is shown, otherwise it is hidden. This setting is only valid for unfolded panels.

`visibility_values`

The list of values for which the widget should be visible if the `visibility_switch` property matches one them. Optional.

cv_trim_knob

Knob that is used to trim the level of CV input signals.

You cannot change the appearance of this knob. Use the `Trim_Knob.sg` scenegraph in the Jukebox Part Library to specify its location.

None

```
jbox.cv_trim_knob{
  graphics = { ... },

  socket = ... ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
graphics	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
socket	The property path of a socket in the "/cv_inputs" property set object. Required.

pitch_wheel

A knob that is used for controlling `jbox.performance_pitchbend` properties. The widget will snap back to the midpoint of the animation when released.

```
None
jbox.pitch_wheel{
  graphics = { ... },

  value = ... ,

  show_remote_box = ... ,
  show_automation_rect = ... ,
  tooltip_position = ... ,
  tooltip_template = { ... },
```

```
visibility_switch = ... ,  
visibility_values = { ... } ,  
,
```

Attributes.

ATTRIBUTE	DESCRIPTION
<code>graphics</code>	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
<code>value</code>	Property path to the <code>document_owner</code> or <code>gui_owner</code> property to which this widget should be connected. Required. The property must be <code>ajbox.performance_pitchbend</code> property.
<code>show_remote_box</code>	If <code>true</code> , instruct the host to show a GUI indication if this widget is under Remote™ control. Optional. If <code>show_remote_box</code> is not specified, it defaults to <code>true</code> . The setting of this attribute may be ignored by the host.

<code>show_automation_ rect</code>	If <code>true</code> , instruct the host to show a GUI indication if this widget is automated. Optional. If <code>show_automation_rect</code> is not specified, it defaults to <code>true</code> . The setting of this attribute may be ignored by the host.
--	--

<code>tooltip_position</code>	Specifies the position of the tooltip for this widget (for hosts that are capable of showing tooltips). Optional. Must be one of the strings <code>"bottom_left"</code> , <code>"bottom"</code> , <code>"bottom_right"</code> , <code>"right"</code> , <code>"top_right"</code> , <code>"top"</code> , <code>"top_left"</code> , <code>"left"</code> or <code>"no_tooltip"</code> (disables tooltips for this widget). If <code>tooltip_position</code> is not specified, the host is free to choose any tooltip position. The setting of this attribute may be ignored by the host.
-------------------------------	--

<code>tooltip_template</code>	Formatting string for the tooltip text. Optional. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string. If the substring <code>^0</code> is present in the string, it is replaced by the (localized) property value. If the substring <code>^1</code> is present in the string, it is replaced by the (localized) property name. The <code>^0</code> and <code>^1</code> substrings may appear only once in the formatting string. If <code>tooltip_template</code> is not specified, the host is free to choose the formatting. The setting of this attribute may be ignored by the host.
-------------------------------	--

<code>visibility_switc h</code>	Property path to a discrete (stepped) number property. The value of the <code>visibility_switch</code> property determines (dynamically) whether the widget is shown or hidden. Optional. If <code>visibility_switch</code> is specified, then <code>visibility_values</code> must be specified too. When the value of the <code>visibility_switch</code> property matches one of the values in <code>visibility_values</code> the widget is shown, otherwise it is hidden. This setting is only valid for unfolded panels.
-------------------------------------	---

`visibility_value`
`s`

The list of values for which the widget should be visible if the `visibility_switch` property matches one them. Optional.

toggle_button

A button that toggles the value of a discrete (stepped) number property between 0.0 (unchecked) and (`steps - 1.0`) (checked) for every push. For boolean properties, the button toggles between `false` (unchecked) and `true` (checked).

The animation for this button must contain two or four frames.

If the animation contains two frames, the first is used when the button is unchecked and the second when the button is checked. If the animation contains four frames, the first frame is used when the button is unchecked (off), the second frame when the button is unchecked (off) and held by the user, the third when the button is checked (on), and the fourth is used when the button is checked (on) and held by the user.

None

```
jbox.toggle_button{
  graphics = { ... },

  value = ... ,

  show_remote_box = ... ,
  show_automation_rect = ... ,
  tooltip_position = ... ,
  tooltip_template = { ... },
```



```
visibility_switch = ... ,
visibility_values = { ... } ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
<code>graphics</code>	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
<code>value</code>	Property path to the <code>document_owner</code> or <code>gui_owner</code> property to which this widget should be connected. Required. The property must be a discrete (stepped) number or boolean property.
<code>show_remote_box</code>	If <code>true</code> , instruct the host to show a GUI indication if this widget is under Remote™ control. Optional. If <code>show_remote_box</code> is not specified, it defaults to <code>true</code> . The setting of this attribute may be ignored by the host.

<code>show_automation_ rect</code>	If <code>true</code> , instruct the host to show a GUI indication if this widget is automated. Optional. If <code>show_automation_rect</code> is not specified, it defaults to <code>true</code> . The setting of this attribute may be ignored by the host.
--	--

<code>tooltip_position</code>	Specifies the position of the tooltip for this widget (for hosts that are capable of showing tooltips). Optional. Must be one of the strings " <code>bottom_left</code> ", " <code>bottom</code> ", " <code>bottom_right</code> ", " <code>right</code> ", " <code>top_right</code> ", " <code>top</code> ", " <code>top_left</code> ", " <code>left</code> " or " <code>no_tooltip</code> " (disables tooltips for this widget). If <code>tooltip_position</code> is not specified, the host is free to choose any tooltip position. The setting of this attribute may be ignored by the host.
-------------------------------	---

<code>tooltip_template</code>	Formatting string for the tooltip text. Optional. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string. If the substring <code>^0</code> is present in the string, it is replaced by the (localized) property value. If the substring <code>^1</code> is present in the string, it is replaced by the (localized) property name. The <code>^0</code> and <code>^1</code> substrings may appear only once in the formatting string. If <code>tooltip_template</code> is not specified, the host is free to choose the formatting. The setting of this attribute may be ignored by the host.
-------------------------------	--

<code>visibility_switc h</code>	Property path to a discrete (stepped) number property. The value of the <code>visibility_switch</code> property determines (dynamically) whether the widget is shown or hidden. Optional. If <code>visibility_switch</code> is specified, then <code>visibility_values</code> must be specified too. When the value of the <code>visibility_switch</code> property matches one of the values in <code>visibility_values</code> the widget is shown, otherwise it is hidden.
-------------------------------------	---

`visibility_value`
`s`

The list of values for which the widget should be visible if the `visibility_switch` property matches one them. Optional.

momentary_button

A button that sets the value of a discrete (stpped) number property to `(steps - 1.0)` when held and `0.0` when released. For boolean properties, the button sets the property to `true` when held and `false` when released.

The animation for this button must contain two frames. The first frame is used when the button is released and the second when the button is held.

None

```
jbox.momentary_button{
  graphics = { ... },

  value = ... ,

  show_remote_box = ... ,
  show_automation_rect = ... ,
  tooltip_position = ... ,
  tooltip_template = { ... },

  visibility_switch = ... ,
  visibility_values = { ... } ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
<code>graphics</code>	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
<code>value</code>	Property path to the <code>document_owner</code> or <code>gui_owner</code> property to which this widget should be connected. Required. The property must be a discrete (stepped) number or boolean property.
<code>show_remote_box</code>	If <code>true</code> , instruct the host to show a GUI indication if this widget is under Remote™ control. Optional. If <code>show_remote_box</code> is not specified, it defaults to <code>true</code> . The setting of this attribute may be ignored by the host.
<code>show_automation_rect</code>	If <code>true</code> , instruct the host to show a GUI indication if this widget is automated. Optional. If <code>show_automation_rect</code> is not specified, it defaults to <code>true</code> . The setting of this attribute may be ignored by the host.
<code>tooltip_position</code>	Specifies the position of the tooltip for this widget (for hosts that are capable of showing tooltips). Optional. Must be one of the strings <code>"bottom_left"</code> , <code>"bottom"</code> , <code>"bottom_right"</code> , <code>"right"</code> , <code>"top_right"</code> , <code>"top"</code> , <code>"top_left"</code> , <code>"left"</code> or <code>"no_tooltip"</code> (disables tooltips for this widget). If <code>tooltip_position</code> is not specified, the host is free to choose any tooltip position. The setting of this attribute may be ignored by the host.

`tooltip_template`

Formatting string for the tooltip text. Optional. You must use the `jbox.ui_text()` toolbox function to obtain the string. If the substring `^0` is present in the string, it is replaced by the (localized) property value. If the substring `^1` is present in the string, it is replaced by the (localized) property name. The `^0` and `^1` substrings may appear only once in the formatting string. If `tooltip_template` is not specified, the host is free to choose the formatting. The setting of this attribute may be ignored by the host.

`visibility_switch`

Property path to a discrete (stepped) number property. The value of the `visibility_switch` property determines (dynamically) whether the widget is shown or hidden. Optional. If `visibility_switch` is specified, then `visibility_values` must be specified too. When the value of the `visibility_switch` property matches one of the values in `visibility_values` the widget is shown, otherwise it is hidden. This setting is only valid for unfolded panels.

`visibility_values`

The list of values for which the widget should be visible if the `visibility_switch` property matches one them. Optional.

step_button

A button that steps through the range of values of a discrete (stepped) number property. If the property is at its maximum value and the button is pressed again, the property will be set to 0.0.

The animation for this button must contain two frames. The first frame is used when the button is released and the second when the button is held.

None

```
jbox.step_button{
  graphics = { ... },

  value = ... ,
  increasing = ... ,

  show_remote_box = ... ,
  show_automation_rect = ... ,
  tooltip_position = ... ,
  tooltip_template = { ... },

  visibility_switch = ... ,
  visibility_values = { ... } ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
graphics	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
value	Property path to the <code>document_owner</code> or <code>gui_owner</code> property to which this widget should be connected. Required. The property must be a discrete (stepped) number or boolean property.

<code>increasing</code>	Specifies whether the property value should be increased (<code>true</code>) or decreased (<code>false</code>) when the button is pressed Optional. If <code>increasing</code> is omitted, it will default to <code>true</code> .
<code>show_remote_box</code>	If <code>true</code> , instruct the host to show a GUI indication if this widget is under Remote™ control. Optional. If <code>show_remote_box</code> is not specified, it defaults to <code>true</code> . The setting of this attribute may be ignored by the host.
<code>show_automation_rect</code>	If <code>true</code> , instruct the host to show a GUI indication if this widget is automated. Optional. If <code>show_automation_rect</code> is not specified, it defaults to <code>true</code> . The setting of this attribute may be ignored by the host.
<code>tooltip_position</code>	Specifies the position of the tooltip for this widget (for hosts that are capable of showing tooltips). Optional. Must be one of the strings "bottom_left", "bottom", "bottom_right", "right", "top_right", "top", "top_left", "left" or "no_tooltip" (disables tooltips for this widget). If <code>tooltip_position</code> is not specified, the host is free to choose any tooltip position. The setting of this attribute may be ignored by the host.
<code>tooltip_template</code>	Formatting string for the tooltip text. Optional. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string. If the substring <code>^0</code> is present in the string, it is replaced by the (localized) property value. If the substring <code>^1</code> is present in the string, it is replaced by the (localized) property name. The <code>^0</code> and <code>^1</code> substrings may appear only once in the formatting string. If <code>tooltip_template</code> is not specified, the host is free to

choose the formatting. The setting of this attribute may be ignored by the host.

`visibility_switch`

Property path to a discrete (stepped) number property. The value of the `visibility_switch` property determines (dynamically) whether the widget is shown or hidden. Optional. If `visibility_switch` is specified, then `visibility_values` must be specified too. When the value of the `visibility_switch` property matches one of the values in `visibility_values` the widget is shown, otherwise it is hidden. This setting is only valid for unfolded panels.

`visibility_values`

The list of values for which the widget should be visible if the `visibility_switch` property matches one them. Optional.

radio_button

A button that is a member of a radio button group. All `radio_button` widgets that are assigned to the same property are considered to be in the same group. When one of the buttons in the group is pressed (checked), all other buttons in the group are released (unchecked).

The animation for this button must contain two frames. The first frame is used when the button is released and the second when the button is held.

None

```
jbox.radio_button{
```



```

graphics = { ... },

value = ... ,
index = ... ,

show_remote_box = ... ,
show_automation_rect = ... ,
tooltip_position = ... ,
tooltip_template = { ... },

visibility_switch = ... ,
visibility_values = { ... } ,
},

```

Attributes.

ATTRIBUTE	DESCRIPTION
<code>graphics</code>	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
<code>value</code>	Property path to the <code>document_owner</code> or <code>gui_owner</code> property to which this widget should be connected. Required. The property must be a discrete (stepped) number or boolean property.

<code>index</code>	Specifies the value to set the property to when this button is pressed. Required. Must be an integer in the range <code>[0, ... , steps - 1]</code> where <code>steps</code> is the number of discrete steps of the property.
<code>show_remote_box</code>	If <code>true</code> , instruct the host to show a GUI indication if this widget is under Remote™ control. Optional. If <code>show_remote_box</code> is not specified, it defaults to <code>true</code> . The setting of this attribute may be ignored by the host.
<code>show_automation_rect</code>	If <code>true</code> , instruct the host to show a GUI indication if this widget is automated. Optional. If <code>show_automation_rect</code> is not specified, it defaults to <code>true</code> . The setting of this attribute may be ignored by the host.
<code>tooltip_position</code>	Specifies the position of the tooltip for this widget (for hosts that are capable of showing tooltips). Optional. Must be one of the strings <code>"bottom_left"</code> , <code>"bottom"</code> , <code>"bottom_right"</code> , <code>"right"</code> , <code>"top_right"</code> , <code>"top"</code> , <code>"top_left"</code> , <code>"left"</code> or <code>"no_tooltip"</code> (disables tooltips for this widget). If <code>tooltip_position</code> is not specified, the host is free to choose any tooltip position. The setting of this attribute may be ignored by the host.
<code>tooltip_template</code>	Formatting string for the tooltip text. Optional. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string. If the substring <code>^0</code> is present in the string, it is replaced by the (localized) property value. If the substring <code>^1</code> is present in the string, it is replaced by the (localized) property name. The <code>^0</code> and <code>^1</code> substrings may appear only once in the formatting string. If <code>tooltip_template</code> is not specified, the host is free to

choose the formatting. The setting of this attribute may be ignored by the host.

`visibility_switch`

Property path to a discrete (stepped) number property. The value of the `visibility_switch` property determines (dynamically) whether the widget is shown or hidden. Optional. If `visibility_switch` is specified, then `visibility_values` must be specified too. When the value of the `visibility_switch` property matches one of the values in `visibility_values` the widget is shown, otherwise it is hidden. This setting is only valid for unfolded panels.

`visibility_values`

The list of values for which the widget should be visible if the `visibility_switch` property matches one them. Optional.

up_down_button

A "seesaw" button that can be used to increase (+1) or decrease (-1) the value of a discrete (stepped) number property. The property value does not "wrap around" when at its maximum or minimum.

If the number of steps for the property is ≥ 200 , the delta increase/decrease will be $(1/100)$ of the range of the property.

If the user holds the button, the value is updated continuously until it reaches its maximum/minimum value.

When the user interacts within the top half of the widget boundaries, the value is increased. When the user interacts with the bottom half of the widget boundaries, the value is decreased. (If the optional `inverted` flag is set to `true`, however, the top half of the widget decreases the value and vice versa.)

The animation must contain three frames. The first frame corresponds to when the button is in the neutral position. The second frame corresponds to when the user holds the "up" part of the button. The third frame corresponds to when the user holds the "down" part of the button.

None

```
jbox.up_down_button{
  graphics = { ... },

  value = ... ,
  inverted = ... ,

  show_remote_box = ... ,
  show_automation_rect = ... ,
  tooltip_position = ... ,
  tooltip_template = { ... },

  visibility_switch = ... ,
  visibility_values = { ... } ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
-----------	-------------

<code>graphics</code>	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
-----------------------	---

<code>value</code>	Property path to the <code>document_owner</code> or <code>gui_owner</code> property to which this widget should be connected. Required. The property must be a discrete (stepped) number property.
--------------------	--

<code>inverted</code>	If <code>true</code> , then clicking in the upper part of the widget will decrease the property value and vice versa. Optional.
-----------------------	---

<code>show_remote_box</code>	If <code>true</code> , instruct the host to show a GUI indication if this widget is under Remote™ control. Optional. If <code>show_remote_box</code> is not specified, it defaults to <code>true</code> . The setting of this attribute may be ignored by the host.
------------------------------	---

<code>show_automation_rect</code>	If <code>true</code> , instruct the host to show a GUI indication if this widget is automated. Optional. If <code>show_automation_rect</code> is not specified, it defaults to <code>true</code> . The setting of this attribute may be ignored by the host.
-----------------------------------	--

<code>tooltip_position</code>	Specifies the position of the tooltip for this widget (for hosts that are capable of showing tooltips). Optional. Must be one of the strings " <code>bottom_left</code> ", " <code>bottom</code> ", " <code>bottom_right</code> ", " <code>right</code> ", " <code>top_right</code> ", " <code>top</code> ", " <code>top_left</code> ", " <code>left</code> " or " <code>no_tooltip</code> " (disables tooltips for this widget). If <code>tooltip_position</code> is not specified, the
-------------------------------	--

host is free to choose any tooltip position. The setting of this attribute may be ignored by the host.

`tooltip_template`

Formatting string for the tooltip text. Optional. You must use the `jbox.ui_text()` toolbox function to obtain the string. If the substring `^0` is present in the string, it is replaced by the (localized) property value. If the substring `^1` is present in the string, it is replaced by the (localized) property name. The `^0` and `^1` substrings may appear only once in the formatting string. If `tooltip_template` is not specified, the host is free to choose the formatting. The setting of this attribute may be ignored by the host.

`visibility_switch`

Property path to a discrete (stepped) number property. The value of the `visibility_switch` property determines (dynamically) whether the widget is shown or hidden. Optional. If `visibility_switch` is specified, then `visibility_values` must be specified too. When the value of the `visibility_switch` property matches one of the values in `visibility_values` the widget is shown, otherwise it is hidden. This setting is only valid for unfolded panels.

`visibility_values`

The list of values for which the widget should be visible if the `visibility_switch` property matches one them. Optional.

popup_button

An interactive text area that can be used to select among the values of a discrete (stepped) number property. When the user interacts (pushes) inside the widget, the host displays a menu that contains all available values for the property (after they have been mapped using `ui_type`, see [\[ui_type\]](#)). The value chosen by the user is displayed as text in the text area.

None

```
jbox.popup_button{
  graphics = { ... },

  value = ... ,

  text_style = ... ,
  text_color = { ... },

  show_remote_box = ... ,
  show_automation_rect = ... ,

  visibility_switch = ... ,
  visibility_values = { ... } ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
graphics	Defines the graphics defintion to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
value	Property path to the <code>document_owner</code> or <code>gui_owner</code> property to which this widget should be connected. Required. The property must be a discrete (stepped) number or boolean property.

`text_style`

The font style to use for drawing text and/or numbers into the text area. Required. Must be one of the built-in Jukebox fonts, see section [\[Fonts\]](#).

`text_color`

The color of the font to use for drawing text and/or numbers into the text area. Required. Must be a Lua table with three integer values (`{red, green, blue}`) in the range [0, 255].

`show_remote_box`

If `true`, instruct the host to show a GUI indication if this widget is under Remote™ control. Optional. If `show_remote_box` is not specified, it defaults to `true`. The setting of this attribute may be ignored by the host.

`show_automation_
rect`

If `true`, instruct the host to show a GUI indication if this widget is automated. Optional. If `show_automation_rect` is not specified, it defaults to `true`. The setting of this attribute may be ignored by the host.

`visibility_switc
h`

Property path to a discrete (stepped) number property. The value of the `visibility_switch` property determines (dynamically) whether the widget is shown or hidden. Optional. If `visibility_switch` is specified, then `visibility_values` must be specified too. When the value of the `visibility_switch` property matches one of the values in `visibility_values` the widget is shown, otherwise it is hidden. This setting is only valid for unfolded panels.

`visibility_value`
`s`

The list of values for which the widget should be visible if the `visibility_switch` property matches one them. Optional.

sequence_fader

Horizontal or vertical fader.

The animation for this widget must contain some sort of handle that the user can drag. When the user presses outside of the fader handle (but still within the bounds of the widget), the fader handle will jump automatically to the location where the user pressed, assuming you have configured the widget correctly (see `inset1`, `inset2`, and `handle_size_mm/handle_size` below).

Note: `hit_boundaries` has no effect on the interaction area for this widget. (It is used for the automation indicator in Reason, though.)

None

```
jbox.sequence_fader{
  graphics = { ... },

  value = ... ,

  value_switch = ... ,
  values = { ... },

  orientation = ... ,
  inverted = ... ,
```

```

inset1 = ... ,
inset2 = ... ,
handle_size_mm = ... ,
handle_size = ... ,

show_remote_box = ... ,
show_automation_rect = ... ,
tooltip_position = ... ,
tooltip_template = { ... },

visibility_switch = ... ,
visibility_values = { ... } ,
},

```

Attributes.

ATTRIBUTE	DESCRIPTION
<code>graphics</code>	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline. The animation must include the entire travel distance of the handle. The handle must be in the leftmost / bottom position at the first animation frame. The handle must be in the rightmost / top position at the last animation frame. The amount the handle travels between each animation frame must be constant (i.e., travel distance is linear in frames). The widget boundaries must cover the area that the handle can travel, but may cover an additional area above/below (for vertical faders) or left/right (for horizontal faders) to include, e.g., shadows.

value	Property path to the <code>document_owner</code> or <code>gui_owner</code> property to which this widget should be connected. Optional. The property must be a number or boolean property. If <code>value</code> is not specified, then <code>values</code> and <code>value_switch</code> must be specified.
value_switch	Property path to a <code>document_owner</code> or <code>gui_owner</code> discrete (stepped) number property. The value of the <code>value_switch</code> property determines (dynamically) which of the properties in the <code>values</code> list (see below) to which this widget should be connected. Optional. If <code>value_switch</code> is specified, then <code>values</code> must be specified too. If <code>value_switch</code> is not specified, then <code>value</code> must be specified (and <code>values</code> omitted). The <code>steps</code> setting in the <code>value_switch</code> property must match the length of the <code>values</code> list.
values	List of property paths to <code>document_owner</code> or <code>gui_owner</code> properties to which this widget should be (dynamically) connected. Optional. The properties must be number or boolean properties. If <code>values</code> is specified, then <code>value_switch</code> must be specified too. If <code>values</code> is not specified, then <code>value</code> must be specified (and <code>value_switch</code> omitted). The length of the <code>values</code> list must match the <code>steps</code> setting in the <code>value_switch</code> property. All properties in <code>values</code> must have the same type.
orientation	Specifies the orientation of the fader. Optional. Must have the value <code>"vertical"</code> or <code>"horizontal"</code>. If <code>orientation</code> is omitted, it defaults to <code>"vertical"</code>.

`inverted`

If `true`, then the animation frames will be flipped so that if the animation has N frames, then frame N will be shown for the lowest property value and frame 1 will be shown for the highest property value. Optional. If `inverted` is omitted, it defaults to `false`.

`inset1`

Defines an area to exclude from the hit box of the widget. Optional. Specified in pixels (2D) from the top (vertical orientation) or left (horizontal orientation) edge of the widget boundaries. If `inset1` is omitted, it defaults to 0.

`inset2`

Defines an area to exclude from the hit box of the widget. Optional. Specified in pixels (2D) from the bottom (vertical orientation) or right (horizontal orientation) edge of the widget boundaries. If `inset2` is omitted, it defaults to 0.

`handle_size_mm`

Deprecated. *Specifies the height (vertical orientation) or width (horizontal orientation) of the handle (along the orientation axis). Optional. May only be used in the 3D GUI pipeline. Specified in millimeters. If `handle_size_mm` is omitted, it defaults to 0.*

`handle_size`

Specifies the height (vertical orientation) or width (horizontal orientation) of the handle (along the orientation axis). Optional. May only be used in the 2D GUI pipeline. Specified in pixels. If `handle_size` is omitted, it defaults to 0.

`show_remote_box`

If `true`, instruct the host to show a GUI indication if this widget is under Remote™ control. Optional. If `show_remote_box` is not specified, it defaults to `true`. The setting of this attribute may be ignored by the host.

`show_automation_
rect`

If `true`, instruct the host to show a GUI indication if this widget is automated. Optional. If `show_automation_rect` is not specified, it defaults to `true`. The setting of this attribute may be ignored by the host.

`tooltip_position`

Specifies the position of the tooltip for this widget (for hosts that are capable of showing tooltips). Optional. Must be one of the strings "bottom_left", "bottom", "bottom_right", "right", "top_right", "top", "top_left", "left" or "no_tooltip" (disables tooltips for this widget). If `tooltip_position` is not specified, the host is free to choose any tooltip position. The setting of this attribute may be ignored by the host.

`tooltip_template`

Formatting string for the tooltip text. Optional. You must use the `jbox.ui_text()` toolbox function to obtain the string. If the substring `^0` is present in the string, it is replaced by the (localized) property value. If the substring `^1` is present in the string, it is replaced by the (localized) property name. The `^0` and `^1` substrings may appear only once in the formatting string. If `tooltip_template` is not specified, the host is free to choose the formatting. The setting of this attribute may be ignored by the host.

visibility_switch

Property path to a discrete (stepped) number property. The value of the `visibility_switch` property determines (dynamically) whether the widget is shown or hidden. Optional. If `visibility_switch` is specified, then `visibility_values` must be specified too. When the value of the `visibility_switch` property matches one of the values in `visibility_values` the widget is shown, otherwise it is hidden. This setting is only valid for unfolded panels.

visibility_values

The list of values for which the widget should be visible if the `visibility_switch` property matches one them. Optional.

sequence_meter

Visual representation of a number or boolean property value. Used for dB meters, lamps, etc.

None

```
jbox.sequence_meter{
  graphics = { ... },

  value = ... ,

  visibility_switch = ... ,
  visibility_values = { ... } ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
<code>graphics</code>	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
<code>value</code>	Property path to the <code>rt_owner</code> property to which this widget should be connected. Required. The property must be a number or boolean property.
<code>visibility_switch</code>	Property path to a discrete (stepped) number property. The value of the <code>visibility_switch</code> property determines (dynamically) whether the widget is shown or hidden. Optional. If <code>visibility_switch</code> is specified, then <code>visibility_values</code> must be specified too. When the value of the <code>visibility_switch</code> property matches one of the values in <code>visibility_values</code> the widget is shown, otherwise it is hidden. This setting is only valid for unfolded panels.
<code>visibility_values</code>	The list of values for which the widget should be visible if the <code>visibility_switch</code> property matches one them. Optional.
value_display	

A text area that displays the value of a property as text. Supports user input.

None

```
jbox.value_display{
  graphics = { ... },

  value = ... ,

  value_switch = ... ,
  values = { ... },

  text_style = ... ,
  text_color = { ... },
  value_templates = { ... },
  horizontal_justification = ... ,
  read_only = ... ,

  show_remote_box = ... ,
  show_automation_rect = ... ,
  tooltip_position = ... ,
  tooltip_template = { ... },

  visibility_switch = ... ,
  visibility_values = { ... } ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
<code>graphics</code>	Defines the graphics definition to use for this widget. Required. See section

[\[2D_widget_graphics_specification\]](#) for details for the 2D GUI pipeline.

value	Property path to the property to which this widget should be connected. Optional. The property must be a number, string, or boolean property. If <code>read_only</code> is <code>true</code> , the property must be <code>rt_owner</code> . If <code>read_only</code> is <code>false</code> , the property must be <code>document_owner</code> or <code>gui_owner</code> . If <code>value</code> is not specified, then <code>values</code> and <code>value_switch</code> must be specified.
-------	--

value_switch	Property path to a <code>document_owner</code> discrete (stepped) number property. The value of the <code>value_switch</code> property determines (dynamically) which of the properties in the <code>values</code> list (see below) to which this widget should be connected. Optional. If <code>value_switch</code> is specified, then <code>values</code> must be specified too. If <code>value_switch</code> is not specified, then <code>value</code> must be specified (and <code>values</code> omitted). The <code>steps</code> setting in the <code>value_switch</code> property must match the length of the <code>values</code> list.
--------------	--

values	List of property paths to <code>document_owner</code> properties to which this widget should be (dynamically) connected. Optional. The properties must be number, boolean, or string properties. If <code>read_only</code> is <code>true</code> , the properties must be <code>rt_owner</code> . If <code>read_only</code> is <code>false</code> , the properties must be <code>document_owner</code> . If <code>values</code> is specified, then <code>value_switch</code> must be specified too. If <code>values</code> is not specified, then <code>value</code> must be specified (and <code>value_switch</code> omitted). The length of the <code>values</code> list must match the <code>steps</code> setting in the <code>value_switch</code> property. All properties in <code>values</code> must have the same type.
--------	---

<code>text_style</code>	The font style to use for drawing text and/or numbers into the text area. Required. Must be one of the built-in Jukebox fonts, see section [Fonts] .
-------------------------	--

<code>text_color</code>	The color of the font to use for drawing text and/or numbers into the text area. Required. Must be a Lua table with three integer values (<code>{red, green, blue}</code>) in the range <code>[0, 255]</code> .
-------------------------	---

<code>value_templates</code>	List of formatting strings. Optional. May contain one entry, or the same number of entries as the number of entries in <code>values</code> . For each entry, you must use the <code>jbox.ui_text()</code> toolbox function to obtain the corresponding template string. If the substring <code>^0</code> is present in the template string, it is replaced by the (mapped) property value. If the <code>^0</code> substring is not present, the string will be presented as-is. If <code>value_templates</code> is not specified, the host will use the unit formatting specified for the properties in <code>motherboard_def.lua</code> . If <code>value_templates</code> contains one entry, that formatting string will be used for all properties that are bound to the widget.
------------------------------	---

<code>horizontal_justification</code>	Sets the horizontal alignment of the text inside the widget boundaries. Optional. Must be one of the values <code>"left"</code> , <code>"center"</code> , or <code>"right"</code> . If <code>horizontal_justification</code> is omitted, it defaults to <code>"center"</code> .
---------------------------------------	---

<code>read_only</code>	Specifies whether the user may enter data into the widget (<code>false</code>) or not (<code>true</code>). Optional. If <code>read_only</code> is not specified, it defaults to <code>false</code> .
------------------------	--

`show_remote_box`

If `true`, instruct the host to show a GUI indication if this widget is under Remote™ control. Optional. If `show_remote_box` is not specified, it defaults to `true`. The setting of this attribute may be ignored by the host.

`show_automation_
rect`

If `true`, instruct the host to show a GUI indication if this widget is automated. Optional. If `show_automation_rect` is not specified, it defaults to `true`. The setting of this attribute may be ignored by the host.

`tooltip_position`

Specifies the position of the tooltip for this widget (for hosts that are capable of showing tooltips). Optional. Must be one of the strings "bottom_left", "bottom", "bottom_right", "right", "top_right", "top", "top_left", "left" or "no_tooltip" (disables tooltips for this widget). If `tooltip_position` is not specified, the host is free to choose any tooltip position. The setting of this attribute may be ignored by the host.

`tooltip_template`

Formatting string for the tooltip text. Optional. You must use the `jbox.ui_text()` toolbox function to obtain the string. If the substring `^0` is present in the string, it is replaced by the (localized) property value. If the substring `^1` is present in the string, it is replaced by the (localized) property name. The `^0` and `^1` substrings may appear only once in the formatting string. If `tooltip_template` is not specified, the host is free to choose the formatting. The setting of this attribute may be ignored by the host.

visibility_switch

Property path to a discrete (stepped) number property. The value of the `visibility_switch` property determines (dynamically) whether the widget is shown or hidden. Optional. If `visibility_switch` is specified, then `visibility_values` must be specified too. When the value of the `visibility_switch` property matches one of the values in `visibility_values` the widget is shown, otherwise it is hidden. This setting is only valid for unfolded panels.

visibility_values

The list of values for which the widget should be visible if the `visibility_switch` property matches one them. Optional.

custom_display

Custom display that can be updated dynamically. See section [\[display\]](#) for details on how the contents of a custom display is updated. Note that as of Reason 12, the hosting uses vector scaling to upgrade the physical resolution of the display to the current OS and application zoom level.

None

```
jbox.custom_display{
  graphics = {
    main_node = ... ,
  },

  display_width_pixels = ... ,
  display_height_pixels = ... ,

  values = { ... },

  invalidate_function = ... ,
  draw_function = ... ,
```

```

gesture_function = ...

show_remote_box = ... ,
show_automation_rect = ... ,

visibility_switch = ... ,
visibility_values = { ... } ,

background = jbox.image{ path= ... } ,
},

```

Custom display widgets are only allowed on the front panel and folded front panel.

Attributes.

ATTRIBUTE	DESCRIPTION
graphics	Graphics definition that is used to denote the area of the display. Required. Must not be animated. Cannot have <code>hit_boundaries</code> or <code>extra_nodes</code> specifications. Cannot have a <code>boundaries</code> specification. The scenegraph node must have precisely one material subnode of the type <code>customDisplay</code> .
display_width_pixels	Width of the coordinate system used inside the display. Must be a positive integer. Required.

<code>display_height_pixels</code>	Height of the coordinate system used inside the display. Must be a positive integer. Required.
------------------------------------	--

<code>values</code>	List of motherboard properties. Required. When one or several of these properties change the display may have to be repainted. If the display accepts user input, some or all of these properties may be changed by the display (assuming they are in the <code>document_owner</code> or <code>gui_owner</code> scope). The list must contain at least one entry. Allowed property types are number, string, and boolean.
---------------------	---

<code>invalidate_function</code>	String that contains the name of a Lua function in the <code>display.lua</code> file. The function is called by the host when one or several of the properties in the <code>values</code> list have been updated. The function determines which parts of the display (if any) that should be considered dirty (and needs repainting). Optional. If <code>invalidate_function</code> is not specified, the entire display will be considered dirty.
----------------------------------	--

<code>draw_function</code>	String that contains the name of a Lua function in the <code>display.lua</code> file. The function is called by the host when the display needs to be repainted. Required.
----------------------------	--

<code>gesture_function</code>	String that contains the name of a Lua function in the <code>display.lua</code> file. The function is called by the host when the display needs to respond to a user input event. Optional. If <code>gesture_function</code> is not specified, the display will not accept any user input gestures.
-------------------------------	---

`show_remote_box`

If `true`, instruct the host to show a GUI indication if this widget is under Remote™ control. Optional. If `show_remote_box` is not specified, it defaults to `true`. The setting of this attribute may be ignored by the host.

`show_automation_rect`

If `true`, instruct the host to show a GUI indication if this widget is automated. Optional. If `show_automation_rect` is not specified, it defaults to `true`. The setting of this attribute may be ignored by the host.

`visibility_switch`

Property path to a discrete (stepped) number property. The value of the `visibility_switch` property determines (dynamically) whether the widget is shown or hidden. Optional. If `visibility_switch` is specified, then `visibility_values` must be specified too. When the value of the `visibility_switch` property matches one of the values in `visibility_values` the widget is shown, otherwise it is hidden. This setting is only valid for unfolded panels.

`visibility_values`

The list of values for which the widget should be visible if the `visibility_switch` property matches one them. Optional.

`background`

Background image for the display (`jbox.image{path=<resource-name>}`). Optional. If supplied, the display will be cleared to the contents of the image before the `draw_function` is called. Image should be provided in hi-res, as an image file with filename `<resource-name>-HD.png`, with dimensions matching those of the widget graphics attribute. Legacy: Image can (alternatively) still be provided in lo-res, like in earlier SDKs, as an image file with filename `<resource-name>.png`, with dimensions matching the

coordinate system inside the display
(display_width_pixels x display_height_pixels).

static_decoration

A static decoration.

None

```
jbox.static_decoration{
  graphics = {
    main_node = ... ,
  },
  blend_mode = ...,

  visibility_switch = ... ,
  visibility_values = { ... } ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
-----------	-------------

graphics	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline. Cannot be animated. Cannot have <code>hit_boundaries</code> or <code>extra_nodes</code> specifications. Cannot have a <code>boundaries</code> specification.
----------	--

blend_mode	String that acts as a rendering hint for hosts that cannot recreate the full Rack Extension Designer rendering engine for static decoration widgets. Optional. If <code>blend_mode</code> is specified, it must be one of the values "normal" or "luminance". See below for details. If <code>blend_mode</code> is not specified, it defaults to "normal". The <code>blend_mode</code> setting may be ignored by the host.
------------	--

visibility_switch	Property path to a discrete (stepped) number property. The value of the <code>visibility_switch</code> property determines (dynamically) whether the widget is shown or hidden. Optional. If <code>visibility_switch</code> is specified, then <code>visibility_values</code> must be specified too. When the value of the <code>visibility_switch</code> property matches one of the values in <code>visibility_values</code> the widget is shown, otherwise it is hidden. This setting is only valid for unfolded panels.
-------------------	---

visibility_values	The list of values for which the widget should be visible if the <code>visibility_switch</code> property matches one them. Optional.
-------------------	--

Blend modes.

MODE	DESCRIPTION
"normal"	Ordinary alpha-based blending.
"luminance"	Like "normal", except use $\max(A, Y)$ as the alpha value, where A is the original alpha value and Y is the luminance of the RGB channels.

device_name

Visual representation of the name of the device, shown as a tape strip.

You cannot change the appearance of this widget. Use the `Tape_Horizontal.sg` or `Tape_Vertical.sg` scenegraphs in the Jukebox Part Library to specify its location. The Jukebox SDK contains corresponding images for the 2D GUI pipeline.

None

```
jbox.device_name{
  graphics = { ... },
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
<code>graphics</code>	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.

audio_input_socket

An audio input socket. May only be used on the back panel. If an audio socket exists in the motherboard, there must be a corresponding audio socket widget in `hdgui_2D.lua`.

You cannot change the appearance of this widget. Use the `Cable_Attachment_Audio.sg` scenegraph in the Jukebox Part Library to specify its location. The Jukebox SDK contains a corresponding image for the 2D GUI pipeline.

None

```
jbox.audio_input_socket{
    graphics = { ... },

    socket = ... ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
<code>graphics</code>	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
<code>socket</code>	The property path of a socket in the <code>"/audio_inputs"</code> property set object. Required.

audio_output_socket

An audio output socket. May only be used on the back panel. If an audio socket exists in the motherboard, there must be a corresponding audio socket widget in `hdgui_2D.lua`.

You cannot change the appearance of this widget. Use the `Cable_Attachment_Audio.sg` scenegraph in the Jukebox Part Library to specify its location. The Jukebox SDK contains a corresponding image for the 2D GUI pipeline.

None

```
jbox.audio_output_socket{
    graphics = { ... },

    socket = ... ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
graphics	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
socket	The property path of a socket in the <code>"/audio_ouputs"</code> property set object. Required.

cv_input_socket

An cv input socket. May only be used on the back panel. If a cv socket exists in the motherboard, there must be a corresponding cv socket widget in `hdgui_2D.lua`.

You cannot change the appearance of this widget. Use the `CV_Attachment_Audio.sg` scenegraph in the Jukebox Part Library to specify its location. The Jukebox SDK contains a corresponding image for the 2D GUI pipeline.

None

```
jbox.cv_input_socket{
  graphics = { ... },

  socket = ... ,
```

```
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
<code>graphics</code>	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
<code>socket</code>	The property path of a socket in the <code>"/cv_inputs"</code> property set object. Required.
<code>cv_output_socket</code>	

An cv output socket. May only be used on the back panel. If a cv socket exists in the motherboard, there must be a corresponding cv socket widget in `hdgui_2D.lua`.

You cannot change the appearance of this widget. Use the `cv_Attachment_Audio.sg` scenegraph in the Jukebox Part Library to specify its location. The Jukebox SDK contains a corresponding image for the 2D GUI pipeline.

None

```
jbox.cv_output_socket{  
    graphics = { ... },  
  
    socket = ... ,  
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
graphics	Defines the graphics defintion to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
socket	The property path of a socket in the <code>"/cv_outputs"</code> property set object. Required.

patch_browse_group

A group of buttons (with "previous", "next" and "load" symbols) that allows the user to select a patch for the current device.

You cannot change the appearance of this widget. Use the `Patch_Browse_Group.sg` scenegraph in the Jukebox Part Library to specify its location. The Jukebox SDK contains a corresponding image for the 2D GUI pipeline.

None

```
jbox.patch_browse_group{
  graphics = { ... },

  fx_patch = ... ,
  tooltip_position = ... ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
graphics	Defines the graphics defintion to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
fx_patch	If <code>true</code> , instructs the host to use a special tooltip format appropriate for insert effect devices (for hosts that are capable of showing tooltips). Optional. If <code>fx_patch</code> is omitted, it defaults to <code>false</code> . The setting of this attribute may be ignored by the host.

tooltip_position

Specifies the position of the tooltip for this widget (for hosts that are capable of showing tooltips). Optional. Must be one of the strings "bottom_left", "bottom", "bottom_right", "right", "top_right", "top", "top_left", "left" or "no_tooltip" (disables tooltips for this widget). If `tooltip_position` is not specified, the host is free to choose any tooltip position. The setting of this attribute may be ignored by the host.

patch_name

An text area that displays the name of the currently loaded patch, together with a progress indicator during patch or sample loading. May open a patch browser if the user interacts with the widget.

None

```
jbox.patch_name{
  graphics = { ... },

  text_style = ... ,
  fg_color = { ... },
  loader_alt_color = { ... },
  center = ... ,
},
```

The image below illustrates the `loader_alt_color` attribute. The progress indicator is drawn using `fg_color` (black), but it "fades" to the `loader_alt_color` (red).



Attributes.

ATTRIBUTE	DESCRIPTION
graphics	Defines the graphics defintion to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
text_style	The font style to use for drawing text and/or numbers into the text area. Required. Must be one of the built-in Jukebox fonts, see section [Fonts] .
fg_color	The color of the font to use for drawing text and/or numbers into the text area. Required. Must be a Lua table with three integer values (<code>{red, green, blue}</code>) in the range [0, 255].
loader_alt_color	Secondary color of the patch loading progress indicator. Required. Must be a Lua table with three integer values (<code>{red, green, blue}</code>) in the range [0, 255].
center	Sets the horizontal alignment of the text inside the widget boundaries. Optional. If <code>true</code> , the patch name will be centered. If <code>false</code> , the patch name will be left-aligned. If <code>center</code> is omitted, it defaults to <code>false</code> .
sample_browse_group	

A group of buttons (with "previous", "next," "load" and "sample" symbols) which allows the user to select an audio sample file (for the currently active user sample in the device).

You cannot change the appearance of this widget. Use the `Sample_Browse_Group.png` image in the RE2D package to specify its location.

None

```
jbox.sample_browse_group{
  graphics = { ... },
  tooltip_position = ... ,

  visibility_switch = ... ,
  visibility_values = { ... } ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
graphics	Defines the graphics defintion to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.

`tooltip_position`

Specifies the position of the tooltip for this widget (for hosts that are capable of showing tooltips). Optional. Must be one of the strings "bottom_left", "bottom", "bottom_right", "right", "top_right", "top", "top_left", "left" or "no_tooltip" (disables tooltips for this widget). If `tooltip_position` is not specified, the host is free to choose any tooltip position. The setting of this attribute may be ignored by the host.

`visibility_switch`

Property path to a discrete (stepped) number property. The value of the `visibility_switch` property determines (dynamically) whether the widget is shown or hidden. Optional. If `visibility_switch` is specified, then `visibility_values` must be specified too. When the value of the `visibility_switch` property matches one of the values in `visibility_values` the widget is shown, otherwise it is hidden. This setting is only valid for unfolded panels.

`visibility_values`

The list of values for which the widget should be visible if the `visibility_switch` property matches one them. Optional.

sample_drop_zone

(Only available in `hdgui_2D.lua` (RE2D) version 2.0) An area of the panel GUI which accepts samples by drag and drop.

This widget is invisible and only uses the position and size of its graphics. The default sample drop zone that covers the front panel can be disabled - see [\[hd_gui\]](#).

None

```
jbox.sample_drop_zone{
    graphics = { ... },

    user_sample_index = ...,

    visibility_switch = ... ,
    visibility_values = { ... } ,
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
graphics	Defines the graphics defintion to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.
user_sample_index	Specifies which user sample is replaced when user drops a sample onto the widget. Required. Must be an integer in the range [0, ... , user_sample-count - 1] where user_sample-count is the number of user samples in motherboard_def.lua.

`visibility_switch`

Property path to a discrete (stepped) number property. The value of the `visibility_switch` property determines (dynamically) whether the widget is shown or hidden. Optional. If `visibility_switch` is specified, then `visibility_values` must be specified too. When the value of the `visibility_switch` property matches one of the values in `visibility_values` the widget is shown, otherwise it is hidden. This setting is only valid for unfolded panels.

`visibility_values`

The list of values for which the widget should be visible if the `visibility_switch` property matches one them. Optional.

placeholder

The placeholder is an area on the back panel (non-folded only) that must be kept clean from other widgets. It is used as a reserved space where Reason Studios may insert mandatory GUI elements in the future. Specifying a placeholder on the (non-folded) back panel is required.

You cannot change the appearance of this widget. Use the `Placeholder.sg` scenegraph in the Jukebox Part Library to specify its location. The Jukebox SDK contains a corresponding image for the 2D GUI pipeline.

None

```
jbox.placeholder{
  graphics = { ... },
},
```

Attributes.

ATTRIBUTE	DESCRIPTION
<code>graphics</code>	Defines the graphics definition to use for this widget. Required. See section [2D_widget_graphics_specification] for details for the 2D GUI pipeline.

display.lua

The `display.lua` file contains the code that the host executes in order to update the contents of custom displays widgets. If the 45 defines one or several custom displays, the `display.lua` file must be present.

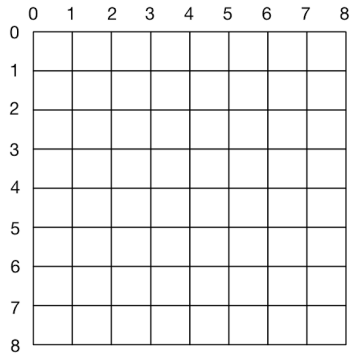
`display.lua` is a standard Lua file that contains a number of function definitions. These functions can be divided into two main categories: callbacks that are used for drawing and callbacks that are used to respond to user input gestures.

The callbacks in `display.lua` are run in isolation from the rest of the 45. They cannot access the Motherboard directly and they cannot have any side effects (except for updating the contents of custom displays). They are also memoryless: it is not possible to create any global variables or other global state in `display.lua`.

Coordinate system

The origo of the coordinate system of a custom display is at the upper left corner. X coordinate values increase towards the right. Y coordinate values increase downwards.

The image below illustrates the coordinate system used for custom displays. Coordinates are between pixels. For example, the coordinate $(0, 0)$ corresponds to the upper left corner of the pixel in the top left corner of the display.



Callbacks for drawing

The callbacks for drawing are either *invalidate functions* that are responsible for invalidating rectangles in a display in response to property changes, or *draw functions* that are responsible for drawing into the display.

Invalidate callback

None

```
void invalidate(  
    table property_values,  
    table last_property_values,  
    table display_info)
```


Called by the host. Bound to the display via the `invalidate_function` entry in `hdgui_2D.lua`.

Based on the current values of the properties that are bound to the display (as declared in `hdgui_2D.lua`), and their values at the last time the `invalidate` function was called, determine if any part of the display has to be redrawn. For each rectangular area that need redrawing, call `jbox_display.invalidate()`. If no call to `jbox_display.invalidate()` is made, the host will not redraw the display.

Parameters

- `property_values` - Index-based array of property values with the same ordering as the `values` entry for the custom display in `hdgui_2D.lua`. For example, the second property in the `values` table would be referred to as `property_values[2]`.
- `last_property_values` - Index-based array of property values at the last time the `invalidate` function was called. Uses the same format as `property_values`.
- `display_info` - Table that contains general information about the display.
 - `width` - The width of the display (same as `display_width_pixels` in `hdgui_2D.lua`).
 - `height` - The height of the display (same as `display_height_pixels` in `hdgui_2D.lua`).

Returns

Nothing.

Draw callback

None

```
void draw(  
    table property_values,  
    table display_info,  
    table dirty_rect)
```

Called by the host. Bound to the display via the `draw_function` entry in `hdgui_2D.lua`.

Based on the current values of the properties that are bound to the display (as declared in `hdgui_2D.lua`), draw into the display.

The host will automatically clip all drawing to the current invalidation rectangle. Use `jbox_display.is_rect_dirty()` (or the `invalidation_rect` input parameter) to cull unnecessary calls to the drawing toolbox.

If the `invalidate` function has invalidated one or more areas in the display, the `draw` function will be called at least once. The host may create unions of invalidation rectangles if needed, so there is no guarantee that the invalidation rectangle that is passed to the `draw` function is equivalent to those registered by the `invalidate` function. The host may call the `draw` function without calling the `invalidate` function first.

Parameters

- `property_values` - Index-based array of property values with the same ordering as the `values` entry for the custom display in `hdgui_2D.lua`. For example, the second property in the `values` table would be referred to as `property_values[2]`.
- `display_info` - Table that contains general information about the display.
 - `width` - The width of the display (same as `display_width_pixels` in `hdgui_2D.lua`).
 - `height` - The height of the display (same as `display_height_pixels` in `hdgui_2D.lua`).
- `dirty_rect` - The rectangle in the image that needs to be redrawn. Table with the following entries:
 - `left` - Integer coordinate.
 - `top` - Integer coordinate.
 - `right` - Integer coordinate.
 - `bottom` - Integer coordinate.

Returns

Nothing.

Gesture response callbacks

The callbacks for responding to user input are either *gesture recognizer functions* that prepares the display for receiving gesture events, or *gesture handler functions* that are responsible for responding to user input events.

Recognize gesture callback

None

```
void recognize_gesture(  
    table property_values,  
    table display_info,  
    table gesture_start_point)
```

Called by the host. Bound to the display via the `gesture_function` entry in `hdgui_2D.lua`.

This callback function is called when a user gesture is initiated inside the custom display.

Parameters

- `property_values` - Index-based array of property values with the same ordering as the `values` entry for the custom display in `hdgui_2D.lua`. For example, the second property in the `values` table would be referred to as `property_values[2]`.
- `display_info` - Table that contains general information about the display.
 - `width` - The width of the display (same as `display_width_pixels` in `hdgui_2D.lua`).
 - `height` - The height of the display (same as `display_height_pixels` in `hdgui_2D.lua`).
- `gesture_start_point` - A table that contains the point at which the gesture was initiated (in the display's coordinate system) and any modifier keys pressed at the time.
 - `x` - X coordinate.
 - `y` - Y coordinate.

- `modifier_key1 (boolean)` - True if key was pressed. Reason usage: Fine tuning values. Corresponding key: Shift(Windows and macOS)
- `modifier_key2 (boolean)` - True if key was pressed. Reason usage: Shortcuts and setting default values. Corresponding key: Ctrl(Windows) and Cmd (macOS)
- `modifier_key3 (boolean)` - True if key was pressed. Reason usage: Open menus, other. Corresponding key: Alt(Windows and macOS)

Returns

Return `nil` to indicate that the display does not respond to this gesture (at this time). Otherwise, return a (non-empty)*gesture definition table* with the following entries:

Popup menus

Custom displays support custom popup-menus. The following example and illustration (indices in red) show a basic outline of the feature. See documentation for `popup_def` below.

None

```
local subItems = {  
  {name="First Sub Item"},  
  {name="Second Sub Item"},  
}
```

```

    {name="Disabled", enabled = false},
    {}, --Will get the title 3
    {name="Checked Sub Item", checked=true},
    {divider=true},
    {name="Last Sub Item"},
}

local myItems = {
    {name= "First Item"},
    {name="Second Item"},
    {name="Disabled", enabled = false},
    {},
    {name="SubMenu", items=subItems},
    {name="Middle Item"},
    {name="Checked Item", checked=true},
    {divider=true},
    {name="Last Item"},
}

gesture_definition = {
    -- popup_def is a new gesture lua table for defining a popup menu.
    -- When popup_def is defined, handlers are optional
    -- on_menu is the callback that is called when a menu item is clicked
    popup_def = {on_menu = "DisplayMenuHandler", items=myItems,
selected=0},
    custom_data = {
    },
}

```

0 First Item

1 Second Item

2 Disabled

3 3

4 SubMenu >

12 Middle Item

✓ Checked Item 13

14

15 Last Item

5 First Sub Item

6 Second Sub Item

7 Disabled

8 3

✓ Checked Sub Item 9

10

11 Last Sub Item

Gesture definition table entries

ENTRY	DESCRIPTION
<code>custom_data</code>	45-specific table suitable for managing the gesture. Must not contain any functions. Optional.
<code>handlers</code>	Table containing <i>handler callbacks</i> that the host should call as the gesture evolves. See below for details. handlers and popup_def are mutually exclusive - only one of them may be set in any gesture definition.

popup_def	Table defining a popup menu that the host should show, instead of running gesture handlers. See below for details. handlers and popup_def are mutually exclusive - only one of them may be set in any gesture definition.
-----------	---

Gesture handlers

ENTRY	DESCRIPTION
on_tap	Lua string that uniquely identifies a function in <code>display.lua</code> . This function is called when a tap/click, or drag gesture is initiated. Optional.
on_update	Lua string that uniquely identifies a function in <code>display.lua</code> . This function is called when a drag gesture is updated (i.e., the user's pointer has moved). Optional.
on_release	Lua string that uniquely identifies a function in <code>display.lua</code> . This function is called when a tap/click or drag gesture ends. Optional.

	Lua string that uniquely identifies a function in <code>display.lua</code> .
<code>on_cancel</code>	This function is called when the user or host cancels a gesture. Optional.

Popup menu definition

ENTRY	DESCRIPTION
<code>on_menu</code>	Lua string that uniquely identifies a function in <code>display.lua</code> . This function is called when a popup menu item is activated/selected. Required.
<code>selected</code>	Zero-based integer index of the initially selected item in the menu when it opens. Must be greater than or equal to zero and less than the number of items. See index in <code>on_menu()</code> below. Optional (no item will be selected if omitted).
<code>items</code>	Table which defines the menu items in the popup menu. Required. See below for details

Popup menu items

ENTRY	DESCRIPTION
<code>name</code>	Name presented for the item in the popup menu in the GUI. Optional. If omitted, the menu will contain the numeric indices of the entries.
<code>checked</code>	Boolean which says if the item is currently checked or not. Optional - default is not checked.
<code>disabled</code>	Boolean set to true to disable the menu item. Optional.
<code>divider</code>	Boolean set to true to make the menu item a divider. Optional. If specified, all other fields of this table must be omitted.
<code>items</code>	Popup sub-menu items to open under this item. Optional. If specified, the item will be a sub-menu. Only allowed at the top level, i.e. only one level of sub-menus is supported.

The `custom_data` table exists while the gesture is active. It is mutable (i.e., it can be modified by the gesture handlers). The `custom_data` table may contain keys that are Lua strings or numbers. The values may be Lua strings, numbers, booleans, or tables.

Gesture handler callbacks

All gesture handler callbacks have the same inputs:

None

```
void on_tap(  
    table property_values,  
    table display_info,  
    table gesture_info,  
    table custom_data)
```

```
void on_update(  
    table property_values,  
    table display_info,  
    table gesture_info,  
    table custom_data)
```

```
void on_release(  
    table property_values,  
    table display_info,  
    table gesture_info,  
    table custom_data)
```

```
void on_cancel(  
    table property_values,  
    table display_info,  
    table gesture_info,  
    table custom_data)
```

Parameters

- `property_values` - Index-based array of property values with the same ordering as the `values` entry for the custom display in `hdgui_2D.lua`. For example, the second property in the `values` table would be referred to as `property_values[2]`.
- `display_info` - Table that contains general information about the display.
 - `width` - The width of the display (same as `display_width_pixels` in `hdgui_2D.lua`). Integer.
 - `height` - The height of the display (same as `display_height_pixels` in `hdgui_2D.lua`). Integer.
- `gesture_info` - Table that contains information about the gesture.
 - `current_point` - A table that contains the current point of interaction, in the display's coordinate system and any modifier keys pressed at the time.
 - `x` - X coordinate.
 - `y` - Y coordinate.
 - `modifier_key1` (boolean) - True if key was pressed. Reason usage: Fine tuning values. Corresponding key: Shift(Windows and macOS)
 - `modifier_key2` (boolean) - True if key was pressed. Reason usage: Shortcuts and setting default values. Corresponding key: Ctrl(Windows) and Cmd (macOS)
 - `modifier_key3` (boolean) - True if key was pressed. Reason usage: Open menus, other. Corresponding key: Alt(Windows and macOS)
 - `start_point` - A table that contains the starting point of the gesture, in the display's coordinate system and any modifier keys pressed at the time.
 - `x` - X coordinate.
 - `y` - Y coordinate.
 - `modifier_key1` (boolean) - True if key was pressed. Reason usage: Fine tuning values. Corresponding key: Shift(Windows and macOS)
 - `modifier_key2` (boolean) - True if key was pressed. Reason usage: Shortcuts and setting default values. Corresponding key: Ctrl(Windows) and Cmd (macOS)
 - `modifier_key3` (boolean) - True if key was pressed. Reason usage: Open menus, other. Corresponding key: Alt(Windows and macOS)
- `custom_data` - The table passed as `custom_data` in the gesture definition table. May be modified by the gesture handler callbacks (this allows the callbacks to communicate with each other while the gesture is ongoing).

Returns

Gesture handlers must always return a table. If no properties should be changed, return an empty table, otherwise the table must be in the following format:

```
None
{
    gesture_ui_name = jbox.ui_text(...),
    property_changes = {
        ...
    }
}
```

where the entries are as follows:

ENTRY	DESCRIPTION
<code>gesture_ui_name</code>	A (localized) string that is used to identify the gesture in the host's user interface. You must use the <code>jbox.ui_text()</code> toolbox function to obtain the string. If all changed properties are in the <code>gui_owner</code> scope, then <code>gesture_ui_name</code> is optional. If one or several of the changed properties are in another owner scope, then <code>gesture_ui_name</code> is required.
<code>property_changes</code>	Table of property changes. The format is the same as the <code>property_values</code> input parameter. Required. If an entry in the table contains the same property value as the corresponding entry in the <code>property_values</code> input parameter, the host may ignore the entry.

Menu handler callbacks

All menu handler callbacks have the same inputs:

None

```
void on_menu(  
    table property_values,  
    table index,  
    table custom_data)
```

Parameters

- `property_values` - Index-based array of property values with the same ordering as the `values` entry for the custom display in `hdgui_2D.lua`. For example, the second property in the `values` table would be referred to as `property_values[2]`.
- `index` - Zero-based index of the menu-item that was selected. The index for a sub-item is equal to the number of menu items above it in the menu, including the parent item and all sub-menu-items for every sub-menu above it in the menu, and all dividers.
- `custom_data` - Any data to pass from `gesture_definition` to menu handler callback.

Patch file format

The Jukebox patch file format is XML-based. This is the schema:

XML

```

<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="JukeboxPatch">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="DeviceNameInEnglish" type="xs:string" />
        <xs:element name="Properties">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="Object">
                <xs:complexType>
                  <xs:sequence>
                    <xs:element name="Value" minOccurs="1" maxOccurs="unbounded">
                      <xs:complexType>
                        <xs:simpleContent>
                          <xs:extension base="xs:string">
                            <xs:attribute name="property" type="xs:string"
use="required" />
                            <xs:attribute name="type" type="xs:string" use="required"
/>
                          </xs:extension>
                        </xs:simpleContent>
                      </xs:complexType>
                    </xs:element>
                  </xs:sequence>
                <xs:attribute name="name" type="xs:string" use="required" />
              </xs:complexType>
            </xs:element>
          </xs:sequence>
          <xs:attribute name="deviceProductID" type="xs:string"
use="required" />
          <xs:attribute name="deviceVersion" type="xs:string" use="required"
/>
        </xs:complexType>
      </xs:element>
    </xs:sequence>
    <xs:attribute name="version" type="xs:float" use="required" />
  </xs:complexType>
</xs:element>

```

</xs:schema>

TAG	DESCRIPTION
JukeboxPatch	Contains a Jukebox patch. Required. Only one <code>JukeboxPatch</code> tag is allowed in the patch file. Must have one <code>version</code> attribute with the value "1.0".
DeviceNameInEnglish	Human-readable name of the device that created the patch. Required. Must match the <code>long_name</code> entry in <code>info.lua</code> . Only one <code>DeviceNameInEnglish</code> is allowed in the patch file.
Properties	List of property set objects. Required. Must have two attributes: <code>deviceProductId</code> , which must match the <code>product_id</code> entry in <code>info.lua</code> , and <code>deviceVersion</code> , which is the first product version (<code>version_number</code> in <code>info.lua</code>) that is able to interpret this patch. Only one <code>Properties</code> is allowed in the patch file.
Object	A property set object. Required. Must have one attribute: <code>name</code> , which has the value "custom_properties". Only one <code>Object</code> is allowed in the patch file.
Value	A property value. Required for each <code>document_owner</code> property with <code>song</code> or <code>patch</code> persistence. Must have two attributes: <code>property</code> , which must match the name of the property, and <code>type</code> , which must match the Jukebox type of the property ("number", "boolean", or "string"). The data between the start and end tag is the property value itself.

Number properties are stored as ASCII strings.

Boolean properties are stored as ASCII strings: `true` or `false`.

String properties are stored as a hex dump of the bytes in the string (this allows for embedded nulls and other unprintables). For example, the string `"foo"` would be stored as (`f` = 66 in hex, `o` = 6F)

XML

```
<Value property="prop3" type="string">666F6F</Value>
```

Note that in Jukebox, all strings are in UTF-8.

Example:

XML

```
<?xml version="1.0"?>
<JukeboxPatch version="1.0">
  <DeviceNameInEnglish>Synth</DeviceNameInEnglish>
  <Properties deviceProductID="com.company.Synth"
deviceVersion="1.0.0d4">
    <Object name="custom_properties">
      <Value property="prop1" type="number">0.5</Value>
      <Value property="prop2" type="boolean">>false</Value>
      <Value property="prop3" type="string">0F1A5B</Value>
    </Object>
  </Properties>
</JukeboxPatch>
```

Patches with a release `deviceVersion` are always considered to be newer than other configurations of the same product ID. For example, a patch where `deviceVersion` is "1.0.0" is considered to be newer than a patch where `deviceVersion` is "+1.0.0f1".

Important! The full path of a patch name must not exceed 200 characters. The patch file name may not contain any of the following characters: forward slash (/), less than (<), more than (>), colon (:), double quote ("), backslash (\), vertical bar (|), question mark (?), and asterisk (*).

Localized text resources

Localized text strings are defined in text resource databases. There is one database for each language supported by the host. A localized text resource database is specified in the following file:

`Resources/<language>+/texts.lua+`

The following languages are supported by all hosts: `English`, `German`, `French`, and `Japanese`.

The `English` database is required.

Each `texts.lua` file has the following format:

```
None
format_version = "1.0"

texts = {
    ...
}
```

The `texts` table contains key/value pairs, e.g.,

None

```
texts = {  
  ["key"] = "value"  
}
```

Lua sandboxing

In order to promote robustness, the host runs all Jukebox Lua scripts in a protected environment. The ideal behavior would be to re-parse the Lua script in an empty Lua environment before the host calls a function in the script. This is unpractical for performance reasons, however, so Jukebox disables some features of the Lua language in order to produce a *sandboxed* environment that can persist between the host's Lua calls.

The sandbox is disabled when a Lua script is parsed/evaluated for the first time. This makes it possible to create and initialize global tables and other variables.

The sandbox enforces the following rules:

- Updates to anything other than function-local variables is forbidden. This also means that it is forbidden to create a variable in the global scope from inside a function (by omitting the `local` keyword). For example:

None

```
my_table = { foo = 10 } -- OK, as long as table is kept constant  
my_table.foo = 20      -- Not allowed - updates a global table!  
  
my_function = function()
```

```
local a = 10  -- OK, local variable
a = 20  -- OK, modifies local variable

b = 20  -- Not allowed - creates a global variable!

my_table.foo = 20  -- Not allowed - updates a global table!
end
```

-

Updates to upvalues (a.k.a. external local variables; see the section called "Closures" in the Lua manual at www.lua.org) is forbidden. For example:

None

```
my_function = function()
  local a = 10  -- OK, local variable
  local my_function_2 = function()
    a = 10  -- Not allowed - updates an upvalue!
  end
end
end
```

-

Replacing the Jukebox Lua toolbox (see section [\[Lua_toolbox\]](#)) with user-defined functions is forbidden. Example:

None

```
-- Not allowed - redefines a Jukebox toolbox function!
jbox_display.draw_rect = function()
  return 10
end
```

Important! The current implementation of the Lua sandbox causes the array size operators (`#` and `getn`) to return zero for arrays in the global scope. Example:

None

```
my_array = {
  [1] = 10,
  [2] = 20,
  [3] = 30
}

local size_1 = #my_array -- Beware - returns zero!
local size_2 = table.getn(my_array) -- Beware - returns zero!

-- Iterating over global tables works as expected:
for index, value in pairs(my_array) do
  ...
end
```

Lua toolbox

This section contains specifications for Lua functions that are made available by the host in various Lua scripts. The **context** subsection in each function specification indicates from where the function can be called.

The specifications use the following data types:

TYPE	DESCRIPTION
------	-------------

<code>string</code>	Lua string
<code>boolean</code>	Lua boolean
<code>number</code>	Lua number
<code>table</code>	Lua table
<code>sample_ref</code>	Opaque Jukebox-specific type that holds a reference to a sample
<code>blob_ref</code>	Opaque Jukebox-specific type that holds a reference to a blob
<code>native_obj_ref</code>	Opaque Jukebox-specific type that holds a reference to a native object
<code>ui_text</code>	Opaque Jukebox-specific type that holds a localized string
<code>any_type</code>	<code>string</code> , <code>number</code> , <code>boolean</code> , <code>sample_ref</code> , <code>blob_ref</code> , or <code>native_obj_ref</code> (not <code>ui_text</code>)

ui_text

`ui_text jbox.ui_text( string key )`

Return an opaque value that represents a localized string.

Parameters

- `key` - The key of the string (in the `texts.lua` files).

Context

- Realtime controller
- `ui_type` (`data_to_gui` and `gui_to_data`)
- Custom displays

load_sample_async

```
sample_ref jbox.load_sample_async( string sample )
```

Load a sample. This function does not block and may return before the sample is fully loaded.

Parameters

- `sample` - Path to the sample to load, e.g., `"/Private/mySample.wav"`

Context

- Realtime controller

See also `jbox.get_sample_info()` (See [\[get_sample_info\]](#))

load_blob_async

```
blob_ref jbox.load_blob_async( string blob )
```

Load a BLOB. This function does not block and may return before the BLOB is fully loaded.

Parameters

- `blob` - Path to the BLOB to load, e.g., `"/Private/myBlob.data"`

Context

- Realtime controller

See also

`jbox.get_blob_info()` (Sec. [\[get_blob_info\]](#))

make_native_object_rw

```
native_obj_ref jbox.make_native_object_rw(  
    string operation,  
    table arguments )
```

Create a new mutable (read/write) native object. Calling `make_native_object_rw` will generate a call to the corresponding `JBox_Export_CreateNativeObject()` C++ function. The function will block until the native object has been created.

Mutable native objects can be written to in `JBox_Export_RenderRealtime()`.

Parameters

- `operation` - A string that can be used by the C++ function to make decisions about what to return
- `arguments` - Lua table with arguments to pass on to the C++ function

Context

- Realtime controller

See also

`jbox.make_native_object_ro()` (Sec. [\[make_native_object_ro\]](#))

make_native_object_ro

```
native_obj_ref jbox.make_native_object_ro(  
    string operation,  
    table arguments )
```

Create a new immutable (read only) native object. Calling `make_native_object_ro` will generate a call to the corresponding `JBox_Export_CreateNativeObject()` C++ function. The function will block until the native object has been created.

Immutable native objects cannot be written to in `JBox_Export_RenderRealtime()`.

Parameters

- `operation` - A string that can be used by the C++ function to make decisions about what to return
- `arguments` - Lua table with arguments to pass on to the C++ function

Context

- Realtime controller

See also

`jbox.make_native_object_rw()` (Sec. [\[make_native_object_rw\]](#))

is_sample

```
boolean jbox.is_sample( any_type value )
```

Return `true` if `iValue` is of type `sample_ref`, `false` otherwise.

Parameters

- `value` - The value to check

Context

- Realtime controller

is_blob

```
boolean jbox.is_blob( any_type value )
```

Return `true` if `iValue` is of type `blob_ref`, `false` otherwise.

Parameters

- `value` - The value to check

Context

- Realtime controller

`is_native_object`

```
boolean jbox.is_native_object( any_type value )
```

Return `true` if `iValue` is of type `native_obj_ref`, `false` otherwise.

Parameters

- `value` - The value to check

Context

- Realtime controller

get_sample_info

```
table jbox.get_sample_info( sample_ref sample )
```

Obtain information about a sample. The function returns a Lua table with the following entries:

- `frame_count (number)` - The total number of frames in the sample
- `resident_count (number)` - The number of loaded frames at the time of the call
- `channels (number)` - The number of channels (1 = mono, 2 = stereo)
- `sample_rate (number)` - The sample rate in Hz
- `state (number)` - 0 if the sample reference is nil or there is an error, 1 if the sample has been partially loaded, 2 if the sample has finished loading

Parameters

- `sample` - The sample for which to obtain information.

Context

- Realtime controller, display

See also

`jbox.load_sample_async()` (Sec [\[load_sample_async\]](#))

get_sample_meta_data

```
table jbox.get_sample_meta_data( sample_ref sample )
```

Obtain information about a sample. The function returns a Lua table with the following entries:

- `frame_count (number)` - The total number of frames in the sample
- `resident_frame_count (number)` - The number of loaded frames at the time of the call
- `channels (number)` - The number of channels (1 = mono, 2 = stereo)
- `sample_rate (number)` - The sample rate in Hz
- `load_status (string)` - The load status of the file. Can be "nil", "partially resident", "resident", "has_errors" or "missing"
- `root_key (number)` - The current value of the root key **sample parameter**
- `tune_cents (number)` - The current value of the tune cents **sample parameter**
- `play_range_start (number)` - The start of current play range **sample parameter** in frames

- `play_range_end (number)` - The end of current play range **sample parameter** in frames
- `loop_range_start (number)` - The start of current loop range **sample parameter** in frames
- `loop_range_end (number)` - The end of current loop range **sample parameter** in frames
- `loop_mode (string)` - The current loop mode **sample parameter**. Can be "loop_off", "loop_forward" or "loop_backward_forward"

Parameters

- `sample` - The sample for which to obtain information.

Context

- Realtime controller, display

get_blob_info

```
table jbox.get_blob_info( blob_ref blob )
```

Obtain information about a BLOB. The function returns a Lua table with the following entries.

- `size (number)` - The total size of the BLOB in bytes
- `resident_size (number)` - The number of loaded bytes at the time of the call
- `state (number)` - 0 if the BLOB reference is nil or there is an error, 1 if the BLOB has been partially loaded, 2 if the BLOB has finished loading

Parameters

- `blob` - The blob for which to obtain information.

Context

- Realtime controller

See also

`jbox.load_blob_async()` (Sec [\[load_blob_async\]](#))

make_empty_sample

```
sample_ref jbox.make_empty_sample()
```

Returns a reference to the empty sample (zero frames).

Context

- Realtime controller

make_empty_blob

```
blob_ref jbox.make_empty_blob()
```

Returns a reference to the empty BLOB (zero bytes).

Context

- Realtime controller

make_empty_native_object

```
native_object_ref jbox.make_empty_native_object()
```

Returns a reference to the empty native object (`NULL` pointer).

Context

- Realtime controller

load_property

```
any_type jbox.load_property( string path )
```

Returns the value of a property in the motherboard. If the property is of type string, boolean, or number, the corresponding Lua value is returned. If the property holds a sample, BLOB, or native object, a reference to the value is returned.

Parameters

- `path` - The motherboard path for the property, e.g., `"/custom_properties/myProperty"`

Context

- Realtime controller

store_property

```
void jbox.store_property( string path, any_type value )
```

Store a value in a property in the motherboard.

Parameters

- `path` - The motherboard path for the property, e.g., `"/custom_properties/myProperty"`
- `value` - The new value for the property

Context

- Realtime controller

format_integer_as_string

```
string jbox.format_integer_as_string( number value )
```

Casts the specified value to an integer and returns an UTF-8 string representation of it. The host is free to choose an appropriate casting rule.

Parameters

- `value` - The value to convert

Context

- `ui_type` (`data_to_gui` and `gui_to_data`)
- Custom displays

format_number_as_string

```
string jbox.format_number_as_string(  
    number value,  
    number decimals )
```

Return an UTF-8 string with a (localized) representation of a floating-point number.

Parameters

- `value` - The value to convert
- `decimals` - The number of decimals to round the value to before converting

Context

- `ui_type` (`data_to_gui` and `gui_to_data`)
- Custom displays

`format_note_length_value_as_string`

```
string jbox.format_note_length_value_as_string(  
    number numerator,  
    number denominator,  
    boolean isTriplet )
```

Given a numerator and a denominator, return a string representing the corresponding note length (e.g., "1/4", "4/16", "1/4T").

Parameters

- `numerator` - The numerator of the note value
- `denominator` - The denominator of the note value
- `isTriplet` - If `true`, then add a "T" to the end of the string (used to denote a triplet)

Context

- `ui_type` (`data_to_gui` and `gui_to_data`)
- Custom displays

expand_template_string

```
ui_text jbox.expand_template_string( ui_text template, ui_text, ... )
```

Utility text formatting function for localized text strings. The function returns a `ui_text` value.

Parameters

- `template` - A localized template string that contains zero or more argument tokens (`^0`, `^1`, `^2`, etc.).

The other parameters are localized strings that will replace the tokens in the template string.

Context

- `ui_type` (`data_to_gui` and `gui_to_data`)
- Custom displays

value_as_string

None

```
string jbox.value_as_string(  
    number property_index,  
    any_type value )
```

Utility text formatting function for running the textualizer function for a property on a value. The function returns a `ui_text` value.

Parameters

- `property_index` - The index of a property indirection inside the display scripts.
- `value` - A value of the same type as the property at `property_index`

Context

- Custom displays

trace

None

```
void jbox.trace( string string )
```

Write a string to the debugging log.

Parameters

- `string` - The string to write

Context

- Realtime controller
- `ui_type` (`data_to_gui` and `gui_to_data`)
- Custom displays

invalidate

None

```
void jbox_display.invalidate( table rect )
```

Invalidate a rectangle in a custom display.

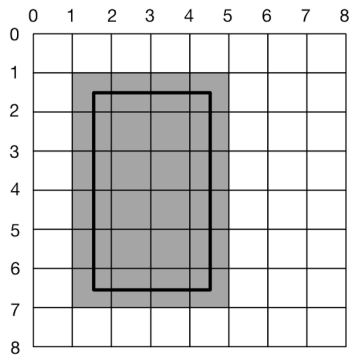
Parameters

- `rect` - A table with the following entries:
 - `left` - Floating-point coordinate. Required.
 - `top` - Floating-point coordinate. Required.
 - `right` - Floating-point coordinate. Required.
 - `bottom` - Floating-point coordinate. Required.

The invalidated pixels will be those that are inside the mathematical rectangle that is defined by rounding the `left` and `top` coordinates down to the nearest integer (floor) and rounding the `right` and `bottom` coordinates up to the nearest integer (ceil). Note that pixel coordinates are between the pixels. The image below shows the area invalidated by the following call:

None

```
jbox_display.invalidate({left=1.5, top=1.5, right=4.5, bottom=6.5})
```



Context

- Custom displays (invalidate function)

is_rect_dirty

None

```
void jbox_display.is_rect_dirty( table rect )
```

Return `true` if the specified rectangle intersects the current dirty rectangle.

Parameters

- `rect` - A table with the following entries:
 - `left` - Floating-point coordinate. Required.
 - `top` - Floating-point coordinate. Required.
 - `right` - Floating-point coordinate. Required.
 - `bottom` - Floating-point coordinate. Required.

The rectangle coordinates are interpreted in the same way as for `jbox_display.invalidate()`.

Context

- Custom displays (draw function)

draw_rect

None

```
void jbox_display.draw_rect( table rect, table color )
```

Draw an anti-aliased, filled rectangle in a custom display.

Parameters

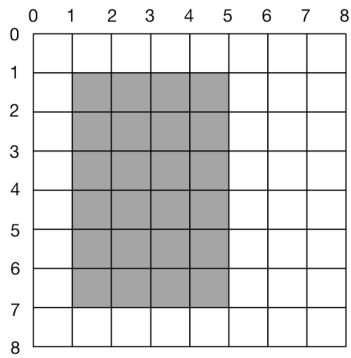
- `rect` - The rectangle to fill. Table with the following format:
 - `left` - Floating point coordinate. Required.
 - `top` - Floating point coordinate. Required.
 - `right` - Floating point coordinate. Required.
 - `bottom` - Floating point coordinate. Required.
- `color` - The sRGB color to use for drawing. Table with the following format:
 - `r` - Red color component. Integer in the range [0, 255]. Required.
 - `g` - Green color component. Integer in the range [0, 255]. Required.
 - `b` - Blue color component. Integer in the range [0, 255]. Required.
 - `a` - Alpha color component. Integer in the range [0, 255]. Optional. If omitted, the alpha component will be set to 255.

The filled pixels will be those that are covered by the mathematical rectangle that is defined by the `rect` parameter. Conceptually, the pixel color will be set according to the area covered by the rectangle. In practice, the exact color may vary depending on the host and/or the platform the host is running on. Pixel coordinates are between the pixels.

The image below illustrates the result of the following call:

None

```
jbox_display.draw_rect(  
    {left=1.0, top=1.0, right=5.0, bottom=7.0},  
    {r=128, g=128, b=128})
```



Context

- Custom displays (draw function)

draw_line

None

```
void jbox_display.draw_line(  
    table start,  
    table end,  
    table color,  
    number thickness,  
    string cap_style )
```

Draw an anti-aliased line in a custom display.

Parameters

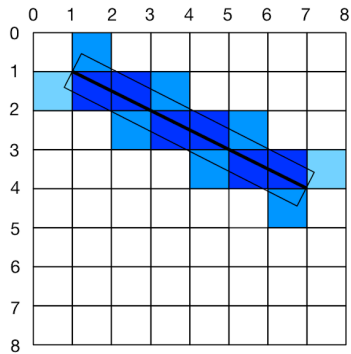
- `start` - The start point of the line. Table with the following format:
 - `x` - Floating point coordinate. Required.
 - `y` - Floating point coordinate. Required.
- `end` - The end point of the line. Table with the following format:
 - `x` - Floating point coordinate. Required.
 - `y` - Floating point coordinate. Required.
- `color` - The sRGB color to use for drawing. Table with the following format:
 - `r` - Red color component. Integer in the range [0, 255]. Required.
 - `g` - Green color component. Integer in the range [0, 255]. Required.
 - `b` - Blue color component. Integer in the range [0, 255]. Required.
 - `a` - Alpha color component. Integer in the range [0, 255]. Optional. If omitted, the alpha component will be set to 255.
- `thickness` - The thickness of the line (in pixels). Optional. Must be ≥ 1.0 . If `thickness` is specified, then `cap_style` must be specified too.
- `cap_style` - Cap style. Optional. Must be one of the following strings: "butt", "round", or "square". If `cap_style` is specified, then `thickness` must be specified too.

Conceptually, pixels will be filled according to the following principle: Create a n -pixel wide mathematical rectangle (where n is the line thickness) around the mathematical line defined by `start` and `end` parameters, then set the pixel color according to the area covered by the rectangle. In practice, the exact color may vary depending on the host and/or the platform the host is running on. Pixel coordinates are between the pixels.

The image below illustrates the result of the following call:

None

```
jbox_display.draw_line({x=1.0, y=1.0}, {x=7.0, y=4.0}, {r=0, g=0, b=255})
```



Context

- Custom displays (draw function)

draw_lines

None

```
void jbox_display.draw_lines(  
    table points,  
    table color,  
    string shape_closed,  
    number thickness,  
    string cap_style,  
    string join_style )
```

Connect a sequence of points with anti-aliased line segments in a custom display.

Parameters

- `points` - The points to connect. Table with two or more entries, where each entry is a `{x, y}` coordinate table:
 - `x` - Floating point coordinate.
 - `y` - Floating point coordinate.
- `color` - The sRGB color to use for drawing. Table with the following format:
 - `r` - Red color component. Integer in the range [0, 255]. Required.
 - `g` - Green color component. Integer in the range [0, 255]. Required.
 - `b` - Blue color component. Integer in the range [0, 255]. Required.
 - `a` - Alpha color component. Integer in the range [0, 255]. Optional. If omitted, the alpha component will be set to 255.
- `shape_closed` - Specifies whether the shape is closed or not. Required. Must be "open" or "closed". If "closed", then the last point in the `points` list will be automatically connected to the first point in the `points` list.
- `thickness` - The thickness of the line (in pixels). Optional. Must be ≥ 1.0 . If `thickness` is specified, then `cap_style` and `join_style` must be specified too.
- `cap_style` - Cap style. Optional. Must be one of the following strings: "butt", "round", or "square". If `cap_style` is specified, then `thickness` and `join_style` must be specified too.

- `join_style` - Join style. Optional. Must be one of the following strings: "bevel", "round", or "miter". If `join_style` is specified, then `thickness` and `cap_style` must be specified too.

Conceptually, pixels will be filled according to the following principle: Create a n-pixel wide mathematical rectangle (where n is the line thickness) around each mathematical line segment, then set the pixel color according to the area covered by the rectangles. In practice, the exact color may vary depending on the host and/or the platform the host is running on. Pixel coordinates are between the pixels.

The following call draws a rectangle:

None

```
local points = {
  {x=0.0, y=0.0},
  {x=5.0, y=0.0},
  {x=5.0, y=5.0},
  {x=0.0, y=5.0},
}
jbox_display.draw_lines(points, {r=0, g=0, b=255}, "closed")
```

Context

- Custom displays (draw function)

draw_polygon

None

```
void jbox_display.draw_polygon( table vertices, table color )
```

Draw a filled polygon in a custom display. The polygon is assumed to be convex.

Parameters

- `vertices` - The polygon vertices. Table with three or more entries, where each entry is a `{x, y}` coordinate table:
 - `x` - Floating point coordinate.
 - `y` - Floating point coordinate.
- `color` - The sRGB color to use for drawing. Table with the following format:
 - `r` - Red color component. Integer in the range [0, 255]. Required.
 - `g` - Green color component. Integer in the range [0, 255]. Required.
 - `b` - Blue color component. Integer in the range [0, 255]. Required.
 - `a` - Alpha color component. Integer in the range [0, 255]. Optional. If omitted, the alpha component will be set to 255.

Conceptually, the filled pixels will be those that are covered by the mathematical polygon that is defined by the `vertices` table, and the pixel color will be set according to the area covered by the polygon. In practice, the exact color may vary depending on the host and/or the platform the host is running on. Pixel coordinates are between the pixels.

The following call draws a filled rectangle:

None

```
local vertices = {  
    {x=0.0, y=0.0},  
    {x=5.0, y=0.0},  
    {x=5.0, y=5.0},  
    {x=0.0, y=5.0},  
}  
jbox_display.draw_polygon(vertices, {r=0, g=0, b=255})
```

Context

- Custom displays (draw function)

draw_text

None

```
void jbox_display.draw_text(  
    table rect,  
    string alignment,  
    string text,  
    string text_style,  
    table color )  
  
void jbox_display.draw_text(  
    table rect,  
    string alignment,  
    string text,  
    string text_style,  
    table color )
```

```
table rect,  
string alignment,  
ui_text text,  
string text_style,  
table color )
```

Draw text into a custom display.

Parameters

- `rect` - Rectangle for positioning the text. The text will be centered vertically in the rectangle. The horizontal positioning is determined by the `alignment` parameter. `rect` is a table with the following format:
 - `left` - Floating point coordinate. Required.
 - `top` - Floating point coordinate. Required.
 - `right` - Floating point coordinate. Required.
 - `bottom` - Floating point coordinate. Required.
- `alignment` - Horizontal alignment. Lua string. Must be one of the following:
 - `"left"` - Draw the text aligned to the left edge of the rectangle.
 - `"right"` - Draw the text aligned to the right edge of the rectangle.
 - `"center"` - Draw the text centered in the rectangle.
- `text` - The text to draw. Lua string or `ui_text`.
- `text_style` - The font to use. Lua string. Must be one of the built-in Jukebox fonts, see section [\[Fonts\]](#).
- `color` - The sRGB color to use for drawing. Table with the following format:
 - `r` - Red color component. Integer in the range [0, 255]. Required.
 - `g` - Green color component. Integer in the range [0, 255]. Required.
 - `b` - Blue color component. Integer in the range [0, 255]. Required.
 - `a` - Alpha color component. Integer in the range [0, 255]. Optional. If omitted, the alpha component will be set to 255.

The pixels that are set by this function and their exact color may vary depending on the host and/or the platform the host is running on. Pixel coordinates are between the pixels.

Context

- Custom displays (draw function)

Texture files

All texture files must be stored as uncompressed TARGA or PNG, 24-bit RGB (8 bits per color component) or 32-bit RGBA (8 bits per color component). If a RGBA texture is provided for a RGB material setting, the alpha channel will be ignored.

For normal maps and tangent maps, the conversion from RGB (in the range [0.0, 1.0]) to XYZ (in the range [-1.0, 1.0]) is carried out as follows:

$$X = (2.0 * R) - 1.0$$

$$Y = (2.0 * G) - 1.0$$

$$Z = (2.0 * B) - 1.0$$

[[2D_graphics_definitions]] == 2D GUI graphics definitions (device_2D.lua)

The `device_2D.lua` file contains 2D graphics definitions. It has the following format for all types except Players:

None

```
format_version = "2.0"
```

```
front = {
```

```
    ...
```

```
}
```

```
back = {
```

```
    ...
```

```
}
```

```
folded_front = {
```

```
    ...
```

```
}
```

```
folded_back = {
```

```
    ...
```

```
}
```

Players have the following format:

None

```
format_version = "2.0"
```

```
panel_type = "note_player"
```

```
front = {
```

```
    ...
```

```
},
```

```
back = {
```

```
    ...
```

```
},
```

ENTRY

DESCRIPTION

<code>format_version</code>	File format version of the <code>device_2D.lua</code> file. Must be "2.0". Required.
-----------------------------	--

<code>panel_type</code>	Indicates if the device is a Player. Optional.
-------------------------	--

<code>front</code>	Graphics definitions for the front panel. Required.
--------------------	---

<code>back</code>	Graphics definitions for the back panel. Required.
-------------------	--

<code>folded_front</code>	Graphics definitions for the folded front panel. Required if not Player.
---------------------------	--

<code>folded_back</code>	Graphics definitions for the folded back panel. Required if not Player.
--------------------------	---

A graphics definition has the following format:

None

```
... = {  
  offset = { ..., ... },  
  { path = "...", frames = ... },  
}
```


COMPONENT	DESCRIPTION
<code>offset</code>	Position of the upper left corner of the widget. Required. Measured in pixels (along the X and Y axis, respectively) from the parent graphics definition's offset.
<code>path</code>	Name of a static image or a film strip image, without any file extension. Optional.
<code>frames</code>	For film strip resources, the number of animation frames. Optional.

Graphics definitions may be nested, e.g.,

None

```
{
  offset = { 200, 100 },
  {
    offset = { 0, -50 },
    Label = { path = "Label" }
    {
      offset = { 0, 50 },
      Knob1 = { path = "Knob", frames = 64 }
    }
    {
      offset = { 100, 50 },
      Knob2 = { path = "Knob", frames = 64 }
    }
  }
}
```

Runtime floating point contract

The floating point operating environment in Jukebox is fixed and managed up by the host. The following rules apply:

- Rounding mode is "round to nearest", where values with a midway fraction are rounded towards the nearest even integer.
- Denormals are treated as zero and/or flushed to zero. For platforms where this is not possible in hardware, the compiler and host will automatically insert normalizations in the LLVM bitcode at strategic locations.
- No exceptions are generated for any floating point operations. This means that you must use the `isnan()` and/or `isinf()` standard library functions to verify the outcome of computations. Writing invalid floating point values to any property or output buffer will blow the fuse.
- The compiler is allowed to perform optimizations similar to `fp:fast` (Visual Studio) and `-ffast-math` (gcc). Expect different results of near-precision operations in different builds and on different platforms.
- The host's floating point representation is undefined. You cannot assume that it is IEEE 754 or any other specific representation.

Alignment of memory allocations

Memory that you allocate with `malloc()` and `new` will be aligned to 16 bytes. However, to get optimal performance for `JBox_FFTRealForward()` and `JBox_FFTRealInverse()` you should make sure the pointer you provide to these functions is aligned according to `JBox_GetOptimalFFTAlignment()`.

Vector types (SIMD)

Jukebox SDK 2.1 and onwards have limited support for vector types.

Important!

To remain cross-platform and future-proof, Jukebox officially supports only a subset of LLVM's vector instruction feature set. This means that if you use a vector type feature that is not listed below, your Rack Extension may not run in future Jukebox hosts (including Reason), even if it compiles and works in the current version of Reason.

Note that there is no guarantee that using vector types will produce faster code; the result depends on the native platform on which the Rack Extension will run. In general, floating point vector computations are more likely to be accelerated than integer vector computations.

Basic rules

All vector types must be declared as precisely 128 bits (16 bytes) long.

Vectors may only be formed from the following base types:

- 8-bit integer, signed or unsigned (16 vector elements)
- 16-bit integer, signed or unsigned (8 vector elements)
- 32-bit integer, signed or unsigned (4 vector elements)
- 32-bit float (4 vector elements)
- 64-bit float (2 vector elements)

64-bit integer and floating point elements are not supported.

Declaration and initialization of vector types

Vector types are declared using the `__vector_size__` keyword, like this:

None

```
typedef TJBox_Int8 int16x8 __attribute__((__vector_size__(16)));
typedef TJBox_UInt8 uint16x8 __attribute__((__vector_size__(16)));

typedef TJBox_Int16 int8x16 __attribute__((__vector_size__(16)));
typedef TJBox_UInt16 uint8x16 __attribute__((__vector_size__(16)));

typedef TJBox_Int32 int4x32 __attribute__((__vector_size__(16)));
typedef TJBox_UInt32 uint4x32 __attribute__((__vector_size__(16)));
typedef TJBox_Float32 float4x32 __attribute__((__vector_size__(16)));

typedef TJBox_Float64 float2x64 __attribute__((__vector_size__(16)));
```

The argument to `__vector_size__` is the number of bytes in the vector, which must be equal to 16 (128 bits).

Vector types are initialized using curly braces, like this:

None

```
int16x8 a = (int16x8){0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15};
int8x16 b = (int8x16){0, 1, 2, 3, 4, 5, 6, 7};
int4x32 c = (int4x32){0, 1, 2, 3};
float4x32 d = (float4x32){0.0f, 0.1f, 0.2f, 0.3f};
float2x64 d = (float2x64){0.0f, 0.1f};
```

Individual elements of a vector type is accessed using the array index operator, e.g.,

None

```
float4x32 a = {0.0f, 0.1f, 0.2f, 0.3f};  
float b = a[0];      // == 0.0f  
float c = a[1];      // == 0.1f
```

Note that taking the address of an individual element in a vector type is not allowed, e.g.,

None

```
float4x32 a = {0.0f, 0.1f, 0.2f, 0.3f};  
float* ptr = &(a[0]);    // ERROR
```

Supported operations

The following C/C++ operations can be applied to vector types:

- Unary + and -
- +, -, *, /, %
- &, |, ^, ~
- >>, <<
- !, &&, ||
- ==, !=, >, <, >=, <=
- =

Jukebox does not support explicit use of overloaded LLVM intrinsics for standard library functions (e.g., `@llvm.sin.*`, `@llvm.sqrt.*`) on vector types.

Vector type conversions

C-style casts can be used to convert from one vector type to another without modifying the underlying bits, e.g.,

None

```
float4x32 a = (float4x32){0.0f, 1.1f, 2.2f, 3.3f};

int4x32 b = (int4x32)a;
int4x32 c = reinterpret_cast<int4x32>(a);
int4x32 d = static_cast<int4x32>(a);
```

For element-by-element conversion, use the `__builtin_convertvector` keyword, e.g.:

None

```
float4x32 a = (float4x32){0.0f, 1.1f, 2.2f, 3.3f};
int4x32 b = __builtin_convertvector(a, int4x32);
// b will contain {0, 1, 2, 3}
```

The first argument to `__builtin_convertvector` is a vector, and the second argument is a vector type with the same number of elements as the first argument. The result of `__builtin_convertvector` is a vector with

the same element type as the second argument, with a value defined in terms of the action of a C-style cast applied to each element of the first argument.

Note that it is not possible to convert directly from C/C++-style arrays and vector types, e.g.,

None

```
float4x32 values = (float4x32){0.1f, 0.2f, 0.3f, 0.4f};  
float* a = (float*)values;          // ERROR
```

Vector element shuffling

To change the order of the elements in a vector, use the `__builtin_shufflevector` keyword, e.g.:

None

```
float4x32 a = (float4x32){0.1f, 0.2f, 0.3f, 0.4f};  
float4x32 b = (float4x32){0.5f, 0.6f, 0.7f, 0.8f};  
float4x32 c;  
  
// Identity operation - return 4-element vector a.  
c = __builtin_shufflevector(a, a, 0, 1, 2, 3)  
  
// "Splat" element 0 of a into a 4-element result.  
c = __builtin_shufflevector(a, a, 0, 0, 0, 0)  
  
// Reverse 4-element vector a.  
c = __builtin_shufflevector(a, a, 3, 2, 1, 0)
```

```
// Concatenate every other element of 4-element vectors a and b.  
c = __builtin_shufflevector(a, b, 0, 2, 4, 6)  
  
// Shuffle a with some elements being undefined  
c = __builtin_shufflevector(a, a, 3, -1, 1, -1)
```

The first two arguments to `__builtin_shufflevector` are vectors that have the same element type. The remaining arguments are a list of integers that specify the element indices of the first two vectors that should be extracted and returned in a new vector. These element indices are numbered sequentially starting with the first vector, continuing into the second vector. Thus, if the first vector argument is a 4-element vector, index 5 would refer to the second element of the second vector argument. An index of -1 can be used to indicate that the corresponding element in the returned vector is a don't care and can be optimized by the backend. The result of `__builtin_shufflevector` is a vector with the same element type as the argument vectors, but that has an element count equal to the number of indices specified.

Patch Metadata

For 45s that come with patches and/or publicly available sample data, it is strongly recommended to include additional metadata such as categories, tags and author information about those items. This will make them a lot easier for users to find in hosts that support it.

Content metadata resides in a file called `<Product>.rsmeta`, where *<Product>* is the last part of the product ID. For example, the product ID `se.propellerheads.SimplePlayer` would have the corresponding metadata file name `SimplePlayer.rsmeta`.

The metadata file is added to the root directory of the 45 project, the same directory where you also find all the core lua scripts. Although the file comes in a human-readable JSON format, it is strongly recommended that you let Reason or Recon generate the file for you.

Perform the following steps in Reason or Recon 13.1 or later:

1. Build your with the content you want to include in its Resources/Public folder
2. In Reason/Recon's browser, find your device's home folder under Reason Library/Rack Extensions/*Name of the device*
3. Double-click on the folder to make it the current location in the browser
4. Right click on any item or subfolder in the view and choose "Edit Info..." to enter the metadata edit view
5. Select the different items and assign the desired metadata to them. Be sure to avoid using User Tags since they are local to your own Reason installation only
6. When you're done applying metadata, click "Save" to commit your changes
7. Click on the "Up" button in the browser to go up one level
8. Right-click on the device's folder and choose "Export Metadata..."
9. Keep the suggested file name, which should already correspond to the device's product ID
10. Save the file in the root folder of your Jukebox device project

The next time you build a Universal 45 or Local 45, the metadata file will be included.

Modules

Jukebox 45 Exports

This module contains functions that all 45s must implement.

All processing made by a 45 is done in these functions, either in a real time synchronous or idle time asynchronous manner.

Functions

`JBox_Export_RenderRealtime` (section
[\[group-jukebox-45-exports_1gaf8664c9a1dd77d9842c8f6f230e8c5bc\]](#))

Performs realtime audio processing for a 45.

`JBox_Export_CreateNativeObject` (section
[\[group-jukebox-45-exports_1gaa3e9472a1a795f4d8a53b0fa00fdd9a2\]](#))

Generic constructor for creating C++ objects.

Functions for accessing the MOM

These functions are used to access the MOM. They can only be called from `JBox_Export_RenderRealtime`.

Inspecting or updating the motherboard object model is only allowed from the context of `JBox_Export_RenderRealtime`. (`JBox_Export_CreateNativeObject` is called asynchronously and can only perform pure functional tasks based on its inputs.) Calling these functions from the wrong context will disable the 45 instance.

Functions

JBox_GetMotherboardObjectRef (section [\[group-jukebox-mom-functions_1gadab322360912370647391199c0b805df\]](#))

Resolves a motherboard property set object path.

JBox_MakePropertyRef (section [\[group-jukebox-mom-functions_1ga02695111d5cd6df224827a26a5bef214\]](#))

Obtain a property reference.

JBox_GetPropertyTag (section [\[group-jukebox-mom-functions_1gae8b6a9bf4e587fce6ac116b832eb54e4\]](#))

Retrieves the user-defined (or system-defined) tag for a property.

JBox_FindPropertyByTag (section [\[group-jukebox-mom-functions_1gacbb05d420f8cca8cf78a6ff69bc776\]](#))

Retrieves a property via its numerical tag.

```
JBox_LoadMOMProperty (section  
[group-jukebox-mom-functions_1ga406214c653b840af2bde2a4621990f8c])
```

Retrieve the value of a property.

```
JBox_StoreMOMProperty (section  
[group-jukebox-mom-functions_1ga4b86ffbe36c0255661b8af6e28d8756e])
```

Update the value of a property.

```
JBox_LoadMOMPropertyByTag (section  
[group-jukebox-mom-functions_1ga990514545d4a545d640ace42606695cc])
```

Retrieve the value of a property.

```
JBox_StoreMOMPropertyByTag (section  
[group-jukebox-mom-functions_1ga0a6f6121b73c48edf015cf3515051694])
```

Update the value of a property.

```
JBox_LoadMOMPropertyAsNumber (section  
[group-jukebox-mom-functions_1ga9a167cadffabc9be5cec4d1c52feb3cf])
```

Retrieve a number from a specified property.

```
JBox_StoreMOMPropertyAsNumber (section  
[group-jukebox-mom-functions_1ga2d15ea31438ce39b7c6d913e529c98a6])
```

Update the value of a property, assuming it is a number.

Classes / structs

TJBox_BLOBInfo

Information about BLOB dynamic values.

```
#include <JukeboxTypes.h>

struct TJBox_BLOBInfo {
    TJBox_SizeT fSize;
    TJBox_SizeT fResidentSize;
};
```

Data Fields:

`TJBox_SizeT fSize`

The total size of the BLOB in bytes.

`TJBox_SizeT fResidentSize`

The number of bytes of the BLOB that are currently loaded in memory.

TJBox_DSPBufferInfo

Information about DSP buffer dynamic values.

```
#include <JukeboxTypes.h>

struct TJBox_DSPBufferInfo {
    TJBox_Int64 fSampleCount;
};
```

Data Fields:

TJBox_Int64 **fSampleCount**

The total number of samples in the DSP buffer.

TJBox_NoteEvent

Represents a note event.

```
#include <JukeboxTypes.h>

struct TJBox_NoteEvent {
    TJBox_UInt8 fNoteNumber;
```

```
TJBox_UInt8 fVelocity;  
TJBox_UInt16 fAtFrameIndex;  
};
```

Input note events that are received as property diffs can be converted to this type. This is also the data type that are used when outputting note events to the host. A note event contains information about when the event happened (frame index), its note number, and its velocity. Events with a zero velocity turns off a note. Events with a velocity greater than zero turns on a note.

Data Fields:

```
TJBox_UInt8 fNoteNumber
```

The note number of the event, valid values are between 0 and 127.

```
TJBox_UInt8 fVelocity
```

The velocity of the event, valid values are between 0 and 127. 0 velocity means that the note is turned off.

```
TJBox_UInt16 fAtFrameIndex
```


The frame in the batch where the event took place. This value must be between 0 and 63.

TJBox_PropertyDiff

Represents a property change.

```
#include <JukeboxTypes.h>

struct TJBox_PropertyDiff {
    TJBox_Value fPreviousValue;
    TJBox_Value fCurrentValue;
    TJBox_PropertyRef fPropertyRef;
    TJBox_Tag fPropertyTag;
    TJBox_UInt16 fAtIndex;
};
```

Contains information about when the property was changed and its previous and current value.

Data Fields:

TJBox_Value **fPreviousValue**

The value of the property before the change.

TJBox_Value **fCurrentValue**

The current value of the property.

TJBox_PropertyRef **fPropertyRef**

Property reference.

TJBox_Tag **fPropertyTag**

The property tag if one has been assigned, kJBox_InvalidPropertyTag otherwise.

TJBox_UInt16 **fAtFrameIndex**

The frame in the batch where the change took place. This value will normally be between 0 and 63 but can, because of a bug in Reason 6.5, be higher, in which case it can be safely treated as 63.

TJBox_PropertyRef

Property reference.

```
#include <JukeboxTypes.h>

struct TJBox_PropertyRef {
    TJBox_ObjectRef fObject;
    TJBox_PropertyKey fKey;
};
```

Data Fields:

TJBox_ObjectRef **fObject**

Property set object reference to the owner of this property.

TJBox_PropertyKey **fKey**

Name of the property in the object as a zero-terminated ASCII string.

TJBox_SampleInfo

Information about sample dynamic values.

```
#include <JukeboxTypes.h>

struct TJBox_SampleInfo {
    TJBox_Int64 fFrameCount;
    TJBox_Int64 fResidentFrameCount;
    TJBox_UInt32 fChannels;
    TJBox_UInt32 fSampleRate;
};
```

Data Fields:

TJBox_Int64 **fFrameCount**

The total number of frames in the sample.

TJBox_Int64 **fResidentFrameCount**

The number of frames of the sample that are currently loaded in memory.

TJBox_UInt32 **fChannels**

The number of channels in the sample, e.g., 2 for stereo.

TJBox_UInt32 **fSampleRate**

The sample rate of the sample in Hz.

TJBox_SampleMetaData

Returned by JBox_GetSampleMetaData.

```
#include <JukeboxTypes.h>
```

```
struct TJBox_SampleMetaData {  
    TJBox_SampleSpec fSpec;  
    TJBox_SampleParameters fParameters;  
    TJBox_AudioFramePos fResidentFrameCount;  
    TJBox_UInt8 fStatus;  
};
```

Data Fields:

```
TJBox_SampleSpec fSpec
```

```
TJBox_SampleParameters fParameters
```

```
TJBox_AudioFramePos fResidentFrameCount
```

```
TJBox_UInt8 fStatus
```

The sample's load status. May be: 0 - Empty sample, has read errors, missing sample 1 - Loading in progress 2 - Resident Only call Jbox_GetSampleData for resident samples.

TJBox_SampleParameters

The samples parameters in Reason. These may or may not be the same as those of the user_sample's parameters in the motherboard.

```
#include <JukeboxTypes.h>
```

```
struct TJBox_SampleParameters {  
    TJBox_UInt8 fRootNote;  
    TJBox_Int8 fTuneCents;  
    TJBox_AudioFramePos fPlayRangeStart;  
    TJBox_AudioFramePos fPlayRangeEnd;  
    TJBox_AudioFramePos fLoopRangeStart;  
    TJBox_AudioFramePos fLoopRangeEnd;  
    TJBox_UInt8 fLoopMode;
```

```
    TJBox_UInt8 fVolumeLevel;  
};
```

Data Fields:

TJBox_UInt8 **fRootNote**

TJBox_Int8 **fTuneCents**

TJBox_AudioFramePos **fPlayRangeStart**

TJBox_AudioFramePos **fPlayRangeEnd**

TJBox_AudioFramePos **fLoopRangeStart**

TJBox_AudioFramePos **fLoopRangeEnd**

TJBox_UInt8 **fLoopMode**

```
TJBox_UInt8 fVolumeLevel
```

TJBox_SampleSpec

```
#include <JukeboxTypes.h>
```

```
struct TJBox_SampleSpec {  
    TJBox_AudioFramePos fFrameCount;  
    TJBox_UInt32 fChannels;  
    TJBox_UInt32 fSampleRate;  
};
```

Data Fields:

```
TJBox_AudioFramePos fFrameCount
```

The total number of frames in the sample.

```
TJBox_UInt32 fChannels
```

The number of channels in the sample, e.g., 2 for stereo.


```
TJBox_UInt32 fSampleRate
```

The sample rate of the sample in Hz.

TJBox_Value

Opaque data type that holds a dynamic value.

```
#include <JukeboxTypes.h>

struct TJBox_Value {
    TJBox_UInt8 fSecret;
};
```

Dynamic values cannot be stored in C++ objects.

Data Fields:

```
TJBox_UInt8 fSecret
```

Enums

TJBox_ValueType

```
#include <JukeboxTypes.h>

enum TJBox_ValueType {
    kJBox_Nil,
    kJBox_Number,
    kJBox_String,
    kJBox_Boolean,
    kJBox_Sample,
    kJBox_BLOB,
    kJBox_DSPBuffer,
    kJBox_NativeObject,
    kJBox_Incompatible
};
```

TJBox_OnOffBypassStates

```
#include <JukeboxTypes.h>

enum TJBox_OnOffBypassStates {
    kJBox_EnabledOff,
    kJBox_EnabledOn,
    kJBox_EnabledBypass
};
```

TJBox_CustomPropertyTag

```
#include <JukeboxTypes.h>

enum TJBox_CustomPropertyTag {
    kJBox_CustomPropertiesOnOffBypass
};
```

TJBox_TransportTag

```
#include <JukeboxTypes.h>
```

```
enum TJBox_TransportTag {  
    kJBox_TransportPlaying,  
    kJBox_TransportPlayPos,  
    kJBox_TransportTempo,  
    kJBox_TransportFilteredTempo,  
    kJBox_TransportTempoAutomation,  
    kJBox_TransportTimeSignatureNumerator,  
    kJBox_TransportTimeSignatureDenominator,  
    kJBox_TransportRequestResetAudio,  
    kJBox_TransportLoopEnabled,  
    kJBox_TransportLoopStartPos,  
    kJBox_TransportLoopEndPos,  
    kJBox_TransportBarStartPos,  
    kJBox_TransportRequestRun,  
    kJBox_TransportRequestStop,  
    kJBox_TransportMuted,  
    kJBox_TransportPatternStartPos,  
    kJBox_TransportPatternIndex  
};
```

TJBox_EnvironmentTag

```
#include <JukeboxTypes.h>
```

```
enum TJBox_EnvironmentTag {  
    kJBox_EnvironmentMasterTune,  
    kJBox_EnvironmentSystemSampleRate,  
    kJBox_EnvironmentInstanceID,  
    kJBox_EnvironmentDeviceVisible,  
    kJBox_EnvironmentPlayerBypassed  
};
```

TJBox_AudioInputTag

```
#include <JukeboxTypes.h>
```

```
enum TJBox_AudioInputTag {  
    kJBox_AudioInputBuffer,  
    kJBox_AudioInputConnected  
};
```

TJBox_AudioOutputTag

```
#include <JukeboxTypes.h>

enum TJBox_AudioOutputTag {
    kJBox_AudioOutputBuffer,
    kJBox_AudioOutputConnected,
    kJBox_AudioOutputDSPLatency
};
```

TJBox_CVInputTag

```
#include <JukeboxTypes.h>

enum TJBox_CVInputTag {
    kJBox_CVInputValue,
    kJBox_CVInputConnected
};
```

TJBox_CVOutputTag

```
#include <JukeboxTypes.h>

enum TJBox_CVOutputTag {
    kJBox_CVOutputValue,
    kJBox_CVOutputConnected,
    kJBox_CVOutputDSPLatency
};
```

TJBox_UserSampleTag

```
#include <JukeboxTypes.h>

enum TJBox_UserSampleTag {
```

```
    kJBox_UserSampleRootKey,  
    kJBox_UserSampleTuneCents,  
    kJBox_UserSamplePlayRangeStart,  
    kJBox_UserSamplePlayRangeEnd,  
    kJBox_UserSampleLoopRangeStart,  
    kJBox_UserSampleLoopRangeEnd,  
    kJBox_UserSampleLoopMode,  
    kJBox_UserSamplePreviewVolumeLevel,  
    kJBox_UserSampleItem  
};
```

TJBox_DeviceHostTag

```
#include <JukeboxTypes.h>  
  
enum TJBox_DeviceHostTag {  
    kJBox_DeviceHostSampleContext,  
    kJBox_DeviceHostDeleteSample,  
    kJBox_DeviceHostEditSample  
};
```

TJBox_PatternsTag

```
#include <JukeboxTypes.h>  
  
enum TJBox_PatternsTag {  
    kJBox_PatternsLength  
};
```

Typedefs

TJBox_AudioFramePos

Type that specifies a position within an audio buffer.

```
#include <JukeboxTypes.h>
```

```
typedef TJBox_Int32 TJBox_AudioFramePos;
```

An audio frame consists of one sample for each channel in the buffer.

TJBox_AudioSample

Type that represents a single-channel audio sample.

```
#include <JukeboxTypes.h>
```

```
typedef TJBox_Float32 TJBox_AudioSample;
```

TJBox_PropertyKey

Type that represents a property key.

```
#include <JukeboxTypes.h>
```

```
typedef char TJBox_PropertyKey[kJBox_MaxPropertyNameLen+1];
```

TJBox_ObjectName

Type that represents a Motherboard property set object name.

```
#include <JukeboxTypes.h>
```

```
typedef char TJBox_ObjectName[kJBox_MaxObjectNameLen+1];
```

TJBox_ObjectRef

Motherboard property set object reference.

```
#include <JukeboxTypes.h>
```

```
typedef TJBox_Int32 TJBox_ObjectRef;
```

TJBox_Tag

Property tag.

```
#include <JukeboxTypes.h>
```

```
typedef TJBox_UInt32 TJBox_Tag;
```

Variables

kJBox_SilentThreshold

Constant that is used to check for silent audio samples.

```
#include <Jukebox.h>
```

```
const TJBox_Float32 kJBox_SilentThreshold;
```

If the following expression is true the sample should be considered silent:

```
abs(sample) < kJBox_SilentThreshold
```

Defines

JBOX_ASSERT

If the expression given to this macro compares equal to zero (i.e., the expression is false), a message to this effect is presented and the 45 is disabled.

```
#include <Jukebox.h>
```

```
#define JBOX_ASSERT(expression)
```

This macro only has effect in LLVM and Debugging builds of the 45 (not in Deployment).

Parameters:

expression

The expression to be evaluated; is shown in the message.

JBOX_ASSERT_MESSAGE

If the expression given to this macro compares equal to zero (i.e., the expression is false), a message to this effect is presented and the 45 is disabled.

```
#include <Jukebox.h>

#define JBOX_ASSERT_MESSAGE(expression, message)
```

This macro only has effect in LLVM and Debugging builds of the 45 (not in Deployment).

Parameters:

`expression` [in]

The expression to be evaluated; is shown in the message.

`message` [in]

A null-terminated text string that can be used to provide additional information to the user.

JBOX_TRACE

Utility function that calls JBox_Trace with the specified message.

```
#include <Jukebox.h>

#define JBOX_TRACE(message)
```

Parameters:

`message` [in]

The message to trace.

See also:

JBox_Trace()

JBOX_TRACEVALUES

Utility function that calls JBox_TraceValues with the specified input.

```
#include <Jukebox.h>

#define JBOX_TRACEVALUES(template, values, valueCount)
```

Parameters:

`template [in]`

Message to be printed. Zero-terminated ASCII or UTF-8. The substring

`^0`

will be replaced with `iValues[0]`, the substring

`^1`

will be replaced with `iValues[1]`, etc.

`values [in]`

Array of values to be traced.

`valueCount [in]`

Size of the value array.

See also:

`JBox_TraceValues()`

`kJBox_MaxPropertyNameLen`

The maximum length of property names in Jukebox, not including the terminating zero.

```
#include <JukeboxTypes.h>
```

```
#define kJBox_MaxPropertyNameLen
```

KJBox_MaxObjectNameLen

The maximum length of Motherboard property set object names in Jukebox, not including the terminating zero.

```
#include <JukeboxTypes.h>

#define KJBox_MaxObjectNameLen
```

KJBox_NoPatternIndex

A value used by /transport/pattern_index to indicate that no pattern is selected.

```
#include <JukeboxTypes.h>

#define KJBox_NoPatternIndex
```

KJBox_InvalidPropertyTag

Invalid property tag.

```
#include <JukeboxTypes.h>

#define KJBox_InvalidPropertyTag
```

Functions

JBox_Export_RenderRealtime

Performs realtime audio processing for a 45.

```
void JBox_Export_RenderRealtime(  
    void * iInstanceData,  
    TJBox_PropertyDiff[] iPropertyDiffs,  
    TJBox_UInt32 iDiffCount  
)
```

Processes realtime audio streams and reacts to motherboard changes.

JBox_Export_RenderRealtime is usually called in a realtime context, and must perform its work in as short time as possible. (The total time available for audio processing for the entire rack is about 1.5 milliseconds per batch.)

Only a subset of the functions in the Jukebox Toolbox can be called from JBox_Export_RenderRealtime. Calling illegal functions will abort the current JBox_Export_RenderRealtime call and disable the 45 instance.

It is not possible to do memory allocations or time-consuming calculations from JBox_Export_RenderRealtime; use JBox_Export_CreateNativeObject for such tasks.

A list of subscribed properties that have changed since the last call to JBox_Export_RenderRealtime is given as input. Enabling change notification for properties is optional, since it is always possible to inspect the MOM directly from JBox_Export_RenderRealtime. However, without notifications it is possible to miss multiple property changes inside a single batch (e.g., a gate property switching between off and on several times during a batch).

The differences in the list are sorted by their respective frame position within the current batch. The changes have already been applied to the MOM when JBox_Export_RenderRealtime is called.

If an audio output is not written to during a call to JBox_Export_RenderRealtime, that audio output will be considered silent for the current batch. This information is used by the host to optimize the audio rendering chain. If an audio output is written to, all the values in the DSP buffer must be updated. Storing nan:s or inf:s in a DSP buffer is not allowed.

If a CV output is not written to during a call to JBox_Export_RenderRealtime, that output will keep its current value, just like custom properties.

Parameters:

`iInstanceData` [in]

Current value of the "environment/instance" property, for convenience. The value of this property normally represents a native object that holds the C++ state for the 45 instance. The native object is normally created with a call to `make_native_object` when the 45 instance is created.

`iPropertyDiffs` [in]

List of properties that have changed since the last call to JBox_Export_RenderRealtime.

`iDiffCount` [in]

The size of the iPropertyDiffs array.

JBox_Export_CreateNativeObject

Generic constructor for creating C++ objects.

```
void * JBox_Export_CreateNativeObject(  
    const char[] iOperation,  
    TJBox_Value[] iParams,  
    TJBox_UInt32 iParamCount  
)
```

This function is used to create or process data that requires complex calculations, memory allocation, or the use of Jukebox Toolbox functions that are unavailable in JBox_Export_RenderRealtime.

JBox_Export_CreateNativeObject is invoked via the realtime controller Lua script, either as an effect of a property change, or during 45 instance initialization to create the instance data that is passed in each call to JBox_Export_RenderRealtime.

The combination of iOperation and iParams that configure the result of JBox_Export_CreateNativeObject is up to each 45 developer to define.

To get access to the native object created by JBox_Export_CreateNativeObject from JBox_Export_RenderRealtime, the object must be assigned to the motherboard via the realtime controller Lua script.

Native objects are ordinary C/C++ data structures and may contain pointers and references. During JBox_Export_CreateNativeObject you can allocate memory with malloc, new, stl containers etc. All these allocations are stored inside the native object and they can refer to each other. There must be no pointers pointing outside of the native object's allocations.

Instances of TJBox_Value that are obtained and used inside a call to JBox_Export_CreateNativeObject are only valid during call. It is not possible to store any such value in the native object.

All memory allocations are noted by the host so if the host must disable a 45, no resources will leak.

Parameters:

`iOperation` [in]

String key used by the realtime controller Lua script to identify which object to create.

`iParams` [in]

Array of parameter values for the operation identified by iOperation.

`iParamCount` [in]

The size of the iParams array.

Returns:

Pointer to the new object.

JBox_Assert

Notifies the Jukebox host about a failed assert.

```
void JBox_Assert(  
    const char[] iFile,  
    TJBox_Int32 iLine,  
    const char[] iFailedExpression,  
    const char[] iMessage  
)
```

Calling JBox_Assert in Debug builds will cause the host to stop 45 execution and notify the user. The host might terminate the 45 or continue execution depending on the implementation. Calls to JBox_Assert in Deployment builds will terminate the 45 instance without notifying the user.

Parameters:

`iFile` [in]

Name of a source code file from where the message originates. Zero-terminated ASCII or UTF-8.

`iLine` [in]

Line number in the source file where the message originates.

`iFailedExpression` [in]

String that represents the expression that failed. Zero-terminated ASCII or UTF-8.

`iMessage` [in]

String containing the message to present to the user. Zero-terminated ASCII or UTF-8.

JBox_Trace

Requests that the Jukebox host traces a message.

```
void JBox_Trace(  
    const char[] iFile,  
    TJBox_Int32 iLine,  
    const char[] iMessage  
)
```

The trace message might be written to a console or to a file, depending on the host. The host is free to reformat the message as needed, adding timestamps, etc. A newline is added to each message before it is traced.

Parameters:

iFile [in]

Name of a source code file from where the message originates. Zero-terminated ASCII or UTF-8.

iLine [in]

Line number in the source file where the message originates.

iMessage [in]

String containing the message to present to the user. Zero-terminated ASCII or UTF-8.

JBox_TraceValues

Requests that the Jukebox host traces a formatted message containing TJbox_Value:s.

```
void JBox_TraceValues(  
    const char[] iFile,  
    TJBox_Int32 iLine,  
    const char[] iTemplate,  
    TJBox_Value[] iValues,  
    TJBox_Int32 iValueCount  
)
```

Similar to JBox_Trace(), but with simple message formatting capabilities.

Parameters:

`iFile` [in]

Name of a source code file from where the message originates. Zero-terminated ASCII or UTF-8.

`iLine` [in]

Line number in the source file where the message originates. Zero-terminated ASCII or UTF-8.

`iTemplate` [in]

Message to be printed. Zero-terminated ASCII or UTF-8. The substring

`^0`

will be replaced with `iValues[0]`, the substring

¹

will be replaced with `iValues[1]`, etc.

`iValues` [in]

Array of values to be traced.

`iValueCount` [in]

Size of the value array.

JBox_GetMotherboardObjectRef

Resolves a motherboard property set object path.

```
TJBox_ObjectRef JBox_GetMotherboardObjectRef(  
    TJBox_ObjectName iMOMPath  
)
```

This function is used to obtain a reference to any motherboard property set object (built-in or custom). Object references are valid during the whole lifetime of a 45 instance. Trying to resolve an undefined property by specifying an invalid path will blow the fuse.

Parameters:

`iMOMPath` [in]

Path identifier for the motherboard property set object, e.g., "custom_properties", "note_states", "audio_inputs", "cv_outputs", etc.

Returns:

Reference to the specified motherboard property set object.

See also:

`JBox_MakePropertyRef()`

JBox_MakePropertyRef

Obtain a property reference.

```
TJBox_PropertyRef JBox_MakePropertyRef(  
    TJBox_ObjectRef iObject,  
    TJBox_PropertyKey iKey  
)
```

Property references are valid during the whole lifetime of a 45 instance. Specifying an invalid property set object reference or unknown property key will blow the fuse.

Parameters:

`iObject` [in]

Reference to the Motherboard property set object that holds the requested property.

`iKey` [in]

Name of the property.

Returns:

Reference to the specified property.

See also:

JBox_GetMotherboardObjectRef()

JBox_IsReferencingSameProperty

Test two property references for equality.

```
TJBox_Bool JBox_IsReferencingSameProperty(  
    TJBox_PropertyRef iProperty1,  
    TJBox_PropertyRef iProperty2  
)
```

Two property references are equal if they point to the same property key and are inside the same Motherboard property set object.

Parameters:

`iProperty1 [in]`

First property to be compared.

`iProperty2 [in]`

Second property to be compared.

Returns:

True if iProperty1 and iProperty2 represents the same property, false otherwise.

JBox_GetPropertyTag

Retrieves the user-defined (or system-defined) tag for a property.

```
TJBox_Tag JBox_GetPropertyTag(  
    TJBox_PropertyRef iProperty  
)
```

Numerical tags can be assigned to Motherboard properties to facilitate switch statements or looping over ranges of properties.

All property tags inside a specific Motherboard property set object must be unique. The same tag may be reused in different Motherboard property set objects.

Requesting an invalid tag will terminate the 45 instance.

Parameters:

`iProperty` [in]

The property from which the tag is obtained.

Returns:

The tag for the specified property.

JBox_FindPropertyByTag

Retrieves a property via its numerical tag.

```
TJBox_PropertyRef JBox_FindPropertyByTag(  
    TJBox_ObjectRef iObject,  
    TJBox_Tag iTag  
)
```

Requesting an invalid tag will terminate the 45 instance.

Parameters:

`iObject` [in]

Reference to the Motherboard property set object that holds the requested property.

`iTag` [in]

The tag of the property to retrieve.

Returns:

The specified property.

JBox_LoadMOMProperty

Retrieve the value of a property.

```
TJBox_Value JBox_LoadMOMProperty(  
    TJBox_PropertyRef iProperty  
)
```

Requesting the value of an invalid property will terminate the 45 instance. Cannot be called from JBox_Export_CreateNativeObject.

Parameters:

`iProperty` [in]

The property to retrieve.

Returns:

The value of the property.

JBox_StoreMOMProperty

Update the value of a property.

```
void JBox_StoreMOMProperty(  
    TJBox_PropertyRef iProperty,  
    TJBox_Value iValue  
)
```

Only properties in the `rt_owner` scope can be updated. The updated value can be read back directly (using, e.g., `JBox_LoadMOMProperty`), but it will not be propagated to the Motherboard until `JBox_Export_RenderRealtime` returns. Updating the value of an invalid property will terminate the 45 instance. Cannot be called from `JBox_Export_CreateNativeObject`.

Parameters:

`iProperty` [in]

The property to update.

iValue [in]

The new value.

JBox_LoadMOMPropertyByTag

Retrieve the value of a property.

```
TJBox_Value JBox_LoadMOMPropertyByTag(  
    TJBox_ObjectRef iObject,  
    TJBox_Tag iTag  
)
```

Requesting the value of an invalid property will terminate the 45 instance. Cannot be called from JBox_Export_CreateNativeObject.

Parameters:

`iObject` [in]

The Motherboard property set object that contains the property to retrieve.

`iTag` [in]

The tag of the property to retrieve.

Returns:

The value of the property.

JBox_StoreMOMPropertyByTag

Update the value of a property.

```
void JBox_StoreMOMPropertyByTag(  
    TJBox_ObjectRef iObject,  
    TJBox_Tag iTag,  
    TJBox_Value iValue
```

)

Only properties in the `rt_owner` scope can be updated. The updated value can be read back directly (using, e.g., `JBox_LoadMOMProperty`), but it will not be propagated to the Motherboard until `JBox_Export_RenderRealtime` returns. Updating the value of an invalid property will terminate the 45 instance. Cannot be called from `JBox_Export_CreateNativeObject`.

Parameters:

`iObject` [in]

The Motherboard property set object that contains the property to updated.

`iTag` [in]

The tag of the property to update.

`iValue` [in]

The new value.

JBox_GetType

Returns the native type of a Jukebox dynamic value.

```
TJBox_ValueType JBox_GetType(  
    TJBox_Value iValue  
)
```

Parameters:

iValue [in]

The value to examine.

JBox_GetNumber

Returns the specified dynamic value as a TJBox_Float64.

```
TJBox_Float64 JBox_GetNumber(  
    TJBox_Value iValue  
)
```

If the value is not of the correct type, the host will terminate the 45.

Parameters:

iValue [in]

The value to examine.

JBox_GetStringLength

Returns the length of the specified dynamic value, assuming it represents a string.

```
TJBox_UInt32 JBox_GetStringLength(  
    TJBox_Value iValue  
)
```

If the value is not of the correct type, the host will terminate the 45.

Parameters:

`iValue` [in]

The value to examine.

JBox_GetSubstring

Return a substring from the specified dynamic value, assuming it represents a string.

```
void JBox_GetSubstring(  
    TJBox_Value iValue,  
    TJBox_SizeT iStart,  
    TJBox_SizeT iEnd,  
    char[] oString  
)
```

Copies the eight-bit UTF-8 characters in the range `iStart` (inclusive) to `iEnd` (non-inclusive) to the buffer specified by `oString`. The resulting string is zero-terminated automatically. Calling with an empty range is allowed, and returns a single terminating zero. The `oString` buffer is assumed to have enough space to store the substring (plus a terminating zero character). If the value is not of the correct type, the host will terminate the 45.

Parameters:

`iValue` [in]

The value to copy from.

`iStart` [in]

First index (inclusive) to copy from.

`iEnd` [in]

Last index (non-inclusive) to copy from.

`oString` [out]

Array in which to store the substring.

JBox_GetBoolean

Returns the specified dynamic value as a TJBox_Bool.

```
TJBox_Bool JBox_GetBoolean(  
    TJBox_Value iValue  
)
```

If the value is not of the correct type, the host will terminate the 45.

Parameters:

`iValue` [in]

The value to examine.

JBox_GetNativeObjectRO

Obtain a read-only pointer to a native object.

```
const void * JBox_GetNativeObjectRO(  
    TJBox_Value iValue  
)
```

Native objects are created by calls to `JBox_Export_CreateNativeObject`. All native objects are owned by the host. The pointer returned by this function can not be stored; it is only valid during the execution of the current calling context. This function is only valid for read-only native objects that can be shared between 45 instances.

Parameters:

`iValue` [in]

The dynamic value that represents the native object.

Returns:

Pointer to the native object.

See also:

JBox_Export_CreateNativeObject()

JBox_GetNativeObjectRW

Obtain a read/write pointer to a native object.

```
void * JBox_GetNativeObjectRW(  
    TJBox_Value iValue  
)
```

Native objects are created by calls to JBox_Export_CreateNativeObject. All native objects are owned by the host. The pointer returned by this function can not be stored; it is only valid during the execution of the current calling context. This function is only valid for read/write native objects. The objects cannot be shared between 45 instances: each instance must have its own local copy.

Parameters:

iValue [in]

The dynamic value that represents the native object.

Returns:

Pointer to the native object.

JBox_GetSampleInfo

Returns the sample-related information for a dynamic value, assuming it is a sample.

```
TJBox_SampleInfo JBox_GetSampleInfo(  
    TJBox_Value iValue  
)
```

If the value is not of the correct type, the host will terminate the 45.

Parameters:

`iValue` [in]

The value to examine.

See also:

JBox_GetSampleData()

JBox_GetSampleMetaData

Returns the sample-related meta data for a dynamic value, assuming it is a sample. The difference compared to JBox_GetSampleInfo is that sample parameters and sample state are also included.

```
TJBox_SampleMetaData JBox_GetSampleMetaData(  
    TJBox_Value iValue  
)
```

If the value is not of the correct type, the host will terminate the 45.

Parameters:

`iValue` [in]

The value to examine.

See also:

JBox_GetSampleData()

JBox_GetSampleData

Return a range of samples from the specified dynamic value.

```
void JBox_GetSampleData(  
    TJBox_Value iValue,  
    TJBox_AudioFramePos iStartFrame,  
    TJBox_AudioFramePos iEndFrame,  
    TJBox_AudioSample[] oAudio  
)
```

Copies the sample values in the range iStartFrame (inclusive) to iEndFrame (non-inclusive) to the buffer specified by oAudio. Each frame contains one TJBox_AudioSample per channel. Channel data is interleaved in increasing order. A stereo frame is two TJBox_AudioSample:s long. Retrieve the TJBox_SampleInfo (if the sample is a custom_property) or TJBox_SampleMetaData (if it is a user_sample item) for the sample to obtain information about the number of channels. Calling with an empty range has no effect. The oAudio buffer is assumed to have enough space to store the samples: (iEndFrame-iStartFrame)*channelCount. If the value is not of the correct type, the host will terminate the 45.

The sample data is assumed to be resident in the range given by iStartFrame and iEndFrame. The resident frame count should be checked by calling JBox_GetSampleInfo prior to calling JBox_GetSampleData.

Parameters:

`iValue` [in]

The value to copy from.

`iStartFrame` [in]

First index (inclusive) to copy from.

`iEndFrame` [in]

Last index (non-inclusive) to copy from.

`oAudio` [out]

Array in which to store the samples.

See also:

JBox_GetSampleInfo()

JBox_GetBLOBInfo

Returns the BLOB-related information for a dynamic value, assuming it is a BLOB.

```
TJBox_BLOBInfo JBox_GetBLOBInfo(  
    TJBox_Value iValue  
)
```

If the value is not of the correct type, the host will terminate the 45.

Parameters:

`iValue` [in]

The value to examine.

See also:

JBox_GetBLOBData()

JBox_GetBLOBData

Return a range of bytes from the specified dynamic value.

```
void JBox_GetBLOBData(  
    TJBox_Value iValue,  
    TJBox_SizeT iStart,  
    TJBox_SizeT iEnd,  
    TJBox_UInt8[] oData  
)
```

Copies the bytes in the range iStartFrame (inclusive) to iEndFrame (non-inclusive) to the buffer specified by oData. Calling with an empty range has no effect. The oData buffer is assumed to have enough space to store the bytes: (iEnd - iStart). If the value is not of the correct type, the host will terminate the 45.

Parameters:

`iValue` [in]

The value to copy from.

`iStart` [in]

First index (inclusive) to copy from.

`iEnd` [in]

Last index (non-inclusive) to copy from.

`oData` [out]

Array in which to store the bytes.

See also:

JBox_GetBLOBInfo()

JBox_GetDSPBufferInfo

Returns the DSPBuffer-related information for a dynamic value, assuming it is a DSP buffer.

```
TJBox_DSPBufferInfo JBox_GetDSPBufferInfo(  
    TJBox_Value iValue  
)
```

If the value is not of the correct type, the host will terminate the 45.

Parameters:

`iValue` [in]

The value to examine.

See also:

JBox_GetDSPBufferData()

JBox_GetDSPBufferData

Return a range of samples from the specified dynamic value.

```
void JBox_GetDSPBufferData(  
    TJBox_Value iValue,  
    TJBox_AudioFramePos iStartFrame,  
    TJBox_AudioFramePos iEndFrame,  
    TJBox_AudioSample[] oAudio  
)
```

Copies the sample values in the range iStartFrame (inclusive) to iEndFrame (non-inclusive) to the buffer specified by oAudio. Each DSP buffer frame always contains one TJBox_AudioSample. Calling with an empty range has no effect. The oAudio buffer is assumed to have enough space to store the samples: (iEndFrame-iStartFrame). If the value is not of the correct type, the host will terminate the 45.

Parameters:

iValue [in]

The value to copy from.

`iStartFrame` [in]

First index (inclusive) to copy from.

`iEndFrame` [in]

Last index (non-inclusive) to copy from.

`oAudio` [out]

Array in which to store the samples.

See also:

JBox_GetDSPBufferInfo()

JBox_SetDSPBufferData

Writes a range of samples from a local buffer to a dynamic value, assuming it is a DSP buffer.

```
void JBox_SetDSPBufferData(  
    TJBox_Value iValue,  
    TJBox_AudioFramePos iStartFrame,  
    TJBox_AudioFramePos iEndFrame,  
    TJBox_AudioSample[] iAudio  
)
```

Copies the sample values from the iAudio buffer to the range iStartFrame (inclusive) to iEndFrame (non-inclusive) in the DSP buffer. Each DSP buffer frame always contains one TJBox_AudioSample. The specified range must fit in the DSPBuffer; the DSPBuffer will not be resized. Retrieve the TJBox_DSPBufferInfo for the DSP buffer to obtain information about its size. Calling with an empty range has no effect. If the value is not of the correct type, the host will terminate the 45. Jukebox allows DSP buffers to be modified by the real-time code; no other dynamic value-types may be modified (only replaced).

Parameters:

iValue [in]

The value to copy to.

`iStartFrame` [in]

First index (inclusive) to copy to.

`iEndFrame` [in]

Last index (non-inclusive) to copy to.

`iAudio` [in]

Array from which to copy the samples.

See also:

`JBox_GetDSPBufferInfo()`

JBox_SetRTStringData

Sets the contents of an RT owned string.

```
void JBox_SetRTStringData(  
    TJBox_PropertyRef iProperty,  
    TJBox_SizeT iSize,  
    const TJBox_UInt8[] iData  
)
```

Copies iSize bytes from iData to the RT string. iSize cannot be larger than the max_size defined in the motherboard. If the value is not of the correct type, the host will terminate the 45. Jukebox allows RT owned strings to be modified by the real-time code; no other dynamic value-type except DSP Buffers may be modified (only replaced). The maximum supported rate of updates to a single value is once every 25 milliseconds. If this rate is exceeded, the host will drop some updates and warn in the log (and blow fuse if blow fuse on warnings is enabled)

Parameters:

iProperty [in]

The RT string property to update

iSize [in]

Number of bytes to copy

`iData [in]`

Array from which to copy the bytes.

JBox_OutputNoteEvent

Sends a note event to the host.

```
void JBox_OutputNoteEvent (  
    TJBox_NoteEvent iNoteEvent  
)
```

The specified note event will be sent to the host. An event with zero velocity turns off a note. Events with non-zero velocity turns on a note. This function can only be used by note player devices.

Parameters:

`iNoteEvent` [in]

The note event to be outputted.

JBox_AsNoteEvent

Utility function to convert a note state property diff to a note event.

```
TJBox_NoteEvent JBox_AsNoteEvent(  
    TJBox_PropertyDiff iPropertyDiff  
)
```

This function assumes that the property diff is a note statet property diff with a tag that represents the note number (0-127) and a current value that represents the velocity (0-127). The frame index of the property diff must be a value between 0-63. Passing a property diff with other values, will be treated as a programming error. This function can be used by device of all types.

Parameters:

`iPropertyDiff` [in]

The property diff of a note state.

Returns:

The note event based on the values from the property diff.

JBox_MakeNil

Return an empty dynamic value.

```
TJBox_Value JBox_MakeNil()
```

JBox_MakeNumber

Return a dynamic value that represents a number.

```
TJBox_Value JBox_MakeNumber(  
    TJBox_Float64 iNumber  
)
```

Parameters:

`iNumber` [in]

The number to store in the dynamic value.

JBox_MakeBoolean

Return a dynamic value that represents a boolean.

```
TJBox_Value JBox_MakeBoolean(  
    TJBox_Bool iBoolean  
)
```

Parameters:

`iBoolean` [in]

The bool to store in the dynamic value.

JBox_LoadMOMPropertyAsNumber

Retrieve a number from a specified property.

```
TJBox_Float64 JBox_LoadMOMPropertyAsNumber(  
    TJBox_ObjectRef iObject,  
    TJBox_Tag iTag  
)
```

Requesting the value of an invalid property, or if the property is of the incorrect type, will terminate the 45 instance. Cannot be called from JBox_Export_CreateNativeObject.

Parameters:

`iObject` [in]

The Motherboard property set object that contains the property to retrieve.

`iTag` [in]

The tag of the property to retrieve.

Returns:

The value of the property.

JBox_StoreMOMPropertyAsNumber

Update the value of a property, assuming it is a number.

```
void JBox_StoreMOMPropertyAsNumber(  
    TJBox_ObjectRef iObject,  
    TJBox_Tag iTag,  
    TJBox_Float64 iValue  
)
```

Only properties in the `rt_owner` scope can be updated. The updated value can be read back directly (using, e.g., `JBox_LoadMOMProperty`), but it will not be propagated to the Motherboard until `JBox_Export_RenderRealtime` returns. Updating the value of an invalid property, or if the property is of the incorrect type, will terminate the 45 instance. Cannot be called from `JBox_Export_CreateNativeObject`.

Parameters:

`iObject` [in]

The Motherboard property set object that contains the property to updated.

`iTag` [in]

The tag of the property to update.

`iValue` [in]

The new value.

JBox_GetOptimalFFTAlignment

Tells the optimal alignment for JBox_FFTRealForward and JBox_FFTRealInverse.

```
TJBox_Int32 JBox_GetOptimalFFTAlignment()
```

Returns a power-of-two number ≥ 16 (16 is the base alignment of Jukebox).

Function prototype:

```
TJBox_Int32 JBox_GetOptimalFFTAlignment();
```

See also:

JBox_FFTRealForward(), JBox_FFTRealInverse()

JBox_FFTRealForward

Implements the real Fast Fourier Transform (time domain to frequency domain for real-valued signals).

```
void JBox_FFTRealForward(  
    TJBox_Int32 iFFTSize,  
    TJBox_Float32[] ioData  
)
```

Performs in-place forward FFT of real valued single precision float data.

The transform is calculated in-place by reading N consecutive real values and writing N/2 interlaced (re, im) complex pairs. All scaling is handled internally, there is no windowing applied.

Input values re0, re1, re2, re3... produces output values re0, im0, re1, im1...

Works on power-of-two-sized buffers.

Parameters:

`iFFTSize` [in]

Size of the FFT, expressed as the log2 of the input data array size. For example, iFFTSize = 6 corresponds to an input data array size of 64. iFFTSize must be larger than or equal to 6 and must be smaller than or equal to 15.

`ioData` [inout]

Array that contains the input signal to transform. Will contain the resulting transformed frequency-domain data when this function returns. The output values will be divided by the FFT length. The address of ioData must be aligned to 16-bytes. For optimal performance, align ioData according to JBox_GetOptimalFFTAlignment().

See also:

JBox_GetOptimalFFTAlignment()

JBox_FFTRealInverse

Implements the real Inverse Fast Fourier Transform (frequency domain to time domain for real-valued signals).

```
void JBox_FFTRealInverse(  
    TJBox_Int32 iFFTSize,  
    TJBox_Float32[] ioData  
)
```

Performs in-place inverse FFT of single precision float data.

The transform is calculated in-place by reading N/2 interlaced complex pairs and writing N consecutive real values. All scaling is handled internally, there is no windowing applied.

Input values re0, im0, re1, im1... produces output values re0, re1, re2, re3...

Works on power-of-two-sized buffers.

Parameters:

`iFFTSize` [in]

Size of the FFT, expressed as the log2 of the input data array size. For example, `iFFTSize` = 6 corresponds to an input data array size of 64. `iFFTSize` must be larger than or equal to 6 and must be smaller than or equal to 15.

`ioData` [inout]

Array that contains the input signal to transform. Will contain the resulting transformed time-domain data when this function returns. The output values will be divided by the FFT length. The address of `ioData` must be aligned to 16-bytes. For optimal performance, align `ioData` according to `JBox_GetOptimalFFTAlignment()`.

See also:

`JBox_GetMemoryAligment()`

Introduction

Rack Extensions is a system for creating new devices for the Reason rack. The system is designed to make it easy to create and distribute new devices while still keeping the well-known stability, efficiency and longevity of the Reason rack intact.

A Rack Extension device is a first class rack citizen that looks and behaves just like the built-in Reason devices, with the cables, tight sequencer integration, Remote™ control, multi-core DSP and so on.

About this document

The Rack Extension Developer's Guide is an introductory guide for anyone who wants to build Rack Extensions. It provides an overview of the technologies and tools needed, the main concepts you need to learn to develop Rack Extensions and how the Rack Extension distribution model works.

It is expected that the reader of this guide knows about general software development concepts, and knows enough about Reason to see where the Rack Extension concepts described here fit in.

The technology behind Rack Extensions is called **Jukebox**. A device in the Jukebox world is called a **45**. We have tried to be consistent throughout this guide by using the Rack Extension ("RE") terms when talking about the product aspect, and the corresponding Jukebox terms when talking about the technology.

Readers that are familiar with digital signal processing and knows a bit of C++ should be able to create their first Rack Extension using Reason Studios-supplied GUI components after reading this guide, with a little extra help from specification documentation, tutorials, and examples.

This document does not provide detailed information about APIs or step-by-step tutorials on different Rack Extension development tasks. Please refer to the following sources for more information:

- *Reason Operation Manual* - The user guide that is distributed with Reason. Describes how Reason works.

- *Jukebox Concepts* - Describes how the different Jukebox APIs and subsystems work together.
- *Jukebox Scripting Specification* - Reference documentation for all Jukebox related APIs (except C++).
- *Jukebox C++ API* - Reference documentation for the C++ API.
- *2D GUI Designer Manual* - Describes the workflow, the tools and formats involved in creating 2D Jukebox GUIs.
- *GUI Design Guidelines* - Requirements and guidelines for Jukebox GUIs.
- Examples and tutorials - Distributed with the Jukebox SDK.
- Rack Extension Developer Forum - Online forum for information exchange between Rack Extension developers. You can access the forum from <https://developer-support.reasonstudios.com/hc/en-us/community/topics>

About Rack Extensions

Rack Extensions live in the Reason rack, and are integrated in the Reason environment that provides much of the core functionality that is expected of a device in the rack.

The Reason rack environment provides:

- Multi platform support (the same Rack Extension runs on all platforms supported by Reason)
- Scripted data model
- Undo system
- Unified patch format
- ReFill support
- Patch and sample browsing
- Background sample loading
- Audio and CV cables
- Multicore support and thread management
- Sequencer integration
- Remote™ control

This leaves three parts for the Rack Extension developer to implement:

- GUI definition
- Data model definition
- Realtime DSP code

The GUI definition uses various Jukebox-specific file formats together with common and open file formats for describing 2D content.

The Lua scripting language (<http://www.lua.org/>) is used for data model definitions and all scripting needs.

The real-time DSP code is written in C++.

The Rack Extension (RE) life cycle

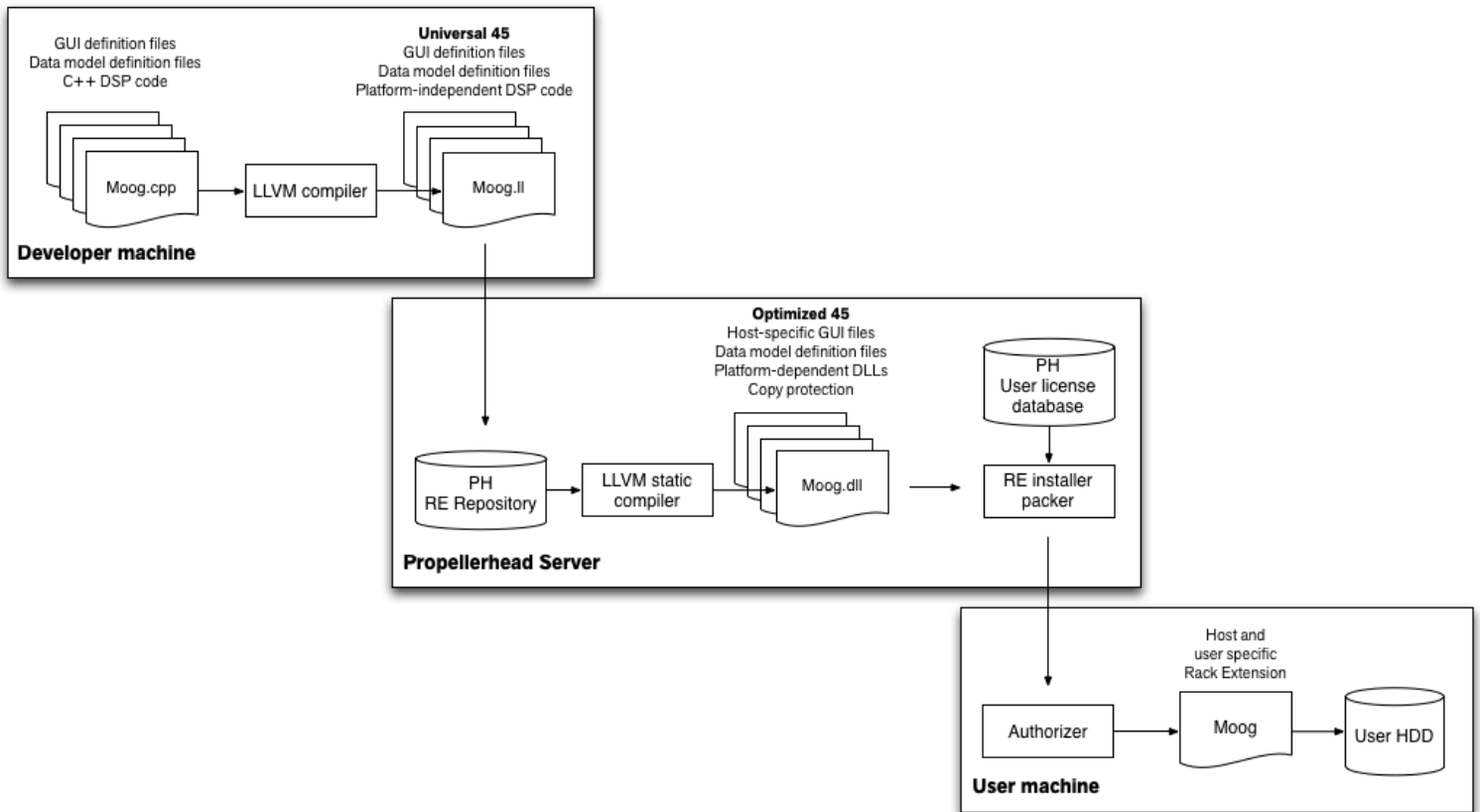
An important design goal for Reason and the Jukebox technology is to take good care of user documents. A Reason song document should live forever. Songs created in Reason 1.0 should load in all subsequent versions, and should sound the same as they did back in the year 2000.

To meet that goal, completed RE's are submitted to Reason Studios, and can only be distributed via the Reason Studios Shop. Once in the Shop, Reason Studios will make sure that submitted RE's will work on all supported platforms, and that they will keep working on all future Reason versions.

This means that the RE must be submitted in a format that is as portable and future proof as possible. Using the tools provided in the Jukebox SDK, all code and resources that are part of an RE are compiled into a platform independent *Universal 45*, ready for submission to the Reason Studios Shop.

When a user later needs a Rack Extension from the Shop, Reason Studios *Authorizer* application downloads and installs the correct binary (called an *Optimized 45*) for the current platform and Reason version. The transformation from platform independent format is done on the server, with no involvement from the developer. The Authorizer also handles copy protection and license management.

Schematic overview of the Rack Extension development workflow.



Jukebox runtime environment

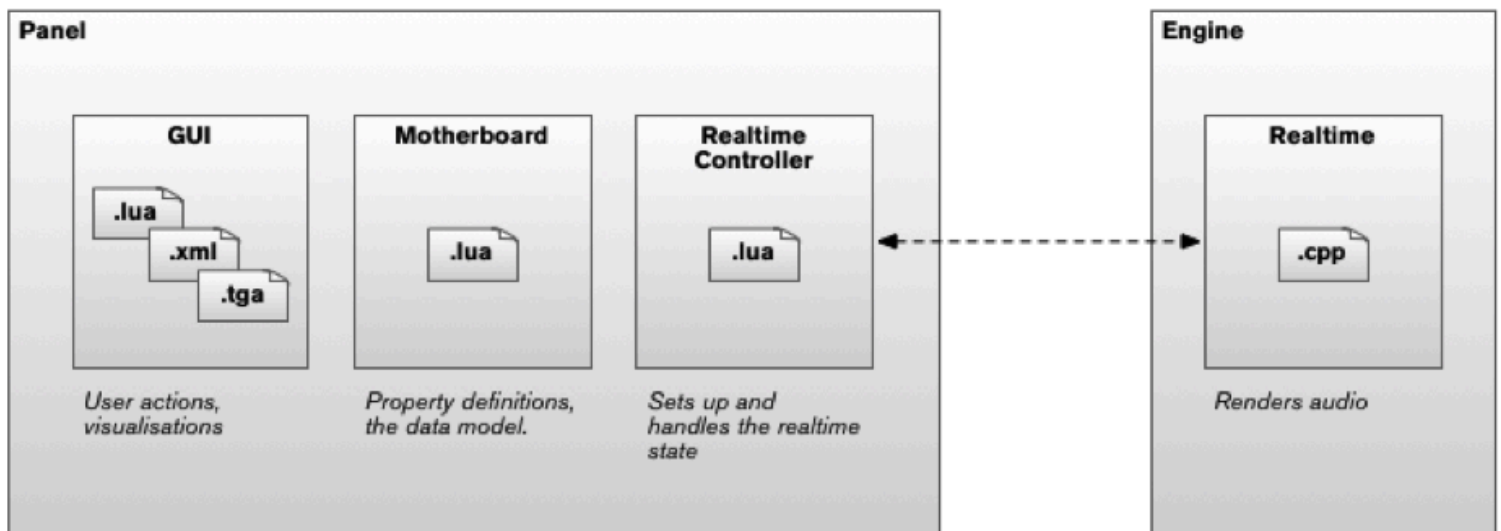
The Jukebox APIs and runtime environment are designed to make it easy for REs to be good rack citizens. Much is data-driven or scripted in Lua. The native code that makes up the realtime part runs in a sandboxed environment, which is both limited in terms of functionality and keeps REs under close supervision.

There is, for example, no direct access to OS libraries, and a Rack Extension that tries to allocate memory while rendering audio will be shut down.

As illustrated in [\[FIG2\]](#), the Jukebox runtime environment is split into four layers:

- GUI
- Motherboard
- Realtime controller
- Realtime

The Jukebox runtime environment.



The following sections describe each layer briefly. More detailed information is available in the other SDK documents.

GUI

The need for Rack Extensions to be future-proof and platform independent puts high demands on the GUI system design. It will not be enough to build the GUI using bitmaps of today's screen resolution. It is also important that the user experience is consistent when it comes to turning knobs, etc.

A Rack Extension GUI must have:

- Scalable graphics that works from lower resolution small displays to higher resolution displays of tomorrow.
- Pixel-unaware drawing and user actions. In fact, there can be no assumptions made on any kind of input method. That is all handled by Reason.

The solution in Jukebox is that the GUI is defined as a set of high- resolution 2D images that are connected to the motherboard via Lua scripts.

Reason Studios provides a *Part Library* that contains reusable parts such as knobs and buttons, but it is perfectly possible to create custom parts as well. The *Part Library* can be downloaded from the developer website.

See the *2D GUI Designer Manual* to get started with 2D user interfaces.

Motherboard

The *motherboard* is where the data model and much of the logic of a 45 lives. This is also the part that provides a lot of the core functionality of the Jukebox environment.

From a developer's standpoint, the motherboard is exposed as a set of objects with properties, and a toolkit of functions for inspecting and operating on these objects.

Part of the *motherboard object model* (MOM) is fixed and provided by Jukebox. This includes properties such as the current sample rate and song position. The rest of the MOM is defined by the developer as *custom properties* and a list of *input and output sockets*. The custom part of the MOM is defined in a Lua script file called `motherboard_def.lua`.

Defining custom properties automatically hooks them into the host environment, which provides support for:

- Undo
- Patches
- Automation
- MIDI mapping
- Remote™ control
- Combinator control
- ReWire integration

Defining input and output sockets for audio and CV enables support for cables and auto-routing.

The motherboard also contains *localized property descriptions* (used in tooltips, etc.) and *textualizers*, i.e., definitions that describe how to convert property values to localized text (used in the sequencer for automation, etc.).

Realtime and realtime controller (RTC)

The realtime part of a Rack Extension is implemented in C++ and runs in a sandboxed environment in the Reason engine.

Reason is split into two main parts, the *panel* and the *engine*. The panel is responsible for drawing the GUI, but also manages the rack and all document handling, sample loading, etc. The engine is where the realtime processing is done. The Reason code is structured so that as much complexity as possible is put on the panel side, keeping the realtime engine as simple as possible. Data is passed between the two using message queues. The panel owns the user document state. When Reason is ReWired to a host, the panel and engine live in two separate processes.

Note: If you are familiar with ReWire development, some terms used in this document can be confusing. The Reason engine implements what in ReWire terms is called a ReWire device. The term device in this document is only used to refer to a virtual apparatus living in the Reason rack.

Jukebox is designed to keep as much functionality as possible away from the realtime context. The APIs are designed so that it is hard to make crucial errors that affect other devices or the Reason engine itself. Misbehaving 45's will be immediately shut down, and the user will be notified.

There is a specific part of the motherboard that is responsible for controlling the realtime state, the *realtime controller*(RTC). In many cases the hardwired behavior of the RTC is enough, but more detailed control can be implemented in a Lua script if needed. An example would be when a set of samples needs to be loaded when a document property changes its value.

The C++ interface that needs to be implemented by the 45 realtime consists of two functions:

C/C++

```
void JBox_Export_RenderRealtime(
    void* iPrivateState,
    const TJBox_PropertyDiff iPropertyDiffs[],
    TJBox_UInt32 iDiffCount);

void* JBox_Export_CreateNativeObject(
    const char iOperation[],
    const TJBox_Value iParams[],
    TJBox_UInt32 iCount);
```

The host continuously calls `RenderRealtime` to process audio. Properties that are marked for subscription in the motherboard definition and that have changed since the last call to `RenderRealtime` will be provided as a list of differences. It is also possible to inspect the MOM directly.

The `CreateNativeObject` function is called by the host to create any C++ data structure that is needed by `RenderRealtime`. Such data structures are called *native objects* in Jukebox. Audio processing is separated from memory allocation for performance reasons, so calls to `CreateNativeObject` can only be initiated from RTC scripts. Accessing native objects is only possible from the realtime code. When a 45 instance is created, `CreateNativeObject` is called to create the private state that is passed in each call to `RenderRealtime` (via the `iPrivateState` pointer).

Both `RenderRealtime` and `CreateNativeObject` have access to a toolkit of C++ functions that provides manipulation of Jukebox dynamic values and other functionality. Note that only `RenderRealtime` has direct access to the MOM; `CreateNativeObject` is purely functional and can only receive inputs via its arguments. The different functions in the toolkit verify that they are called from a supported context.

The sandbox records all memory allocations done in `CreateNativeObject`. This makes it possible to close down a misbehaving 45's at any time without causing any memory leaks. It also gives the sandbox complete control of the realtime state of a 45. Since the rest of the state is controlled by the motherboard, this even makes it possible for Reason to double-buffer 45 instances, or to do speculative rendering.

Audio and CV sockets

A 45 can have any number of audio and CV inputs and outputs for connecting to other Reason devices. Reason takes care of routing and persistence of connections. It is possible to specify auto-routing rules that helps Reason to make decisions on how to auto-connect devices when needed.

Rack Extension Portability

Jukebox keeps the GUI and motherboard definitions portable by using platform independent concepts and scripts, and by using abstractions that hides platform specifics from the developer. The same general idea holds for the realtime code as well; it is not possible to directly access OS functions or create threads for example. But the same binary will not work on all supported platforms. This is where the *LLVM toolchain* comes to the rescue.

The LLVM Project (<http://llvm.org>) is a collection of modular and reusable compiler and toolchain technologies. Despite its name, LLVM has little to do with traditional virtual machines, though it does provide helpful libraries that can be used to build them.

LLVM began as a research project at the University of Illinois, with the goal of providing a modern, SSA-based compilation strategy capable of supporting both static and dynamic compilation of arbitrary programming languages. Since then, LLVM has grown to be an umbrella project consisting of a number of different subprojects, many of which are being used in production by a wide variety of commercial and open source projects as well as being widely used in academic research.

Jukebox uses LLVM to compile C++ code into *LLVM bitcode* and to statically compile that bitcode to the correct native binary for the target platform. Jukebox also takes advantage of the open architecture provided by LLVM to interoperate with the code generation and optimization passes.

The Jukebox SDK provides the latest version of LLVM tool chain currently supported by Reason Studios. That tool chain also contains small changes needed to support the targets defined by Jukebox.

The binaries that are submitted to the Reason Studios Shop are in LLVM bitcode format.

Sandboxing

To make the Reason rack as robust as possible while running 3rd party code, each 45 instance has a *virtual fuse* that will blow when the 45 misbehaves. To be able to detect this, each 45 instance lives in its own sandbox, with limited functionality. When the fuse blows, the user is notified that something went wrong and is encouraged to save the document. Other parts of Reason (including other Rack Extensions) will continue to function as if nothing had happened.

All Lua scripts run in a virtual machine, which makes them sandboxed automatically. Reason has full control of the functions that are available in each Lua VM and how calls out of each VM are handled.

The C++ code does run as native code in the Reason engine, and as such it is more sensitive to other native code that runs in the same process than the Lua scripts. To reduce the risk of error, the C++ APIs have been designed to make it difficult to cause catastrophic problems. Hardware exceptions such as access faults are caught and will blow the fuse. Misusing the C++ APIs or spending too much time during realtime rendering will also blow the fuse.

Patches

Patches in Reason represent a collection of settings for a device that can be saved as an external file, a patch file. Patch files exist as files on disk or in a ReFill. With Jukebox, patches can also live inside a 45.

When a Reason song is saved, the current patch for a device is saved in the song data, essentially embedding the patch file in the song. After the patch has been loaded into a device, there are no dependencies to the loaded patch file, other than for browsing purposes. If a patch refers to samples, however, they are not always embedded in the song. Thus, song files may have external dependencies.

All 45s share a common, text-based patch format that is a representation of a subset of the motherboard custom properties. All reading and writing of these patch files are handled by Reason.

Localization

All texts that are intended for user display can be localized. Reason is currently localized to American English, French, German and Japanese. The localized text resources are specified in a separate Lua file for each language. Localization to English is required. If a localization is provided, all texts must be present in the localization to avoid mixed language interfaces.

Rack Extension distribution

45s are distributed through a single channel, the Reason Studios Shop. When a user buys a 45, the Authorizer application will ensure that the 45 is downloaded and installed, and that the license is written to the user's Ignition Key, if applicable. The Authorizer application is installed together with Reason.

Copy Protection

Copy protection of 45s is handled by the Reason Studios Shop distribution mechanism and by Reason. Copy protection is applied by Reason Studios to submitted 45s before they are added to the shop, and Reason is only able to open protected 45s. There is no need for developers to implement protection for each 45.

A goal of the Jukebox copy protection mechanism is to achieve the same level of security as with the mechanism used for Reason licenses. Each time a user launches Reason, either the Reason Studios Ignition Key or the online verification mechanism in Reason will be used to verify that the user has a license for each 45 installed. Downloaded 45s will be licensed to work only for a specific user. This, combined with a single distribution channel will ensure that only licensed users get access to 45s.

Uploading, submission and validation

During development, testing and debugging 45s are done locally using *Reason Recon*. Recon is a special version of Reason that is able to load *Local 45s*, i.e., local builds of the Rack Extension that resides on the developer's machine. Local 45s cannot be opened in the main version of Reason.

When the 45 is mature enough for external testing, or testing by a larger group, it can be uploaded to the Reason Studios distribution servers. When the 45 has been uploaded and built, beta test versions can be distributed to anyone with a Reason Studios account. Such beta test licenses are time-limited and online-only. Users with a beta test license can download the 45 from their account pages and start testing in the main version of Reason.

When testing is complete and the 45 is ready for release, it must be submitted for acceptance. Reason Studios will then perform a series of acceptance tests. When the device is accepted, a release version of it will be built and stored in the Reason Studios Rack Extension repository.

Before release, a shop product page for the device must be created. Product pages are also submitted for acceptance. When both the device and the product page have been accepted, the device will appear automatically in the Reason Studios Shop.

The same procedure applies to updates to existing products, but additional tests are carried out to ensure that the update is fully backwards compatible with earlier versions. No incompatible changes to properties are allowed: old songs and patches must always be possible to load, and they must sound the same.

Introduction

This document contains information about 2D design-related issues for these two paths.

The documentation distinguishes between *requirements* and *guidelines*. A requirement has to be followed – any device that does not meet a requirement will be rejected. Guidelines do not have to be followed, but you are strongly encouraged to do so. Note that Reason Studios reserves the right to reject your device GUI for any reason, even if it follows all the requirements and guidelines in this document.

Design guidelines for 3D

Important! 3D GUI is deprecated in SDK version 4.2.0

Design guidelines for 2D

The following sections refer to designing the GUI in 2D.

Panel dimensions

- **Requirement.** All Rack Extensions have a front panel, a back panel, a folded front panel, and a folded back panel.

- **Requirement.** The width of all panel images must be 3770 pixels. The Player panels are more narrow, but the images should still be 3770 pixels wide. The left and right margins are automatically cropped. The visible width of a Player device is 3590 pixels (front) or 3180 pixels (back).
- **Requirement.** The height of the device must be an integer multiple of a rack unit (345 pixels). The height must not exceed 9U. The front panel and back panel must have the same height.
- **Requirement.** The height of the folded front panel and the folded back panel must both be 150 pixels. (The width is the same as the non-folded panels.)

Image formats

- **Requirement.** All 2D GUI image assets must be submitted in PNG format.
- **Requirement.** Film strip images must be organized vertically. The first animation frame is at the top of the film strip.
- **Requirement.** The height of a film strip image must be an integer multiple of the number of animation frames.

Custom displays

- **Requirement.** Custom display image resources may not be animated.
- **Requirement.** Static decoration widgets may only overlap custom display widgets.
- **Requirement.** Static decoration image resources may not be animated.

Readability

- **Requirement.** All functionality-related texts on your device panel must be readable in Reason on a present-day 27" monitor.
- *Guideline.* For most fonts, choosing a size of 43 pixels enough to be on the safe side.

Required device parts

In order for Reason to work properly, a number of device parts must exist:

- **Requirement.** Each Effect device must have a on/off/bypass button in the top left corner.
- **Requirement.** Each Player device must have a on/off button. There are no requirements regarding position and design.
- **Requirement.** For devices that support user-loaded patches, the patch name must be shown in a patch display. Next to all patch displays there must be an up/down/file/save button group. This widget set must use the Reason Studios-supplied image resources (2D).
- **Requirement.** Each device must display its user-supplied name on a piece of tape. The tape must be present on all panel versions (front, back, folded front, folded back). The piece of tape must use the Reason Studios-supplied image resource (2D).
- **Requirement.** The back panel of all 45s must include one copy of a “reserved space”, or placeholder, which is represented by a Reason Studios-supplied image (2D). There must be no text or other decorations beneath the placeholder.
- **Requirement.** The back panel of all 45s must include routing symbols (see [\[FIG_2\]](#) below) that explain how the device routes stereo and mono signals. These symbols must use the Reason Studios-supplied icons (Routing_Icon_1, Routing_Icon_2, etc.), but you may change their color if necessary. For details on what the icons mean, see the Jukebox Scripting Specification.
- **Requirement.** The folded back panel of all 45s (except Players) must include a hole where the patch cables originate. You may use your own design for this hole, but it must be present.
- **Requirement.** For devices that use audio/CV sockets on the back panel (these are only allowed on the back panel), the Reason Studios-supplied image (2D) must be used.
- **Requirement.** For devices that use CV trim knobs on the back panel, the Reason Studios-supplied image (2D) must be used.
- **Requirement.** For devices that have visible rack screws, the screws must be aligned horizontally (along the X-axis) with the rack holes.
- **Requirement.** There must be an empty margin of at least 25 pixels (2D) along the left and right edges of all panels, with respect to interaction widgets. That is, no widgets that respond to user input may intersect these margins.
- **Requirement.** All widget boundaries must be completely inside the boundaries of their corresponding panel. That is, no widget boundary may extend beyond the edges of its panel.
- **Requirement.** The fold/unfold arrow in Reason must not overlap with any widget.
- **Requirement.** For instrument devices, there must be a "Note On" indicator on the front and folded front panels.
- *Guideline.* Keep a 25 pixels (2D) empty margin between widget edges and the top and bottom edges of the panel.

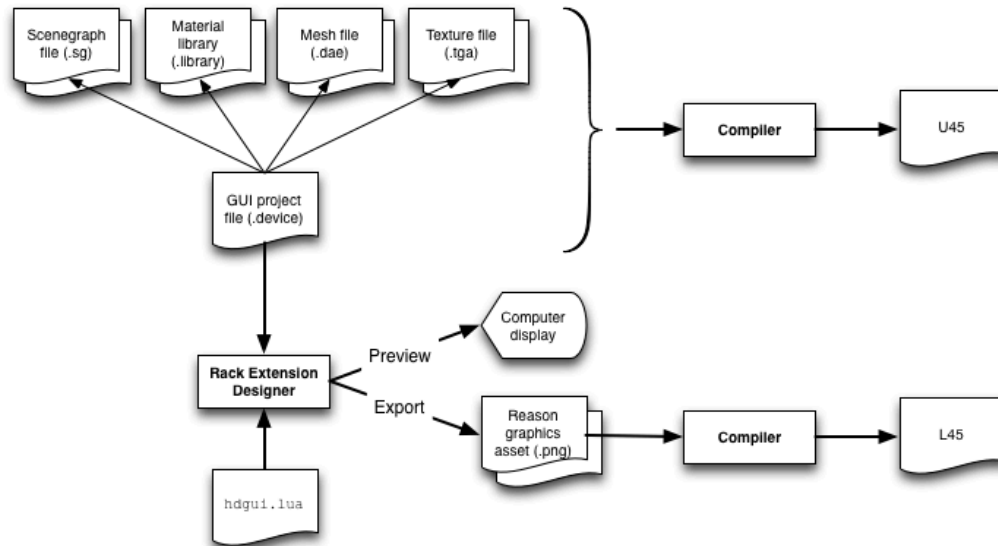


Figure 1. Routing icons

Plausibility

As we see it at Reason Studios, a Rack Extension is an emulated hardware unit - something that could sit in an actual, physical rack. The Reason devices are “95% realistic”: although some “magic” is allowed (a device can be folded, for example), they mostly work as they would have done if they had been built in physical reality. We say that Rack Extensions should be “plausible”. A Rack Extension should have its own character and manufacturer style, but it should also feel at home (fit visually and interaction-wise) in the Reason rack.

Reason Studios strongly recommends that you adhere to the following guidelines. We believe that making your Rack Extension plausible and fit in the rack will make both the Rack Extension, and Reason itself, more successful. One way of testing if your Rack Extension fits in the rack is to fill the rack with existing Reason devices and add your own Rack Extension. Does the result feel OK, or does your Rack Extension stand out in any (negative) way?

- **Requirement..** Your GUI must not interfere with system graphics in Reason, such as frames around parameters (automation indication), blinking red lights (general warning indication), blue/yellow arrows on parameters (remote override indications), and so on.

- **Requirement.** A Rack Extension is a machine used in music production, not a decoration or a promotional space. The company or Rack Extension name logotype must not be too dominant. No advertising banners, etc., are allowed.
- *Guideline.* Use an orthogonal projection ("straight from the front", no perspective).
- *Guideline.* Lighting the device consistently with the Reason Studios devices makes the user experience of the rack consistent. Try to make the direction and contrast of shadows similar to those in the legacy Reason devices.

LIGHT SOURCE	DIRECTION	COLOR (FLOATING POINT RGB)
Main	(-0.08, 0.38, 1.0)	(3.0, 3.0, 3.0)
Environment 1	(-1.0, 0.02, 0.1)	(0.9563, 0.8467, 0.6864)
Environment 2	(1.0, 0.02, 0.1)	(0.7253, 0.8484, 0.9925)
Environment 3	(0.15, 0.0, 1.0)	(0.4369, 0.4369, 0.4369)

- *Guideline.* If the GUI of your Rack Extension is completely flat (e.g., you are re-creating a VST GUI that has no shadows), encapsulate the GUI in a display. This creates a visual separation between the parts of your device that belong to the pseudo-physical world of the Reason rack, and the parts that belong to the Rack Extension's GUI. The NN-XT sampler editor is a recommended source of inspiration.
- *Guideline.* To the extent that it is possible, the size of the device parts of a 45 (knobs, buttons, faders, lamps, etc.) should be plausible.
- *Guideline.* To the extent that it is possible, all device parts that the user can interact with should be positioned plausibly, so that they would be easy and natural to interact with, had the device existed in real life. For example, avoid positioning elements like knobs, buttons, or displays over the rack rails.
- *Guideline.* Try to keep the height of the device as small as possible, to conserve space in the rack.
- *Guideline.* Animated device parts should only be used when they are related to the sound or music. Avoid pointless, disturbing blinking lights, etc.!
- *Guideline.* When a Player device is set to off or bypass all, it should look inactive.

Introduction

The need for Rack Extensions to be future-proof and platform independent puts high demands on the Jukebox GUI system: the Rack Extension that you design today must be possible to view comfortably on more or less any future display! At the same time, Rack Extensions might be very well be possible to run on small, hand-held devices in the future, where the screen resolution is much lower than on a desktop display or a laptop. Hence, a Rack Extension GUI must be easily scalable to both higher and lower resolutions, without requiring any re-submissions from Rack Extension developers. When you have submitted your GUI, Reason Studios will ensure that it works – today and in the future.

The Jukebox SDK allows you to specify Rack Extension GUIs using hi-resolution 2D images.

The 2D-based option offers you the freedom of choosing your own GUI design toolchain: the only technical requirement is that the tools you use are able to produce 2D images (that contain the static and animated parts of the GUI) in the appropriate resolution.



Figure 1. 2D GUI resources must be hi-res to support zooming and Retina displays.

This document is the main source of 2D GUI-related information. It provides an overview of the concepts and tools needed to create 2D GUIs. It is assumed that you have read the *Jukebox Concepts* document so that you have a general idea of the various components of a Rack Extension.

GUI requirements

After you have submitted your Rack Extension for approval, Reason Studios will examine it to ensure that it fulfills a set of requirements and specifications. Some of these requirements are related to the GUI and they are described in detail in the *GUI Design Guidelines* document. The document also contains a number of useful rules-of-thumb and guidelines that will assist you in making a good-looking and usable device.

The SDK documentation also contains a checklist that you should examine closely before you submit your Rack Extension.

Note that Reason Studios reserves the right to reject a device even if it fulfills all requirements in the checklist.

Important! If you are unsure about the GUI requirements, or if you want Reason Studios feedback on a design, do not hesitate to contact developer support.

Main workflow

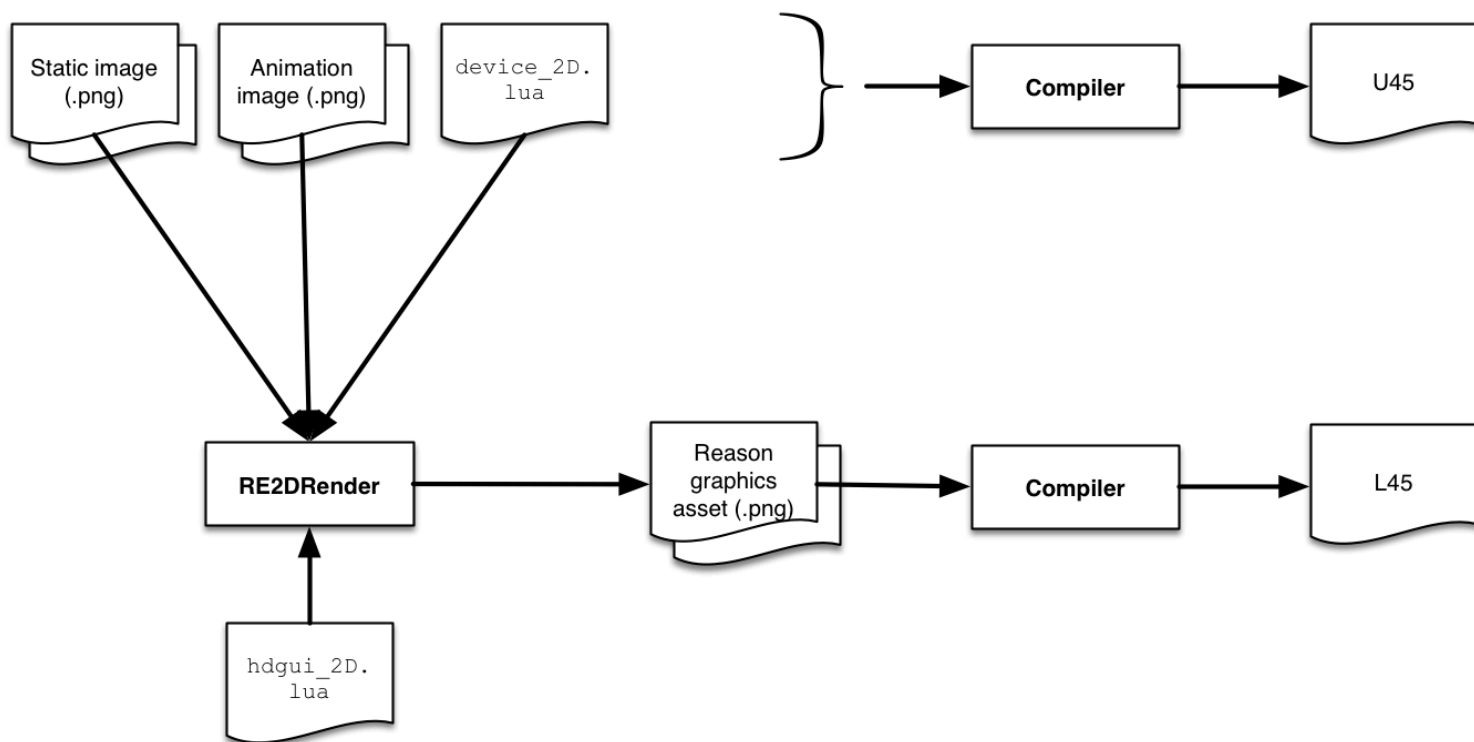


Figure 2. GUI development process overview.

[FIG2a] illustrates the main workflow when creating 2D Rack Extension GUIs for Jukebox. A 2D GUI contains only two types of resource: image files in PNG format, and a file called `device_2D.lua` that describes how the image files should be assembled.

To export a GUI for use in the local 45 (that you can load into Reason Recon), you use a tool called `RE2DRender`. The export step requires that you provide a LUA file called `hdgui_2d.lua`. This file associates the 2D images with Reason-specific widgets and properties and the properties that you have declared in `motherboard_def.lua`. When you create the universal 45 for submission, the compiler will include the 2D assets, `device_2D.lua`, and `hdgui_2D.lua` in the archive.

Basic concepts

There are two types of 2D GUI resources: *static images* and *film strips*.

Static images are primarily used for the four panel backdrops (front, back, folded front and folded back), but are also used to represent areas on the panel where Reason inserts its own graphics (e.g., the sockets where cables connect).

Film strips are used for animated GUI elements, such as knobs, faders, lamps, and buttons ([FIG3]). When the GUI is drawn in Reason, the appropriate animation frame is extracted from the film strip image and is composited onto the panel.

In both types of GUI resource, the alpha channel is used for blending/compositing.

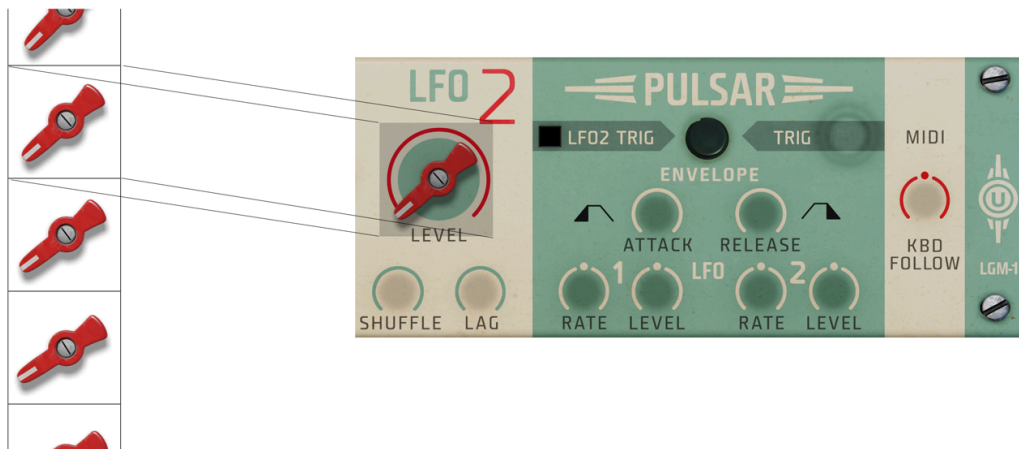


Figure 3. Film strip image resources are used for animation.

Once you have drawn the 2D image resources, you must tell Reason where on the panels they should be drawn by providing a file called `device_2D.lua`. The contents of this file is written in a scripting language called LUA, which is convenient when designing complex GUIs. You don't have to be a programmer to create this file, though: the format is quite straightforward.

Although the `device_2D.lua` file specifies the positioning of various image elements, it does not specify how the user's input (e.g., mouse clicks or drags) should be converted into parameter changes in your device. In Reason, the areas on the device panel that receives user input are called *widgets*. You specify the mapping between graphical elements and widget types in a file called `hdgui_2D.lua` (also in the LUA scripting language).

The reason why Jukebox distinguishes between GUI graphics definitions (`device_2D.lua`) and widget definitions (`hdgui_2D.lua`) is that it may become possible to run your Rack Extension on, e.g., tablets and mobile devices in the future. Such devices may require different widgets and widget behaviors, in which case it would be possible to provide device-specific versions of the `hdgui` file (where applicable), while keeping the `device_2D.lua` file as-is.

[[2D_GUI_resources]] == 2D GUI resources

The main user interface metaphor of Reason is a 19" rack containing devices. Each device (except Players) has four panels:

- Front panel
- Back panel
- Folded front panel (Players don't have this)
- Folded back panel (Players don't have this)

The height of the front and back panels are integer multiples of a *rack unit* (U). The maximum allowed height is 9U. The folded front and back panels have a fixed height. The following table lists the dimensions for the panels:

PANEL	WIDTH (PIXELS)	HEIGHT (PIXELS)
Front panel	3770	345 x N, where N is the number of rack units (1-9)
Player front panel	3590	345 x N, where N is the number of rack units (1-9)
Back panel	3770	345 x N, where N is the number of rack units (1-9)
Player back panel	3180	345 x N, where N is the number of rack units (1-9)
Folded front panel	3770	150

Folded back panel	3770	150
-------------------	------	-----

Even though the Player panels are more narrow than other devices, the images should still be 3770 pixels wide. The margins are automatically cropped in the build pipeline.

Film strips have a vertical layout. All animation frames in a film strip have the same width and height. The first frame sits at the upper left corner of the film strip image, the second frame sits immediately below the first, and so on. For example, the dimensions of a film strip image for a 64-frame animation where the frames are 100x100 pixels is 100 by 6400 pixels.

Some device elements, such as cable sockets and the patch browser buttons, are drawn by Reason. This means that you do not have control over how they look. The Jukebox SDK contains placeholder images that you can use to position such elements on the device panel.

In addition to the panel resources, it's also possible to specify custom snapshots of your device. These snapshots will be used when creating thumbmail sized images of the device, see [\[defining_custom_snapshots\]](#). The following table lists the dimensions for the snapshots:

PANEL	WIDTH (PIXELS)	HEIGHT (PIXELS)
Front snapshot	754	69 x N, where N is the number of rack units (1-9)

GUI graphics definition (device_2D.lua)

The `device_2D.lua` file is where you describe where your GUI elements should appear on the device panels: it contains *graphics definitions*.

Defining the panels

Let's take a look at a simple example! Here are the four panel images:

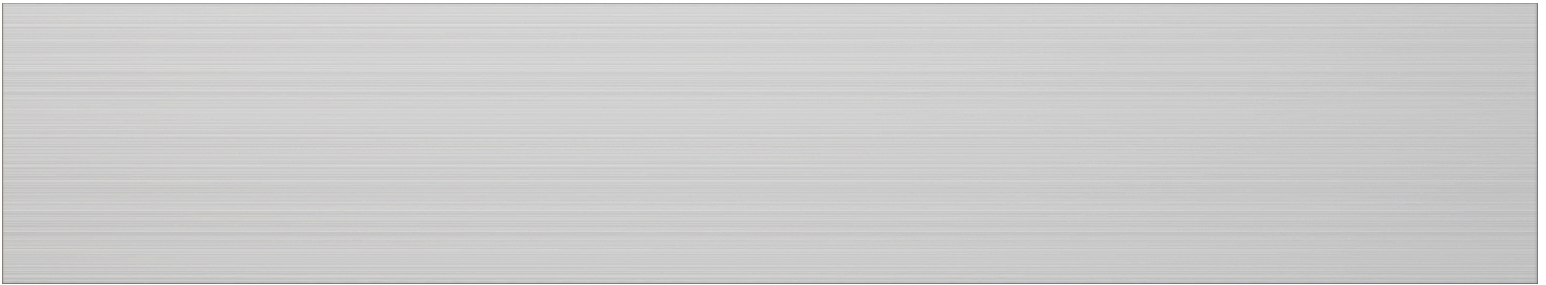


Figure 4. Front panel image (Panel_Front.png)

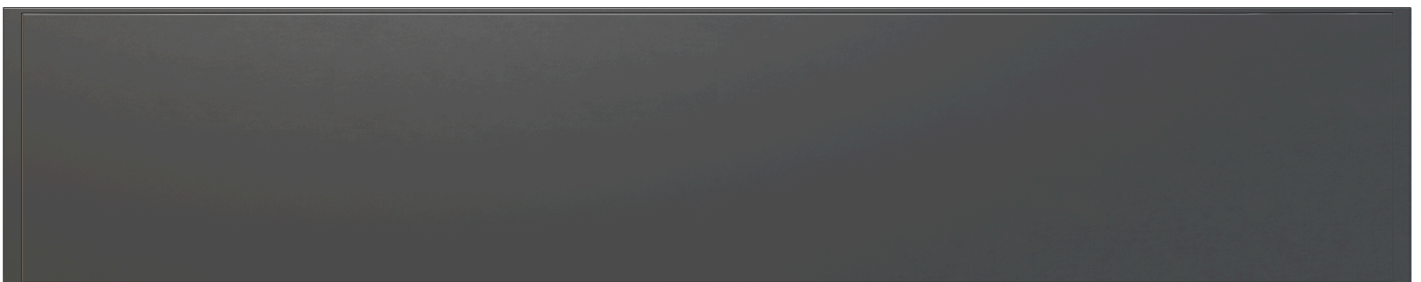


Figure 5. Back panel image (Panel_Back.png)



Figure 6. Folded front panel image (Panel_Folded_Front.png)



Figure 7. Folded back panel image (Panel_Folded_Back.png)

Note that these images are all 3770 pixels wide! The alpha channel for the two back panels has been set to zero on the right and on the left so that the rack rails can show through there when the device is drawn in Reason.

Here is how we specify these panels in `device_2D.lua`:

```
None
format_version = "2.0"

front = {
```

```

        Bg = {
            { path = "Panel_Front" },
        },
    },

    back = {
        Bg = {
            { path = "Panel_Back" },
        },
    },

    folded_front = {
        Bg = {
            { path = "Panel_Folded_Front" },
        },
    },

    folded_back = {
        Bg = {
            { path = "Panel_Folded_Back" },
        },
    },

```

The `format_version` entry tells Jukebox that this file is written in version 2.0 of the `device_2D.lua` format.

The `front`, `back`, `folded_front`, and `folded_back` entries are Lua tables. Each table contains a subtable called `Bg`, the background image for the panel. The `path` entry is the name of an image file to use (without the `.png` extension).

It's not important that we call this table `Bg` - we can call it anything

Note we want, it's just an identifier that must be unique within this panel definition. We will refer to the identifier in `hdgui_2D.lua` later.

Defining the panels for Players

If we are building a Player, we should only specify the front and back panel, since a Player does not have folded panels. Here is how we specify these panels in `device_2D.lua`:

```
None
format_version = "2.0"

panel_type = "note_player"

front = {
    Bg = {
        { path = "Panel_Front" },
    },
},

back = {
    Bg = {
        { path = "Panel_Back" },
    },
},
```

Adding widgets

Now, assume we want to position an animated knob at the center of the front panel. We have a 63-frame film strip image for the knob (only the first frame is shown here):

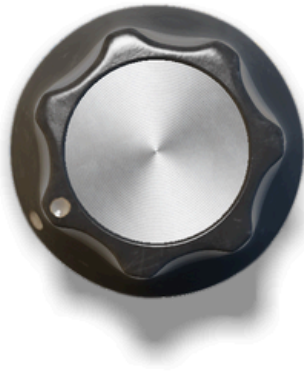


Figure 8. Knob film strip frame (Knob.png)

The front panel is 3770 x 690 pixels and each animation frame in the knob film strip is 210 x 245 pixels, which means that we want the upper left corner of the knob to be at (1780, 223). Here's how to add it to `display_2D.lua`:

```
None
format_version = "1.0"

front = {
    Bg = {
        { path = "Panel_Front" },
    },
    Knob = {
        offset = { 1780, 223 },
        { path = "Knob", frames = 63 }
    },
},

...
```

Again, it's not important that we have chosen the name `Knob` for the knob definition - it's just an identifier that we'll refer to later in `hdgui_2D.lua`.

The `offset` entry specifies where Reason should position the upper left corner of the knob animation frames. The `path` entry is the name of the film strip image file (without extension) and the `frames` entry specifies the number of animation frames in the film strip.

Required widgets

Jukebox requires that all Rack Extension GUIs have a *placeholder widget* on the back panel and a *cable origin widget* on the folded back panel.

The placeholder widget is simply a reserved empty area on the back panel where Reason Studios can insert extra GUI controls in the future, should that be necessary. The size of the placeholder must be 300 x 100 pixels. Reason will not draw the placeholder image in the GUI, so you set its color to anything you like, or you can use the placeholder image that comes with the Jukebox SDK.

The cable origin widget specifies the point on the folded back panel where Reason should attach all the incoming and outgoing cables. The cable origin is the only graphics definition that doesn't have an image: it's just a point.

Here is what our complete example looks like with the two required widgets added:

None

```
format_version = "1.0"
```

```
front = {
    Bg = {
        { path = "Panel_Front" },
    },
    Knob = {
        offset = { 1780, 223 },
        { path = "Knob", frames = 63 }
    },
},

back = {
    Bg = {
        { path = "Panel_Back" },
    },
    Placeholder = {
        offset = { 2100, 200 },
        { path = "Placeholder" }
    },
},

folded_front = {
    Bg = {
        { path = "Panel_Folded_Front" },
    },
},

folded_back = {
    Bg = {
        { path = "Panel_Folded_Back" },
    },
    CableOrigin = {
        offset = { 1885, 75 },
    },
},
```

Nested graphics definitions

Let's say that we have a group of knobs and that we want to be able to position the group as a single entity. We can do this by nesting graphics definitions, like this:

None

```
format_version = "1.0"

front = {
    ...
    {
        offset = { 500, 200 },
        Knob_1 = {
            offset = { -110, 0 },
            { path = "PatchBrowseGroup" },
        }
        Knob_2 = {
            offset = { 110, 0 },
            { path = "PatchBrowseGroup" },
        }
    },
    ...
},

...
```

Note that the identifiers of the graphics definitions (`Knob_1`, `Knob_2`) must be unique within the panel definition.

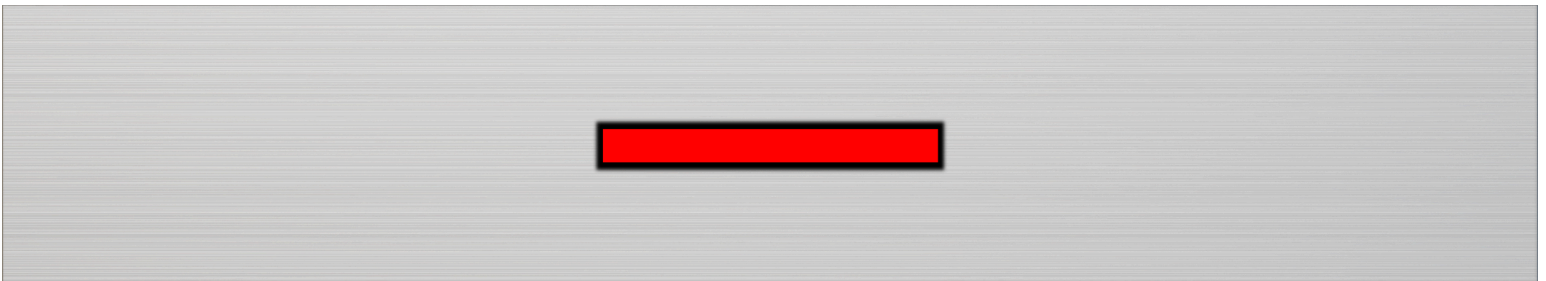
Some widgets, such as `value_display`, are drawn by Reason as text. That is, Reason only uses the graphics definition as a bounding box in which to position the text. As an example, consider the following panel with a display:



The image for the display that sits in the center of the panel looks like this:



We want to use a `value_display` widget to draw text into the display, and we want the text to appear inside the box drawn in red here:



To do this without having to copy the display image into the panel background image we can write the following in `device_2D.lua`:

```
None
format_version = "1.0"

front = {
    ...
    {
        offset = { 1446, 274 },
        { path = "Display" },
        ValueDisplay = {
            offset = { 25, 25 },
            { path = "RedBox" },
        }
    },
    ...
},
...
}
```

Note that we have two different graphics definitions: the first one, which is anonymous, will cause the image `Display.png` to be composited onto the panel at (1446, 274). The second (`ValueDisplay`) is the definition we will refer to `hdgui_2D.lua` to define the text box; it is offset slightly (25 pixels in each direction) in relation to the anonymous graphics definition. Because the `value_display` widget only uses the graphics definition to compute bounds, the red box will not be drawn in Reason.

Widget definitions (hdgui_2D.lua)

The `hdgui_2D.lua` file specifies the way the user can interact with your GUI: it contains the *widget definitions* for the four panels (front, back, folded front, and folded back).

Here is what `hdgui_2D.lua` would look like for the example in section [\[GUI_graphics_definition\]](#) (a panel with a single knob):

None

```
format_version="2.0"

front = jbox.panel{
  graphics = {
    node = "Bg",
  },
  widgets = {
    jbox.analog_knob{
      graphics = {
        node = "Knob"
      },
      value = "/custom_properties/continuous",
    },
  },
},

back = jbox.panel{
  graphics = {
    node = "Bg"
  },
  widgets = {
    jbox.placeholder{
      graphics = {
        node = "Placeholder",
      },
    },
  },
},

folded_front = jbox.panel{
  graphics = {
    node = "Bg",
  },
}
```

```

        widgets = {
        }
    }

    folded_back = jbox.panel{
        graphics = {
            node = "Bg",
        },
        cable_origin = {
            node = "CableOrigin",
        },
        widgets = {
        },
    },

```

The `format_version` field of `hdgui_2D.lua` should be set to "2.0" for this SDK version.

The `front`, `back`, `folded_front`, and `folded_back` entries are Lua tables. Each table contains a `graphics` subtable that specifies which graphics definition to use for the panel background. The `node` entry is an identifier that you have declared in `device_2D.lua`.

The `widgets` table lists the widgets for the panel. In this example, the front panel has a single `jbox.analog_knob` widget. The `value` entry specifies which of the motherboard properties the widget should control.

Note the `jbox.placeholder` entry on the back panel and the `cable_origin` entry on the folded back panel! These widgets are required by Jukebox. The placeholder is positioned just like any other widget (but doesn't have a motherboard property associated with it), whereas the cable origin is part of the back panel definition.

You can find the complete list of widgets in the *Jukebox Scripting Specification* document.

Hideable widgets

Hideable widgets are a way for Rack Extension designers to define a GUI with interchangeable elements such as Thor's filters. An important thing to remember is that when placing widgets that only get their bounds from the `graphics' node` value, is that it won't do to have an anonymous graphics definition for the background image. In order to get things right a `static_decoration` widget can be used and shown/hidden together with the widget. Here z-order becomes important and in `hdgui_2D.lua` widgets get an implicit z-index from the order in which they are defined, so the `static_decoration` widget must be defined first in order to appear behind the other widget.

Defining custom snapshots

Defining custom snapshots of your device is optional and *nota* requirement.

`RE2DRender` generates thumbnail sized snapshots of your device which will be used inside Reason in the device palette, the rack navigator, and the track navigator. If you are not happy with the automatically generated snapshots, it's possible to provide `RE2DRender` with custom snapshots of your device.

Two snapshots are required for regular devices: front and folded front. Folded front is not needed for Players. `RE2DRender` will look in the `GUI2D` folder for the files named:

- `ThumbSnapshotFront.png`
- `ThumbSnapshotFoldedFront.png` (Not needed for players)

If these files exist and have the correct sizes (see [\[TAB2\]](#)), `RE2DRender` will use them when generating the thumbnail images.

The images provided should be snapshots of a realistic state of the device. The recommended way to generate custom snapshots is to:

- load your device in Recon or Reason,
- configure the device so that it has the desired state/look,
- take screenshots in both folded and unfolded state,
- crop the screenshots according to [\[TAB2\]](#),
- name the cropped screenshots `ThumbSnapshotFront.png` and `ThumbSnapshotFoldedFront.png` and move them to the `GUI2D` folder.

Exporting to Reason

As described above, a 2D Rack Extension GUI consists of

- 2D GUI graphics resources (i.e., hi-res static and film strip images)
- A `device_2D.lua` file that contains graphics definitions
- A `hdgui_2D.lua` file that contains widget definitions

These files are included in the universal 45 that you submit to Reason Studios.

Reason Recon, however, currently requires the GUI to be in a slightly different format - the image resources must be resized, and the GUI metadata (such as the widget definitions) have to be tweaked slightly. The Jukebox SDK contains a tool called RE2DRender that does this job for you.

RE2DRender is a command line tool that takes two directory paths as arguments - input and output directory, and optional extra argument hi-res or hi-res-only. The hi-res argument will make it render the GUI into both the hi-res form used by Reason / Recon 12 and the previous lo-res form. The hi-res-only argument will skip the lo-res form as a way to save time while developing and testing the GUI. If RE2DRender is run without hi-res or hi-res-only argument, it will only render the lo-res form of the GUI (like earlier versions of the tool).

Here is how you use it:

First, you must place the GUI files in the correct folder. Create a folder called `GUI2D` in your Rack Extension project folder and put all your GUI resources (hi-res static and film strip images, `device_2D.lua`, and `hdgui_2D.lua`) there.

Next, create a `GUI` folder in your Rack Extension project folder. This is where the output of `RE2DRender` will be placed.

You are now ready to run `RE2DRender`!

Open a Terminal (Mac) or Command Prompt (Windows) and use the `cd` command to change the current directory to the folder that contains `RE2DRender`.

Run `RE2DRender`: the first argument is the absolute path to your `GUI2D` folder and the second argument is the absolute path to your `GUI` folder. The third argument should be `hi-res` or `hi-res-only` if you want to test the GUI in hi-res (Reason / Recon 12 and later).

Mac example (launch .app):

```
cd /Users/jengstrom/JukeboxSDK/RE2DRender
open RE2DRender --args /Users/jengstrom/REdev/MyRE/GUI2D /Users/jengstrom/REdev/MyRE/GUI
hi-res
```

Mac example (run as command line tool):

```
cd /Users/jengstrom/JukeboxSDK/RE2DRender  
RE2DRender.app/Contents/MacOS/RE2DRender /Users/jengstrom/REdev/MyRE/GUI2D  
/Users/jengstrom/REdev/MyRE/GUI hi-res
```

Windows example:

```
cd C:\Users\jengstrom\JukeboxSDK\RE2DRender  
RE2DRender C:\Users\jengstrom\REdev\MyRE\GUI2D C:\Users\jengstrom\REdev\MyRE\GUI hi-res
```

If everything went well, you can now compile the local 45 by invoking the `build.py` script or by compiling it from within XCode or Visual Studio. See the *Jukebox SDK Readme* document for more information.